

UCLouvain

LINFO1121 2026-2027

Algorithmique et Structure de données

Exercice 3: Search Trees

TA LLN: Harold Kiossou, Alice Burlats, Achille Morenville (Competition) + Tutors
TA Charleroi: Roger Kameugne

Prof pierre.schaus@uclouvain.be

Question 3.1.1 Sequential vs BINARY SEARCH

Which of the two SequentialSearchST or BinarySearchST implementations would you use for an application? which performs 10^3 put() and 10^6 get() in random order? Justify.

	Sequential	Binary
Insertion	O(1)	O(n) O(n/2) avg
Search	O(n)	O(log n)

n : number of element in the array

Question 3.1.1 Sequential vs BINARY SEARCH

seq =

bin =

	Sequential (LinkedList)	Binary (Array)
Insertion	O(1)	O(n) O(n/2) avg
Search	O(n)	O(log n)

P : Number of put

G : Number of get

Question 3.1.1 Sequential vs BINARY SEARCH

Costs linked to puts

$$\text{seq} = (\sum_{i=1}^{P} 1)$$

$$\text{bin} = (\sum_{i=1}^{P} i)$$

	Sequential (LinkedList)	Binary (Array)
Insertion	O(1)	O(n) O(n/2) avg
Search	O(n)	O(log n)

P : Number of put

G : Number of get

Question 3.1.1 Sequential vs BINARY SEARCH

Costs linked to puts

+ costs linked to gets

$$\text{seq} = (\sum_{i=1}^{P} 1) + GP$$

$$\text{bin} = (\sum_{i=1}^{P} i) + G \log P$$

	Sequential (LinkedList)	Binary (Array)
Insertion	O(1)	O(n) O(n/2) avg
Search	O(n)	O(log n)

P : Number of put

G : Number of get

Question 3.1.1 Sequential vs BINARY SEARCH

Simplify the sum

	Sequential	Binary
Insertion	O(1) amorti	O(n) O(n/2) avg
Search	O(n)	O(log n)

$$\text{seq} = P + GP$$

$$\text{bin} = \frac{1}{2}P(P+1) + G\log P$$

P : Number of put

G : Number of get

Question 3.1.1 Sequential vs BINARY SEARCH

$$\text{seq} = P + GP \qquad \text{bin} = \frac{1}{2}P(P+1) + G\log P$$

Condition for bin < seq

$$\frac{P(P+1)}{2} + G\log P < P + PG$$

$$G > \frac{P^2 - P}{2(P - \log P)} \sim \frac{P}{2}$$

$$10^6 > \frac{10^3}{2} \rightarrow \text{bin has a lower total cost}$$

3.1.2 Interpolation Search

- Reminder: If k_x is the key sought, k_{lo} is the first key in the table, and k_{hi} is the last key in the table, look first $\lfloor (k_x - k_{lo})/(k_{hi} - k_{lo}) \rfloor$-way through the table rather than halfway.

3.1.2 Interpolation Search

1	3	6	10	15	21	27	34	42	51	61

Let’s search for 51 !

With a classical **Binary Search**, we would check the median

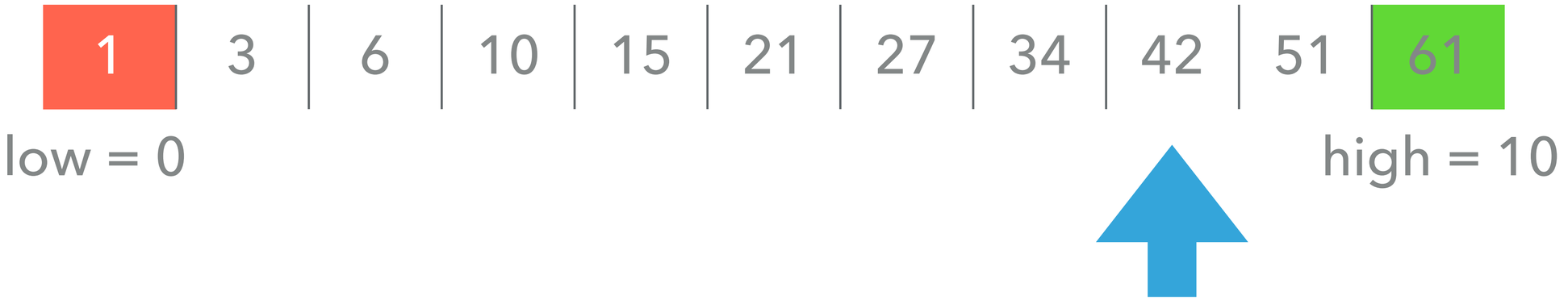
Here, we check on index

$$low + \lfloor (high - low) \cdot (value - keys[low]) / (keys[high] - keys[low]) \rfloor$$

3.1.2 Interpolation Search



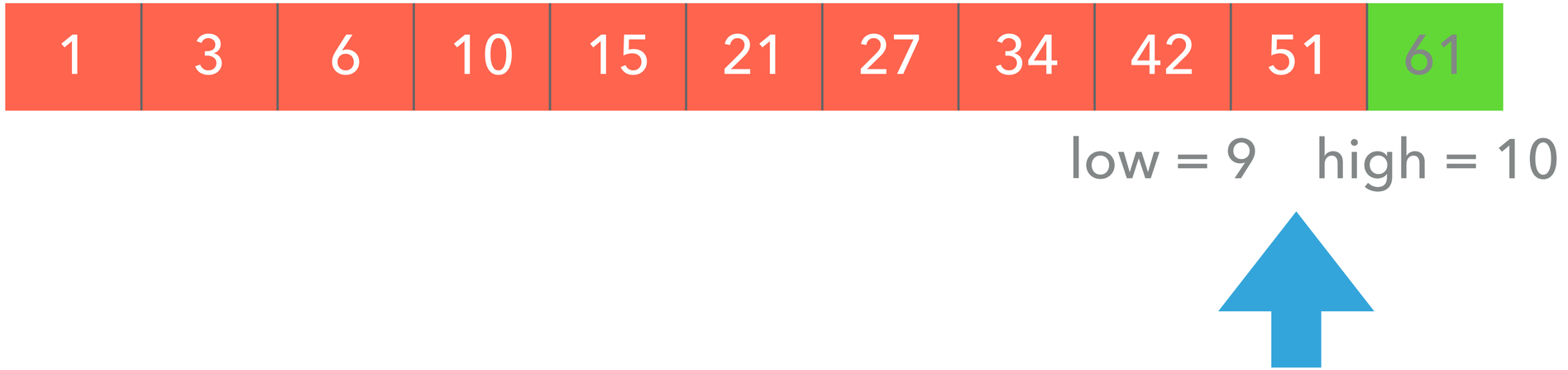
3.1.2 Interpolation Search



$$\text{index} = 0 + \lfloor (10 - 0) \cdot (51 - 1)/(61 - 1) \rfloor = 8$$

$$51 > 42 \rightarrow \text{lo} = \text{index+1}$$

3.1.2 Interpolation Search



$$\text{index} = 9 + \lfloor (10 - 9) \cdot (51 - 51)/(61 - 1) \rfloor = 9$$

We found 51 !

3.1.2 Interpolation Search

```
public int rank(Key key) {
        int low = 0
        int high = keys.length - 1;
        while (low <= high) {
            int index = low + floor((high - low)(key - keys[lo])/( keys[high] - keys[low]));
            if        (keys[index] < key) high = index - 1;
            else if (keys[index] > key) low = index + 1;
            else return mid;
        }
        return lo;
    }
```

BST Reminder

Binary Search Tree properties

It's a tree:

- It's a graph
- Connected
- With n-1 edges for n nodes
- No cycles
- At most one path between each pair of nodes

It's binary:

- At most two children for each nodes
- The **left** subtree must contain only **smaller** keys
- The **right** subtree must contain only **larger** keys

BST Reminder

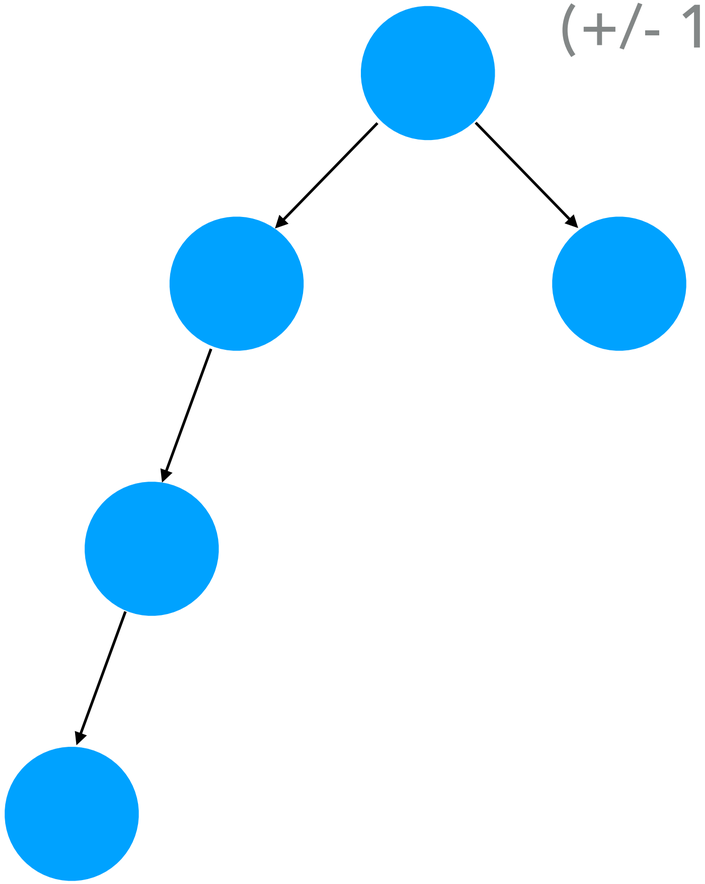
- A tree is **balanced** if the size of every path between the root and any leaf is the same (+/- 1 at most).

- A binary tree is **perfectly balanced** if the size of these paths is $log_2(n) \pm 1$.

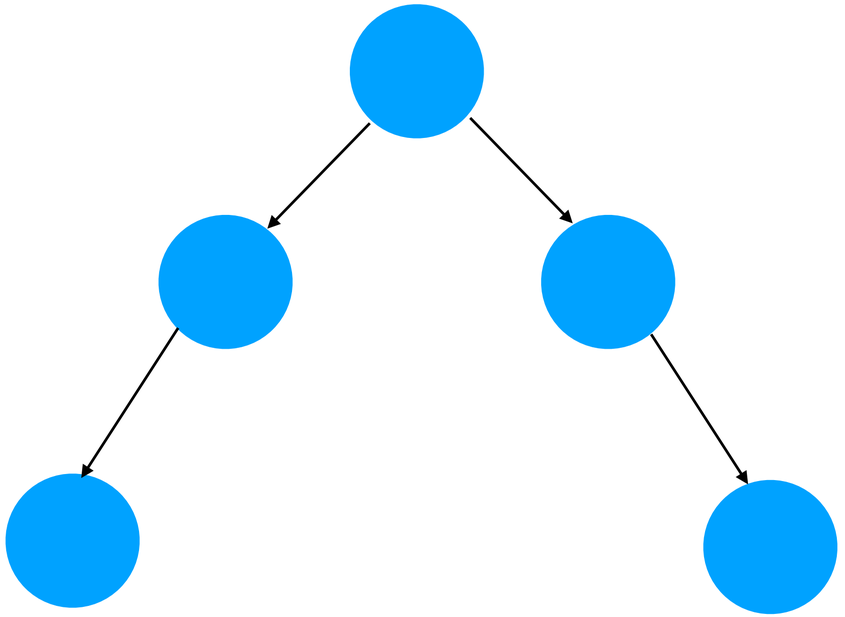
- This corresponds to using all the children of all nodes: all levels except the last one are filled.

Balanced Search Tree

Balanced : all leafs are at the same depth
(+/- 1 at most)

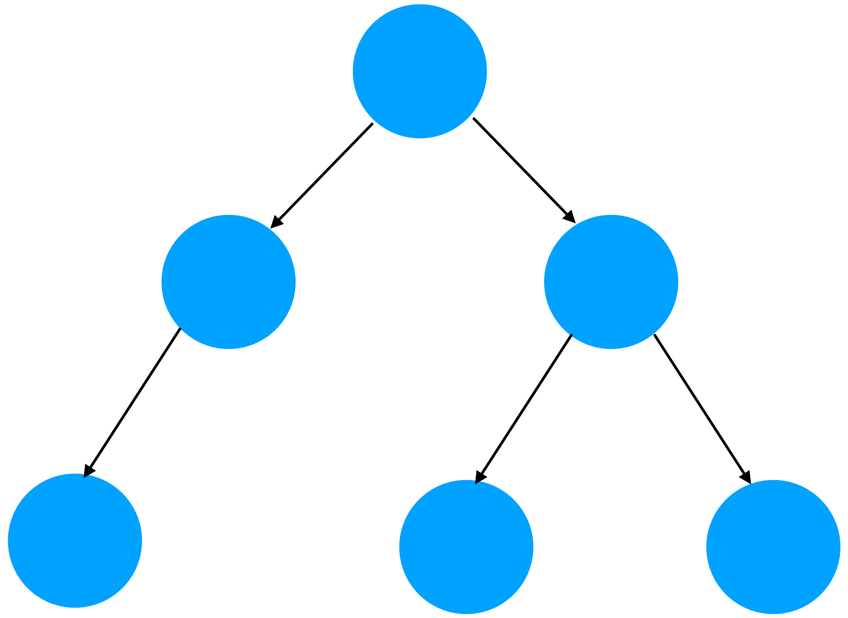


Unbalanced



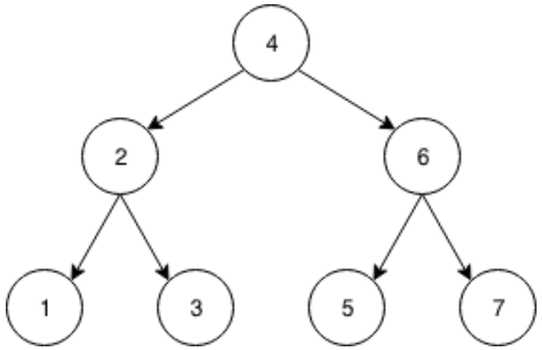
Balanced

Perfectly Balanced Search Tree

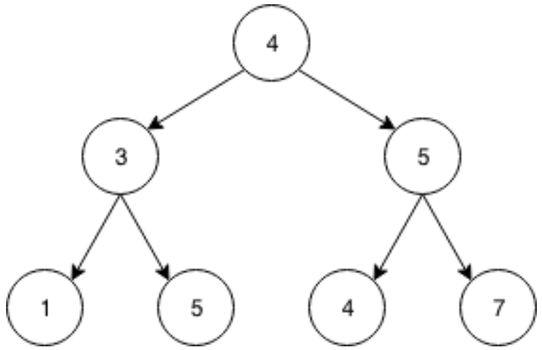


Perfectly balanced if the length of its paths is $log_2(n) \pm 1$.

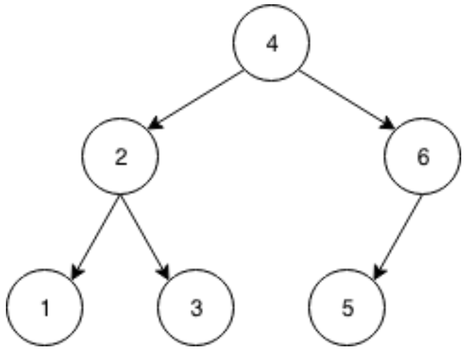
Question 3.1.5 isBST()



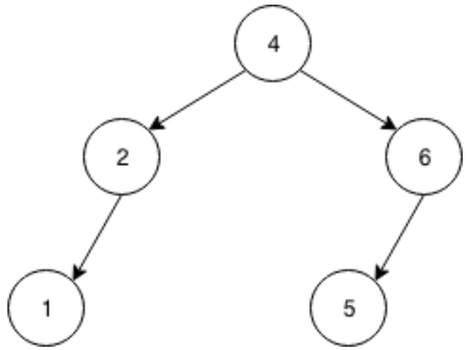
A



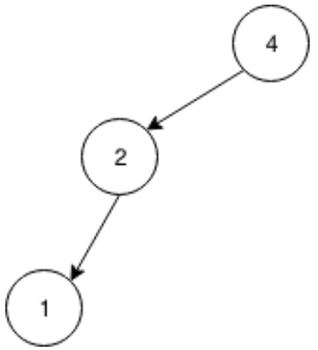
B



C



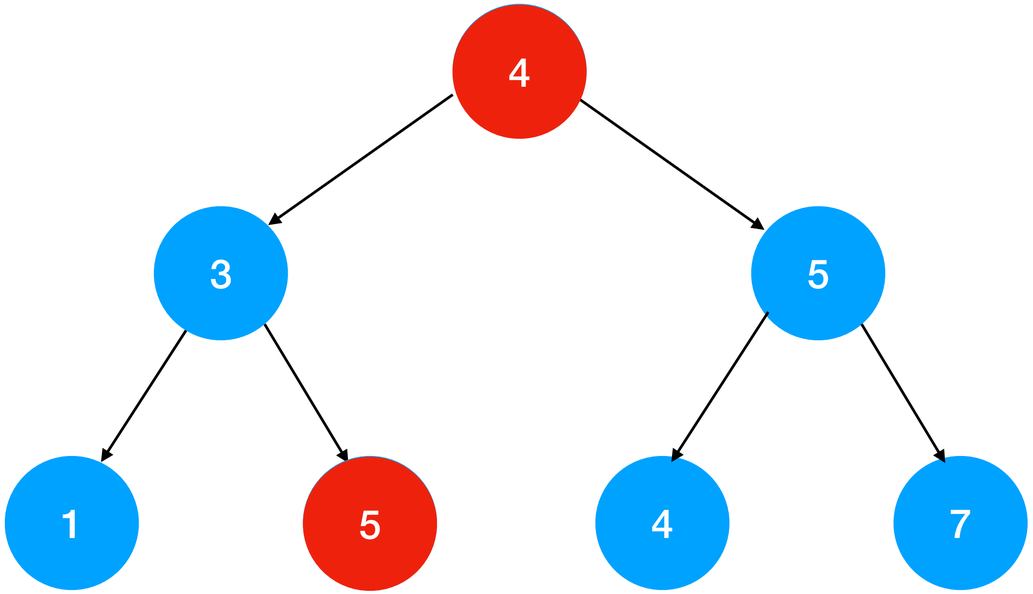
D



E

Question 3.1.5 isBST()

```
private boole    sBST(Node x)
    if (x == nu      rn true;
    // Does the le      h      maller key ?
    if (x.key.compar          ) <= 0) return false;
    // Does the right      a larger key ?
    if (x.key.com          (x          >= 0) return false;
    return isB          ht) &&          .right);
}
```



What does the algorithm return?

Is it a BST ?

Question 3.1.5 isBST()

```
private boolean isBST() {
   return isBST(root, null, null);
}

private boolean isBST(Node x, Key min, Key max) {
   if (x == null) return true;
   if (min != null && x.key.compareTo(min) <= 0) return false;
   if (max != null && x.key.compareTo(max) >= 0) return false;
   return isBST(x.left, min, x.key) && isBST(x.right, x.key, max);
}
```

Add upper and lower bounds



3.1.3 *Caching* key

```
public class BinarySearchST <Key extends Comparable<Key>, Value> {
   private Key[] keys;
   private Value[] vals;
   private int N;
   private Key cacheKey;
   private int cacheRank;
```

```
public int rank(Key key) {
   if (key.compareTo(cacheKey) == 0)
      return cacheRank;

   int lo = 0, hi = N-1;
   while (lo <= hi) {
      int mid = lo + (hi - lo) / 2;
      int cmp = key.compareTo(keys[mid]);
      if      (cmp < 0) hi = mid - 1;
      else if (cmp > 0) lo = mid + 1;
      else {
         cacheKey = key;
         cacheRank = mid;
         return mid;
      }
   }
   return lo;
}
```

Question 3.1.6 Possible Visits

Suppose a certain search tree has keys between 1 and 10.

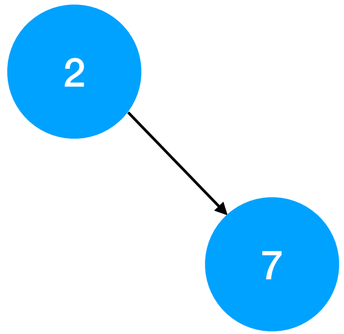
We are looking for key 5.

Which sequence(s) cannot match the sequence of the examined keys?

- 10,9,8,7,6,5
- 4,10,8,6,5
- 1,10,2,9,3,8,4,7,6,5
- 2,7,3,8,4,5
- 1,2,10,4,8,5

Question 3.1.5 Possible visits

2,7,3,8,4,5



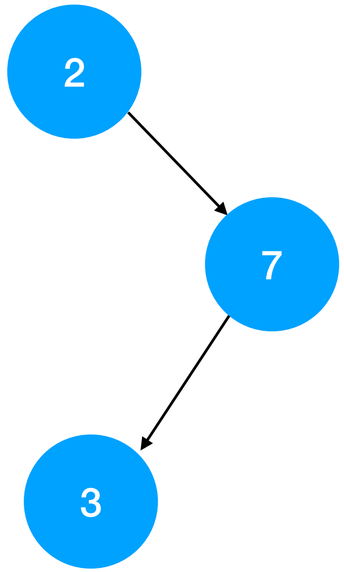
$LB = -\infty$

$UB = +\infty$

$7 > 2 \rightarrow LB = 2$

Question 3.1.6 VISITE POSSIBLES

2,7,3,8,4,5



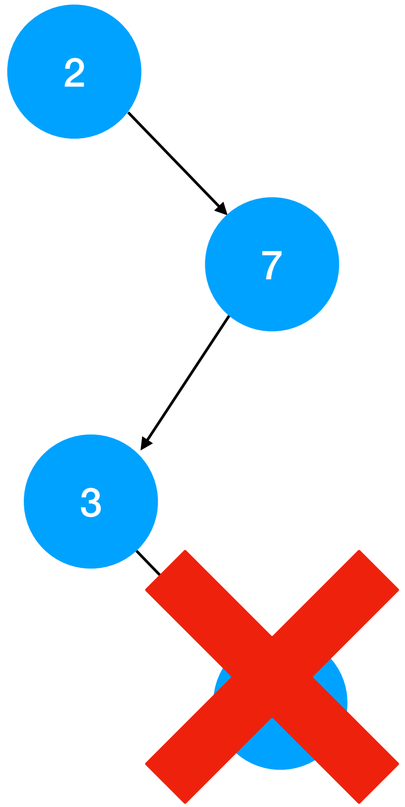
$LB = 2$

$UB = +\infty$

$3 < 7 \rightarrow UB = 7$

Question 3.1.6 VISITE POSSIBLES

2,7,3,8,4,5



LB = 2
UB = 7

8 > 7 → 8 > UB

Question 3.1.6 VISITE POSSIBLES

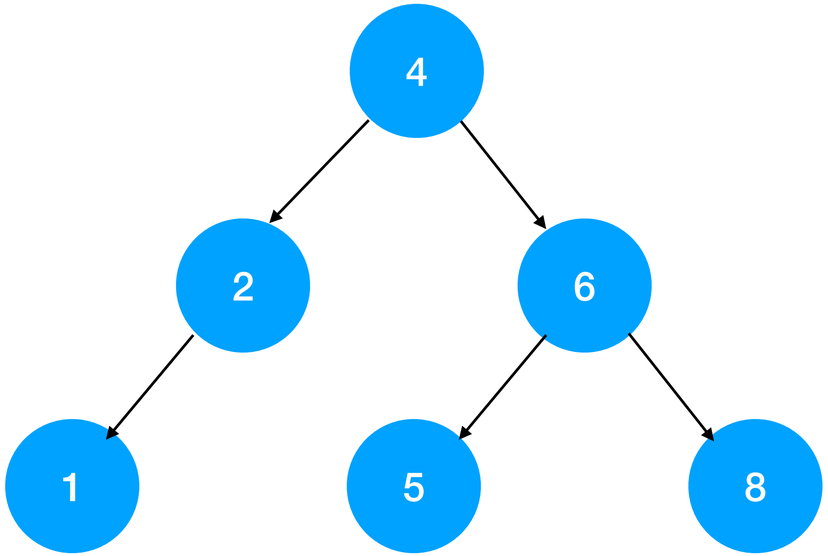
- 10,9,8,7,6,5 ✓
- 4,10,8,6,5 ✓
- 1,10,2,9,3,8,4,7,6,5 ✓
- 2,7,3,8,4,5 ✗
- 1,2,10,4,8,5 ✓

Question 3.1.7 Enumerate keys in increasing order

In-order traversal

```
public static void enumerate(Node parent) {
    enumerate(parent.leftChild)
    System.out.println(parent.key)
    enumerate(parent.rightChild)
}
```

Complexity : O(number of nodes)



Question 3.1.8 insertions in BST and 2-3 trees

Starting from an initially empty binary search tree, how does the tree look like after inserting the keys **12, 5, 10, 3, 13, 14, 15, 17, 18, 15**?

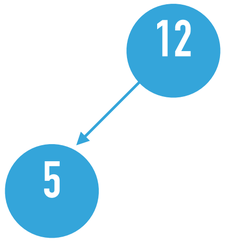
For the same data how would the tree look like for a 2-3 tree?

Question 3.1.8 insertions



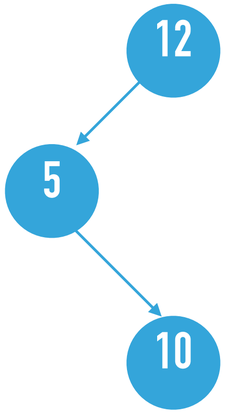
~~12~~, 5, 10, 3, 13, 14, 15, 17, 18, 15

Question 3.1.8 insertions



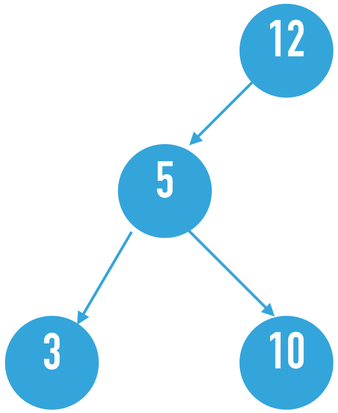
~~12, 5~~, 10, 3, 13, 14, 15, 17, 18, 15

Question 3.1.8 insertions



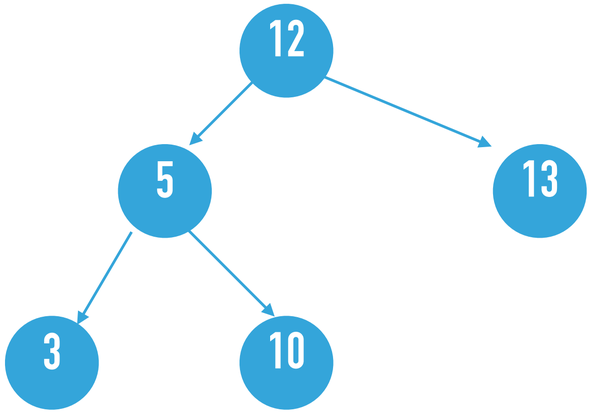
~~12, 5, 10~~, 3, 13, 14, 15, 17, 18, 15

Question 3.1.8 insertions



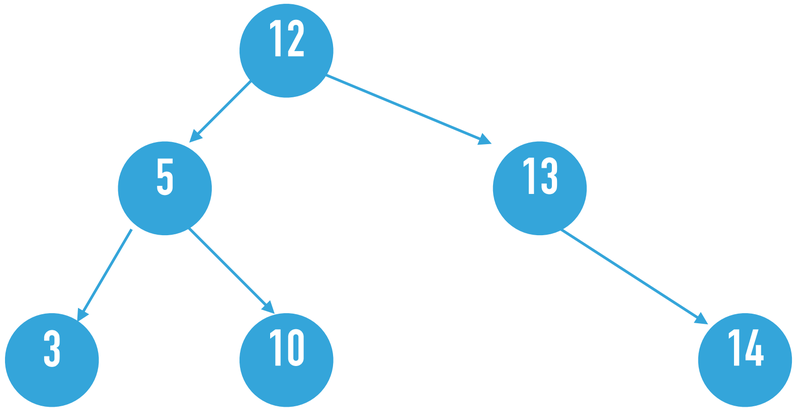
~~12, 5, 10, 3~~, 13, 14, 15, 17, 18, 15

Question 3.1.8 insertions



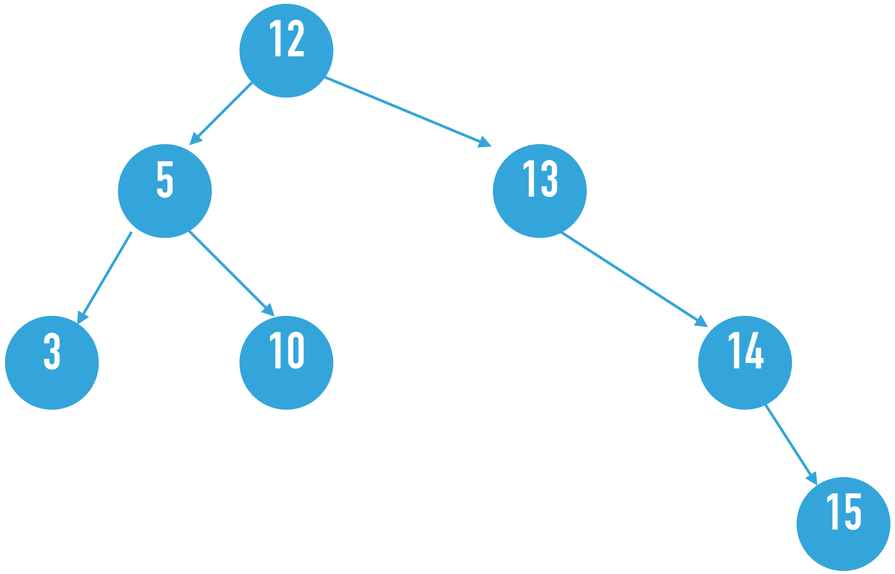
~~12, 5, 10, 3, 13~~, 14, 15, 17, 18, 15

Question 3.1.8 insertions



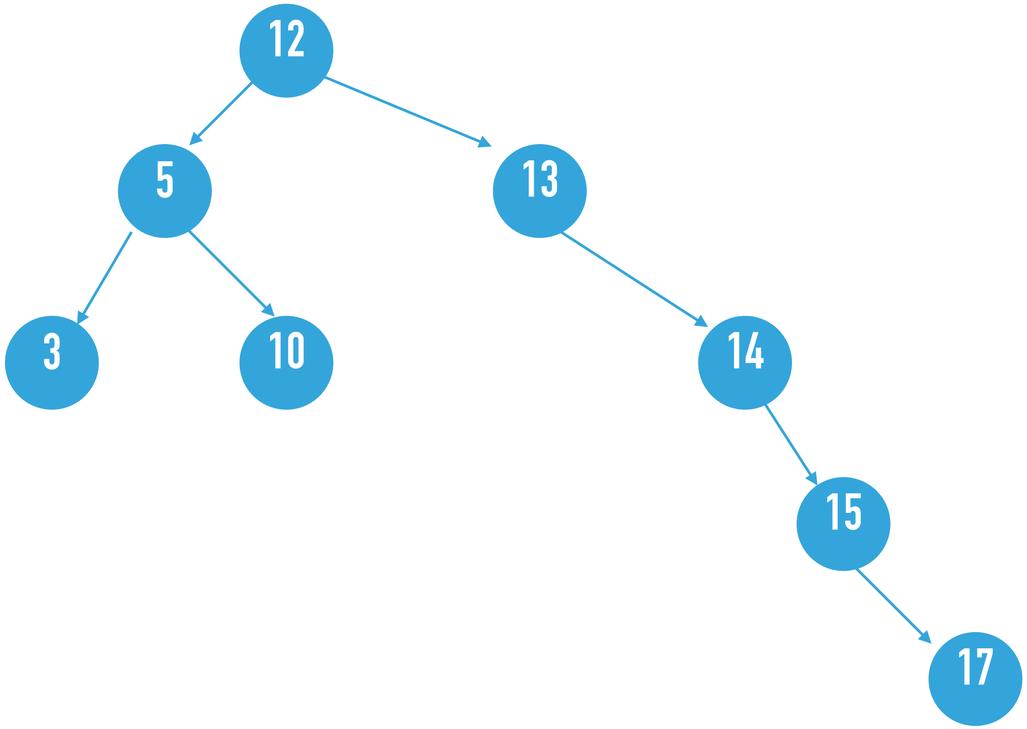
~~12, 5, 10, 3, 13, 14~~, 15, 17, 18, 15

Question 3.1.8 insertions



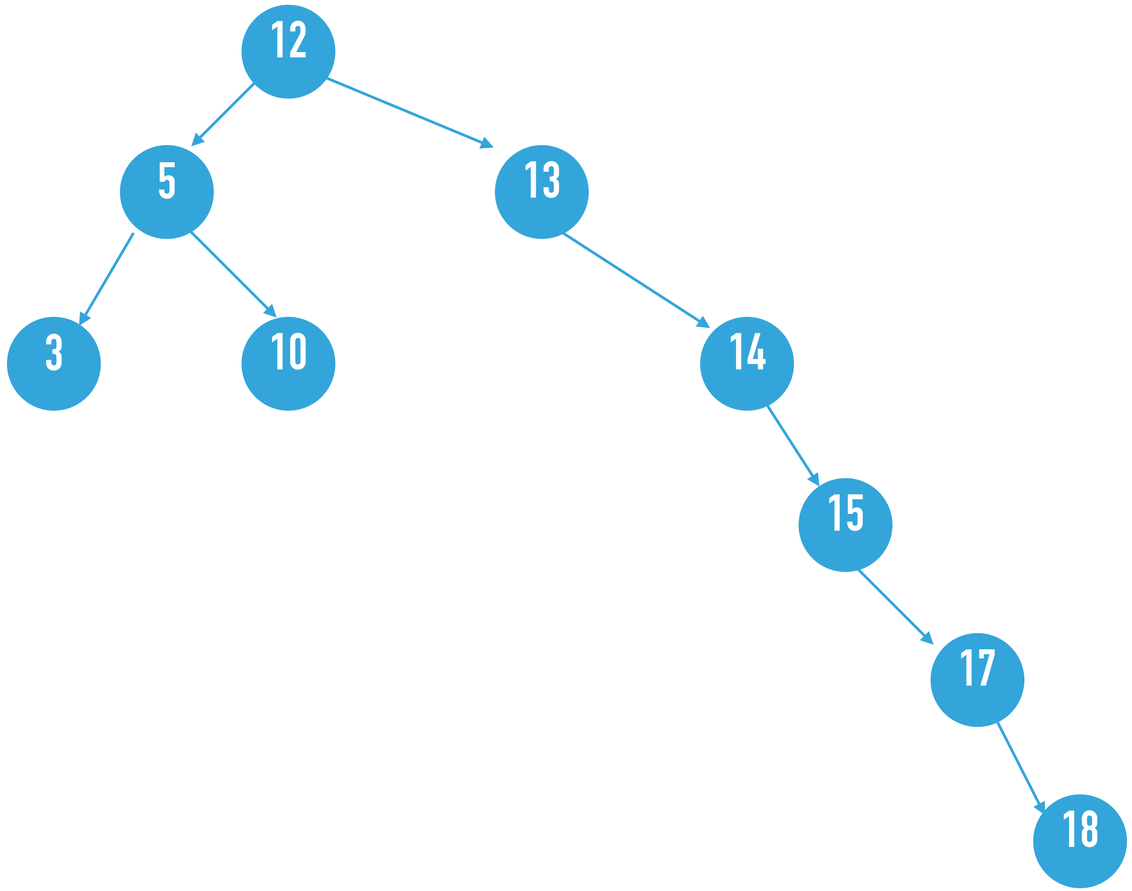
~~12, 5, 10, 3, 13, 14, 15~~, 17, 18, 15

Question 3.1.8 insertions



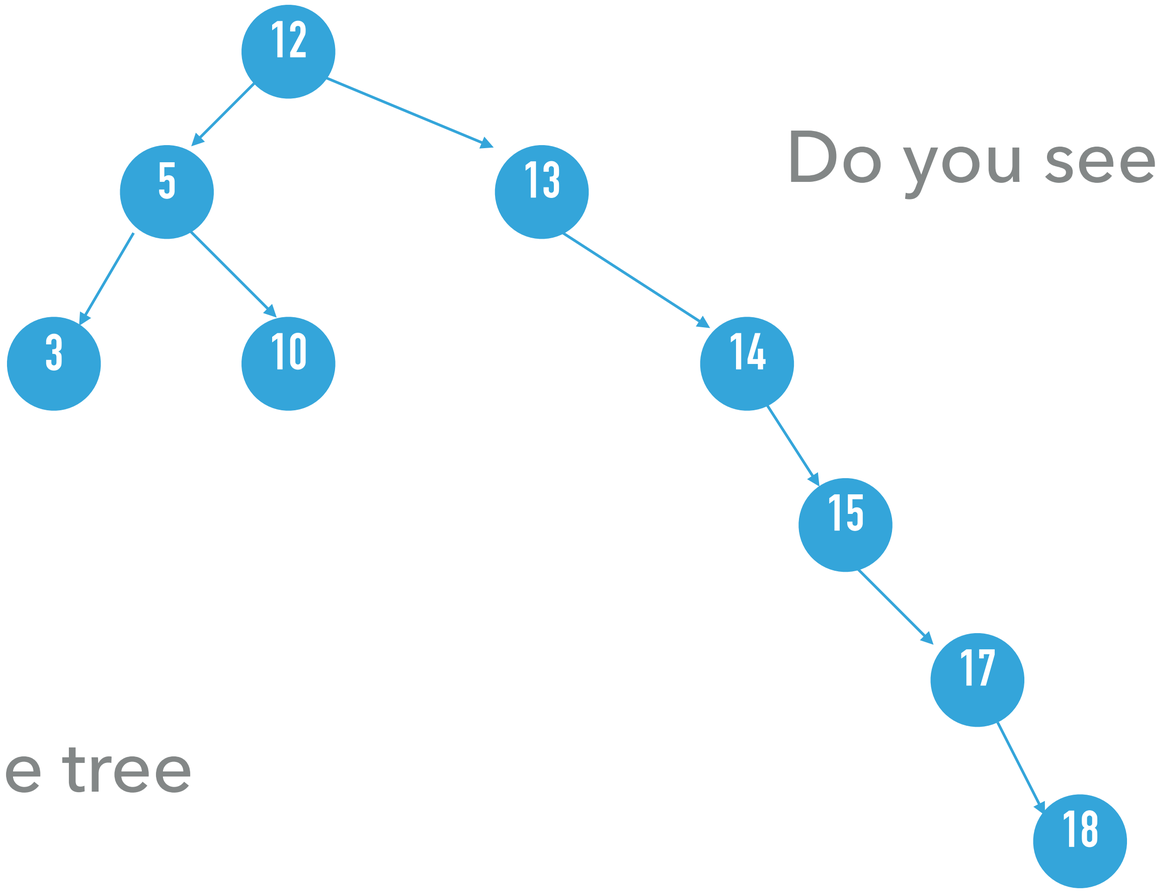
~~12, 5, 10, 3, 13, 14, 15, 17~~, 18, 15

Question 3.1.8 insertions



~~12, 5, 10, 3, 13, 14, 15, 17, 18~~, 15

Question 3.1.8 insertions



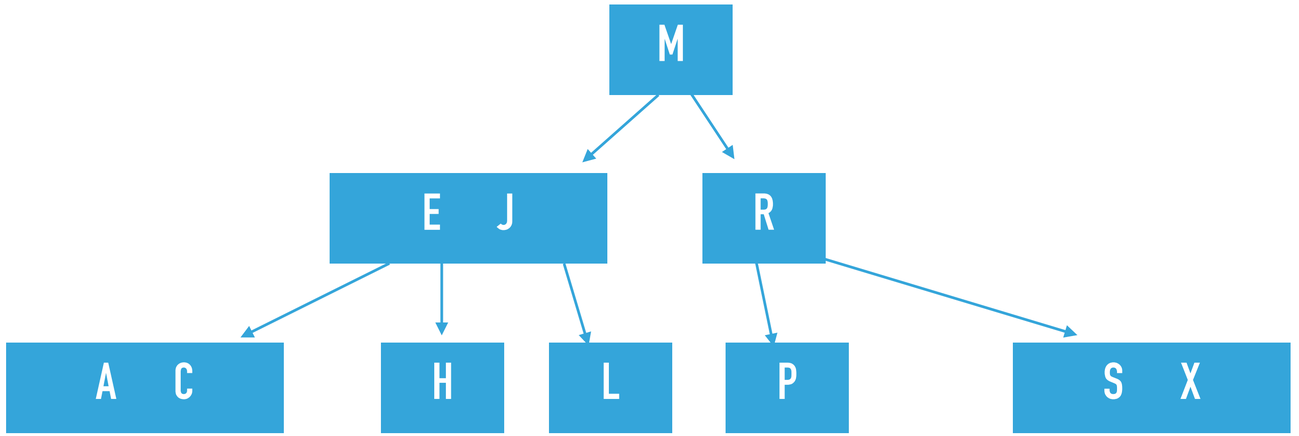
Do you see a problem ?

15 is already in the tree

~~12, 5, 10, 3, 13, 14, 15, 17, 18, 15~~

Arbres 2-3

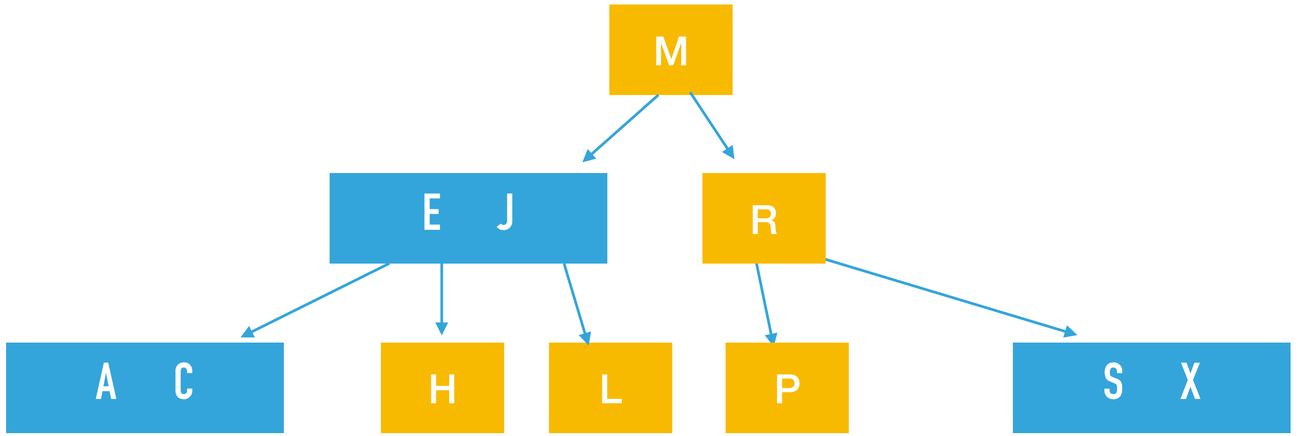
2-3 tree contains either :



Arbres 2-3

2-3 tree contains either :

- 2-nodes, with 1 key and two links
  - On the left : smaller keys
  - On the right : larger keys

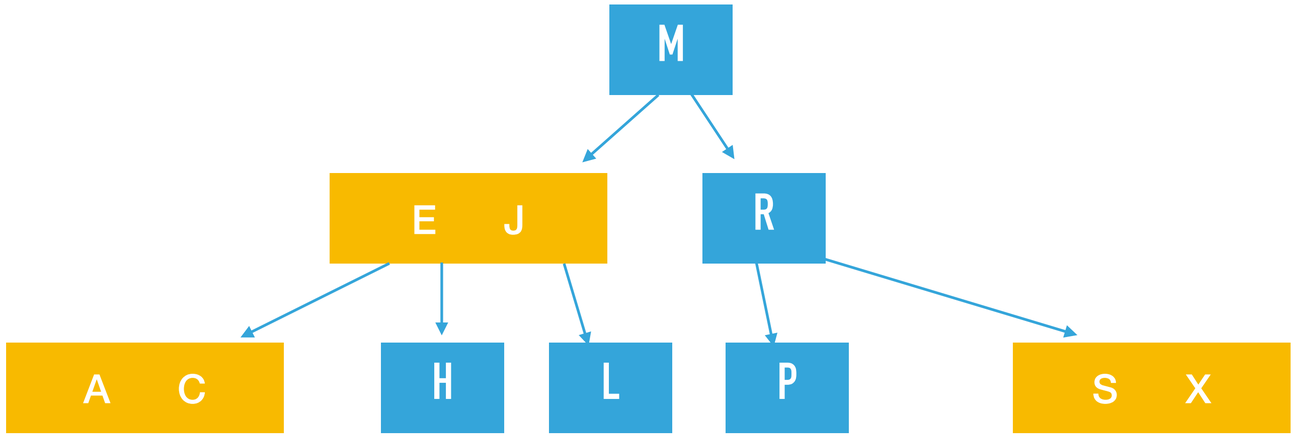


Arbres 2-3

2-3 tree contains either :

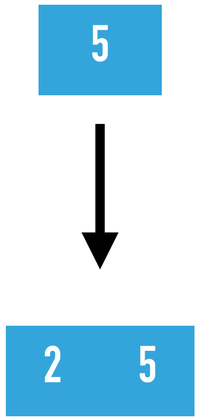
- 2-nodes, with 1 key and two links
  - On the left : smaller keys
  - On the right : larger keys

- 3-nodes, with 2 keys and 3 links
  - On the left : smaller keys
  - On the right : larger keys
  - In the middle : in-between keys

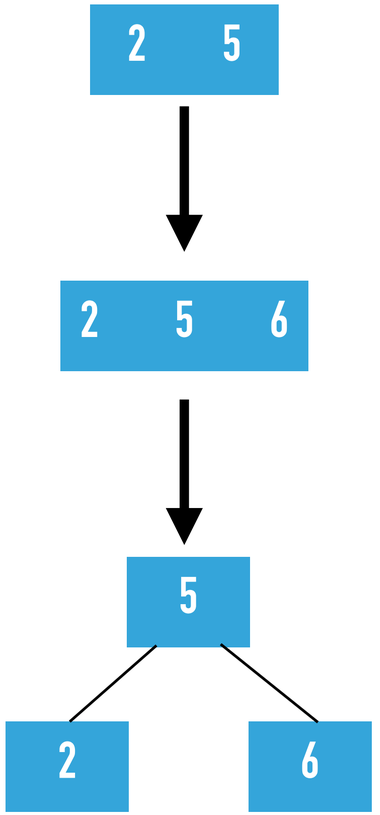


Insertions dans un arbre 2-3

Insertion in a 2-node



Insertion in a 3-node



Question 3.1.9 insertions



~~12~~, 5, 10, 3, 13, 14, 15, 17, 18, 15

Question 3.1.9 insertions



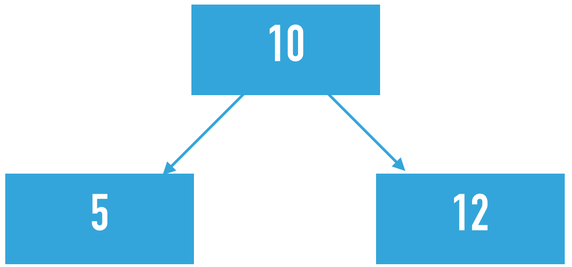
~~12~~, ~~5~~, 10, 3, 13, 14, 15, 17, 18, 15

Question 3.1.9 insertions



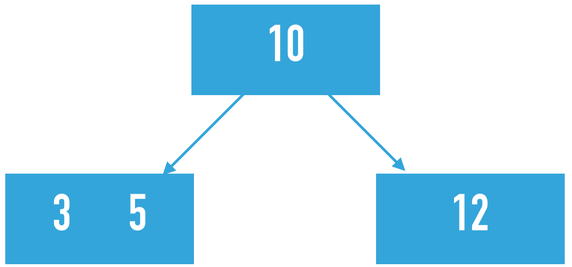
~~12, 5, 10~~, 3, 13, 14, 15, 17, 18, 15

Question 3.1.9 insertions



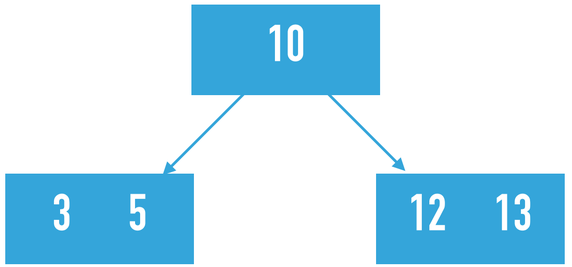
~~12, 5, 10~~, 3, 13, 14, 15, 17, 18, 15

Question 3.1.9 insertions



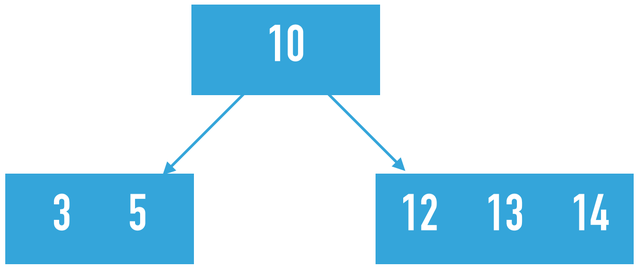
~~12, 5, 10, 3~~, 13, 14, 15, 17, 18, 15

Question 3.1.9 insertions



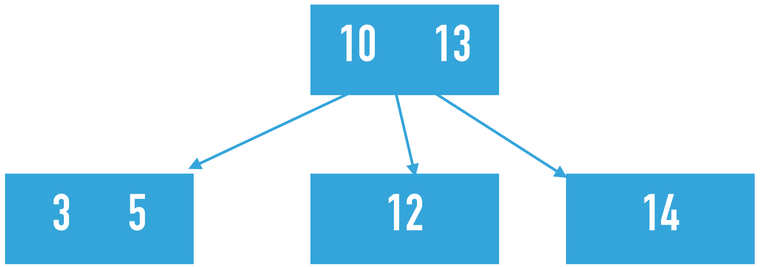
~~12, 5, 10, 3, 13~~, 14, 15, 17, 18, 15

Question 3.1.9 insertions



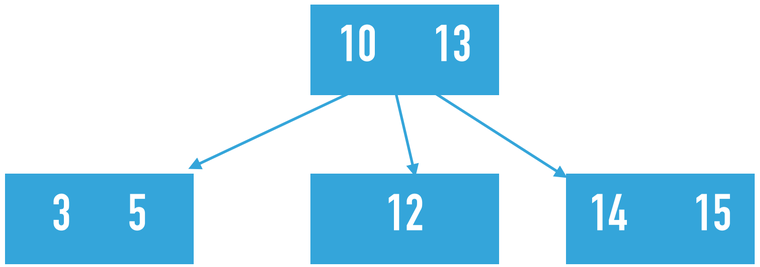
~~12, 5, 10, 3, 13, 14~~, 15, 17, 18, 15

Question 3.1.9 insertions



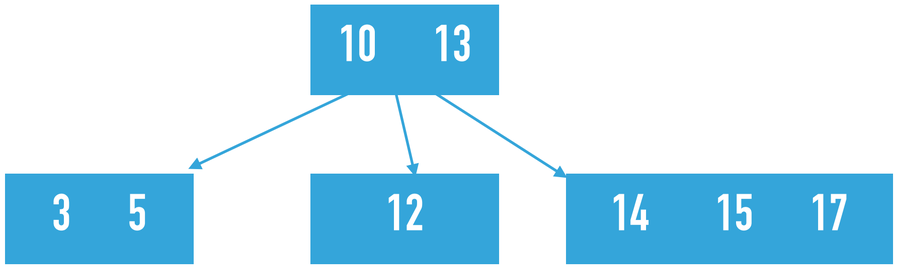
~~12, 5, 10, 3, 13, 14~~, 15, 17, 18, 15

Question 3.1.9 insertions



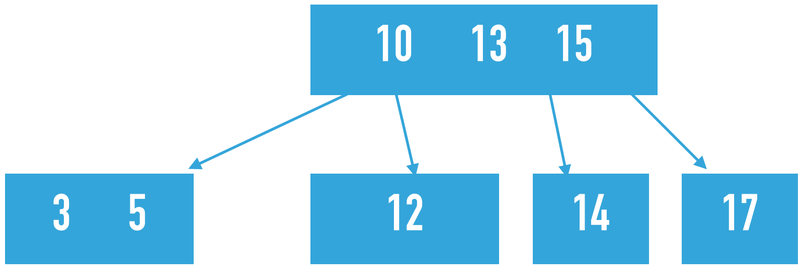
~~12, 5, 10, 3, 13, 14, 15~~, 17, 18, 15

Question 3.1.9 insertions



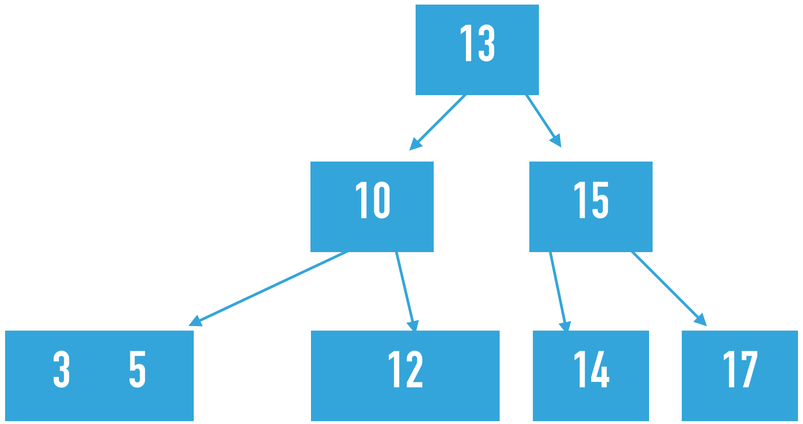
~~12, 5, 10, 3, 13, 14, 15, 17~~, 18, 15

Question 3.1.9 insertions



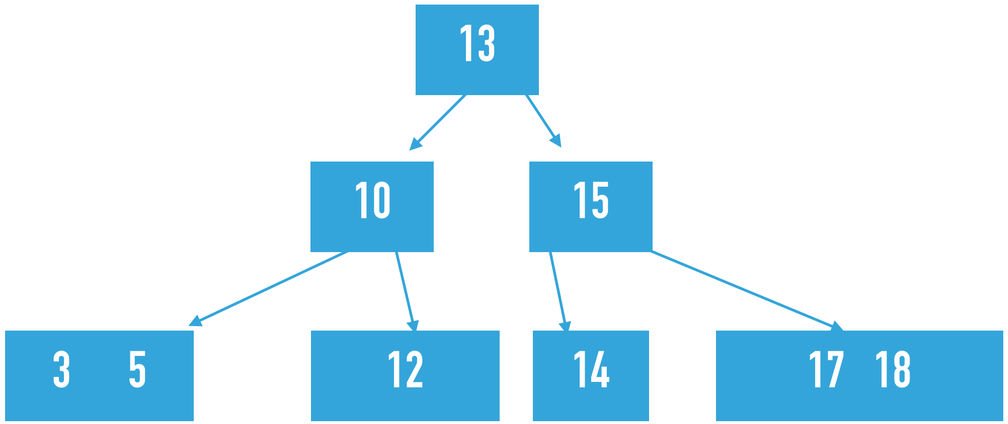
~~12, 5, 10, 3, 13, 14, 15, 17~~, 18, 15

Question 3.1.9 insertions



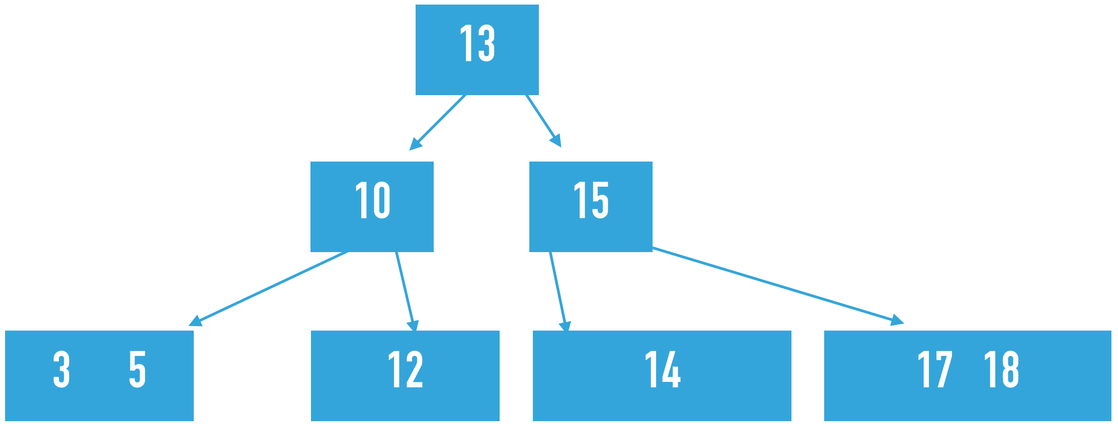
~~12, 5, 10, 3, 13, 14, 15, 17~~, 18, 15

Question 3.1.9 insertions



~~12, 5, 10, 3, 13, 14, 15, 17, 18~~, 15

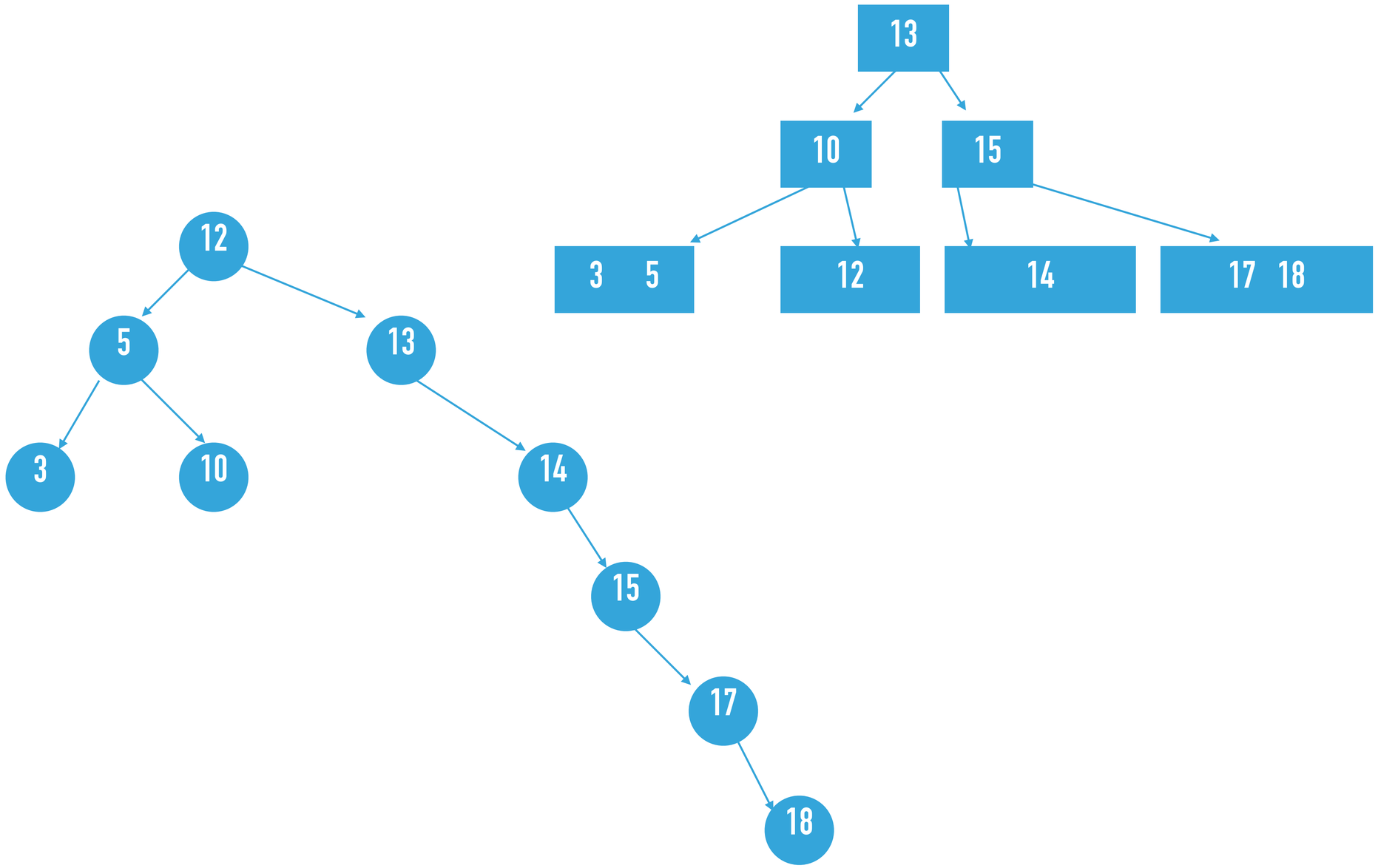
Question 3.1.9 insertions



15 est déjà dans l'arbre

~~12, 5, 10, 3, 13, 14, 15, 17, 18, 15~~

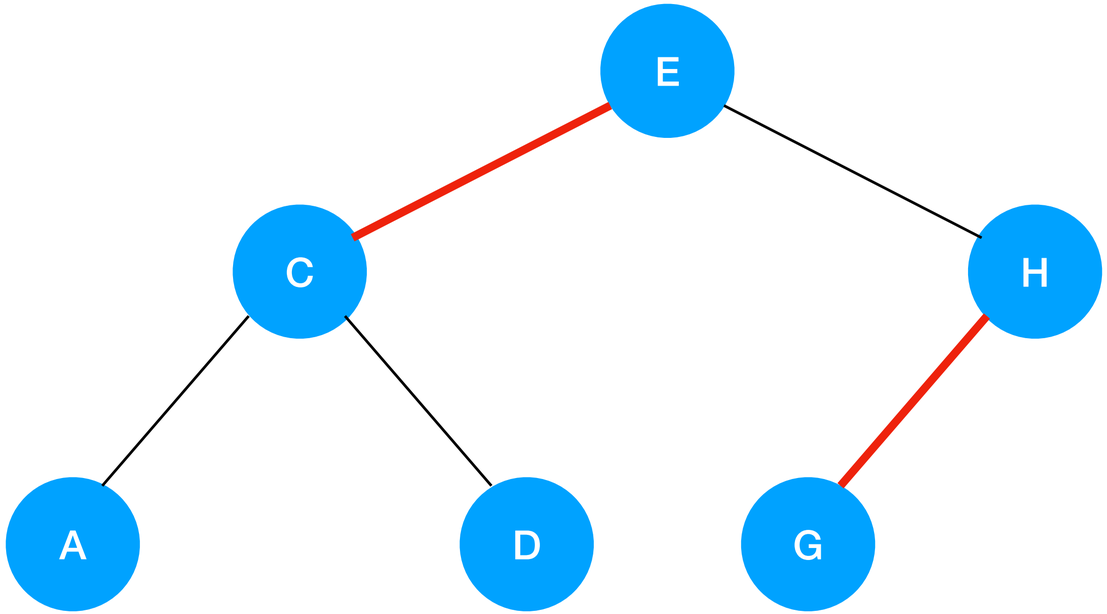
Question 3.1.9 insertions



Arbre Red-Black

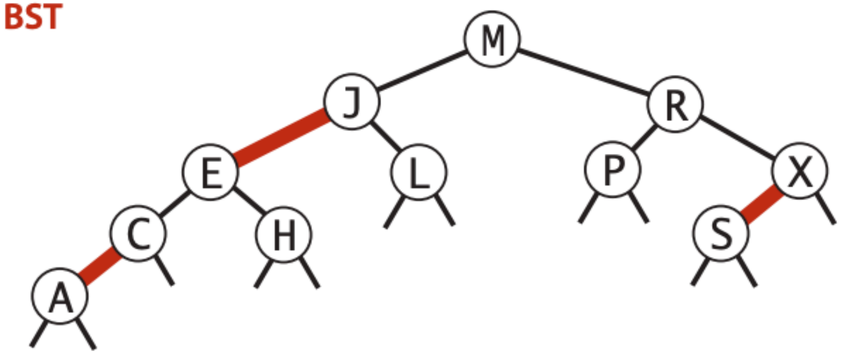
Red-Black Tree :

- 2 types of link (**red** or **black**)
- **Red** links always on **left**
- **No** two **red** links in a row
- **Perfect balance** on **black** links

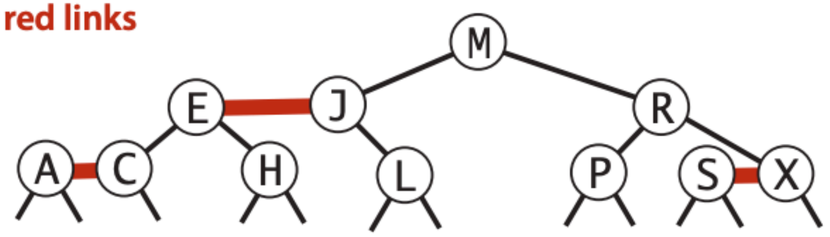


Correspondance 2-3 et Red-Black

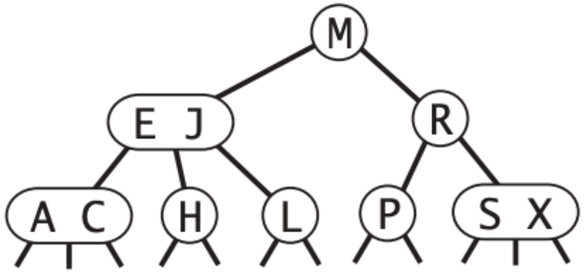
red-black BST



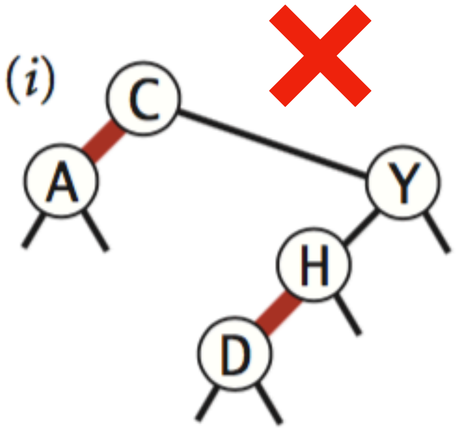
horizontal red links

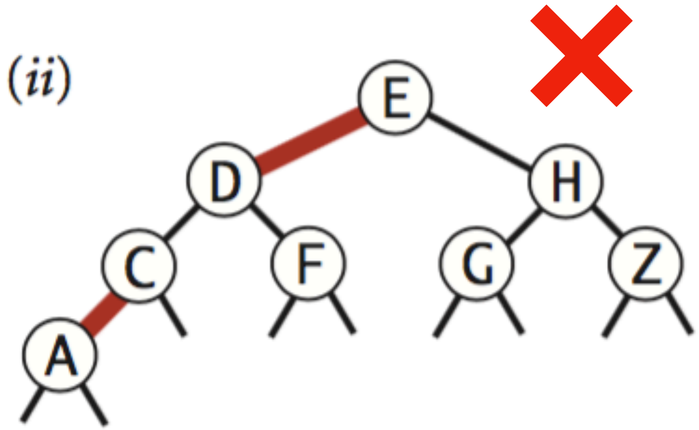


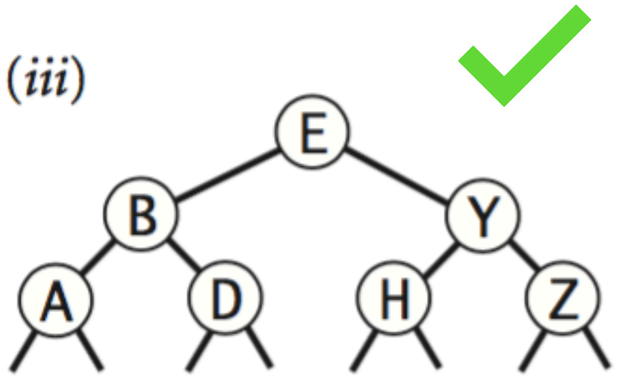
2-3 tree

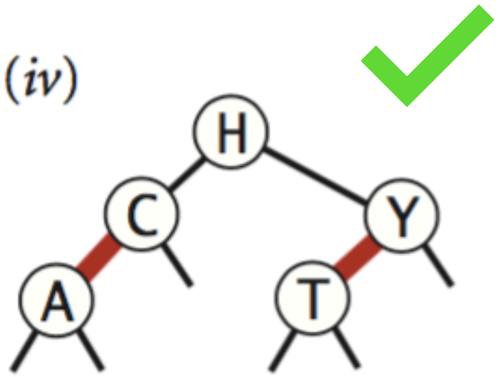


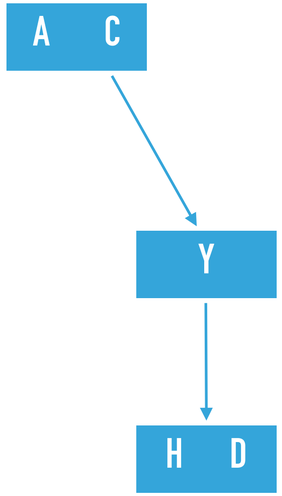
Question 3.1.10: Which one is a valid red-black?

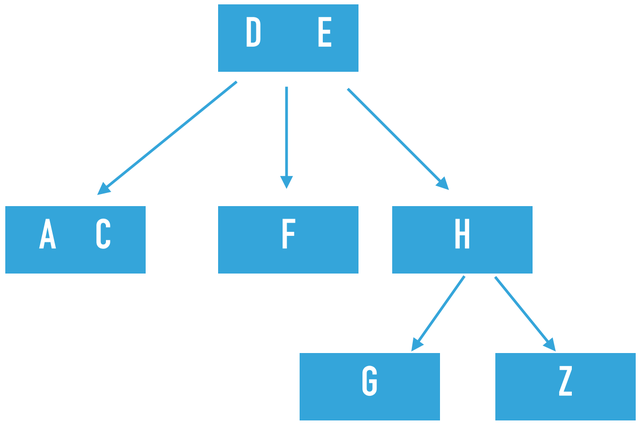


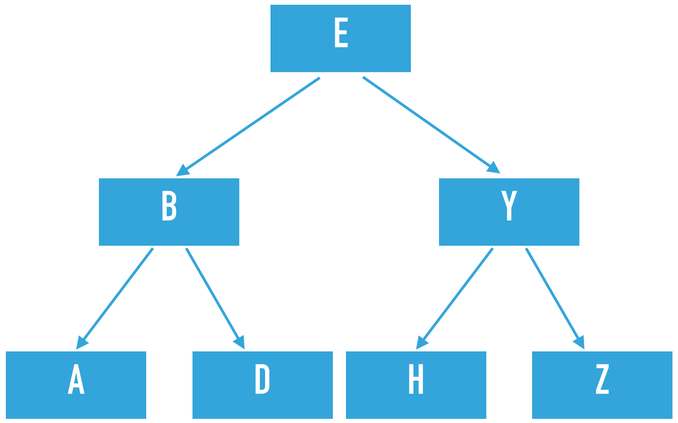


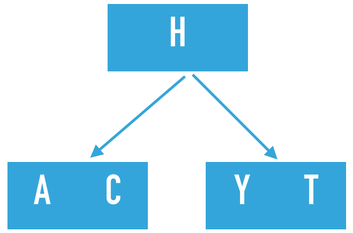












Question 3.1.7 IS RED BLACK

```
private boolean is23() { return is23(root); }

private boolean is23(Node x) {
    if (x == null)
        return true;

    //Est-ce que le lien à droite est rouge ?
    if (isRed(x.right))
        return false;

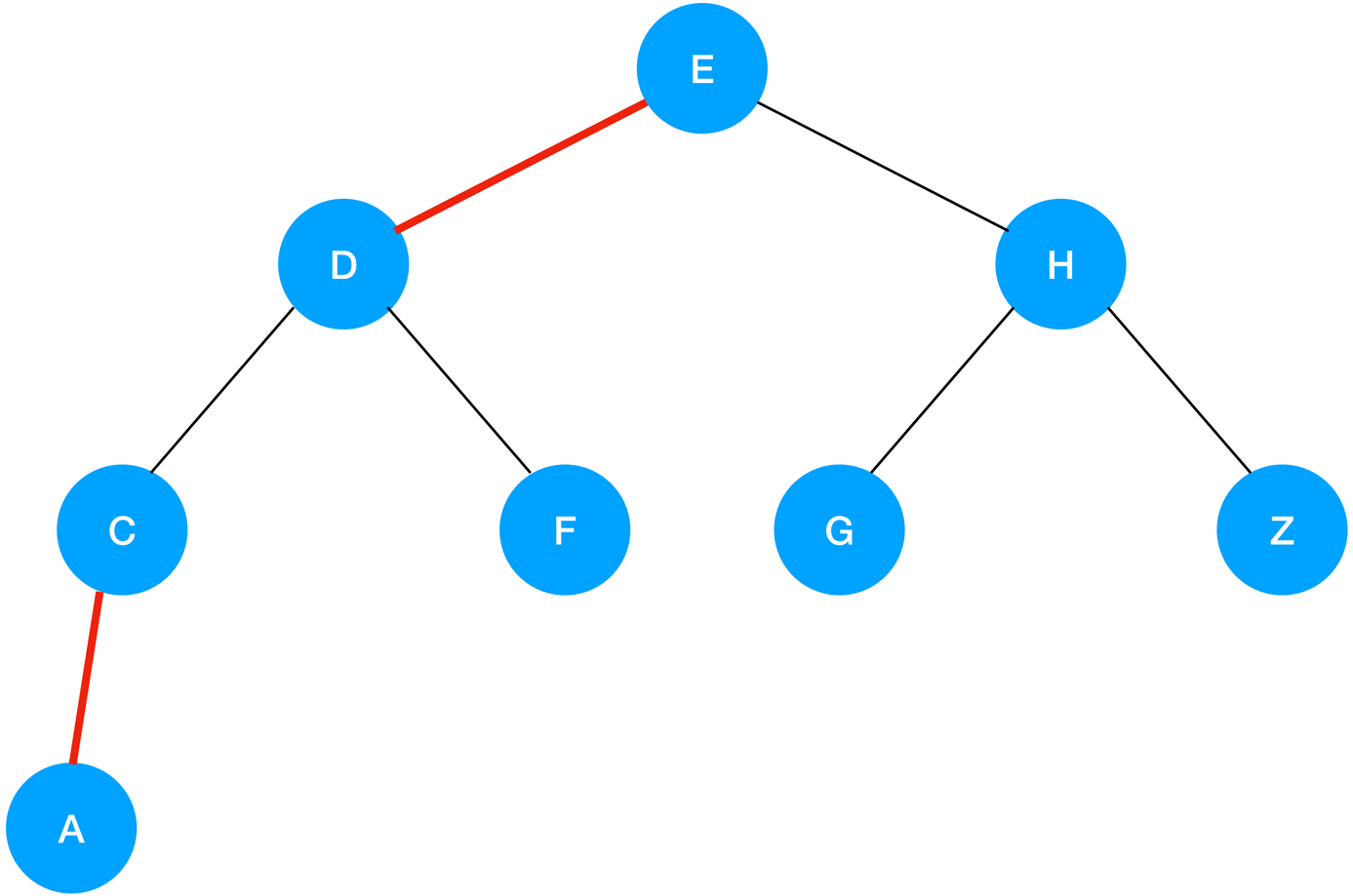
    //Est-ce qu'il y a 2 liens rouges d'affilée ?
    if (x != root && isRed(x) && isRed(x.left))
        return false;
    return is23(x.left) && is23(x.right);
}
```

Question 3.1.7 IS RED BLACK

```
private boolean isBalanced() {
        // number of black links from the root to the min key
        int black = 0;
        Node x = root;
        while (x != null) {
            if (!isRed(x)) black++;
            x = x.left;
        }
        // Do we retrieve this number of links for each leaf ?
        return isBalanced(root, black);
    }


private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

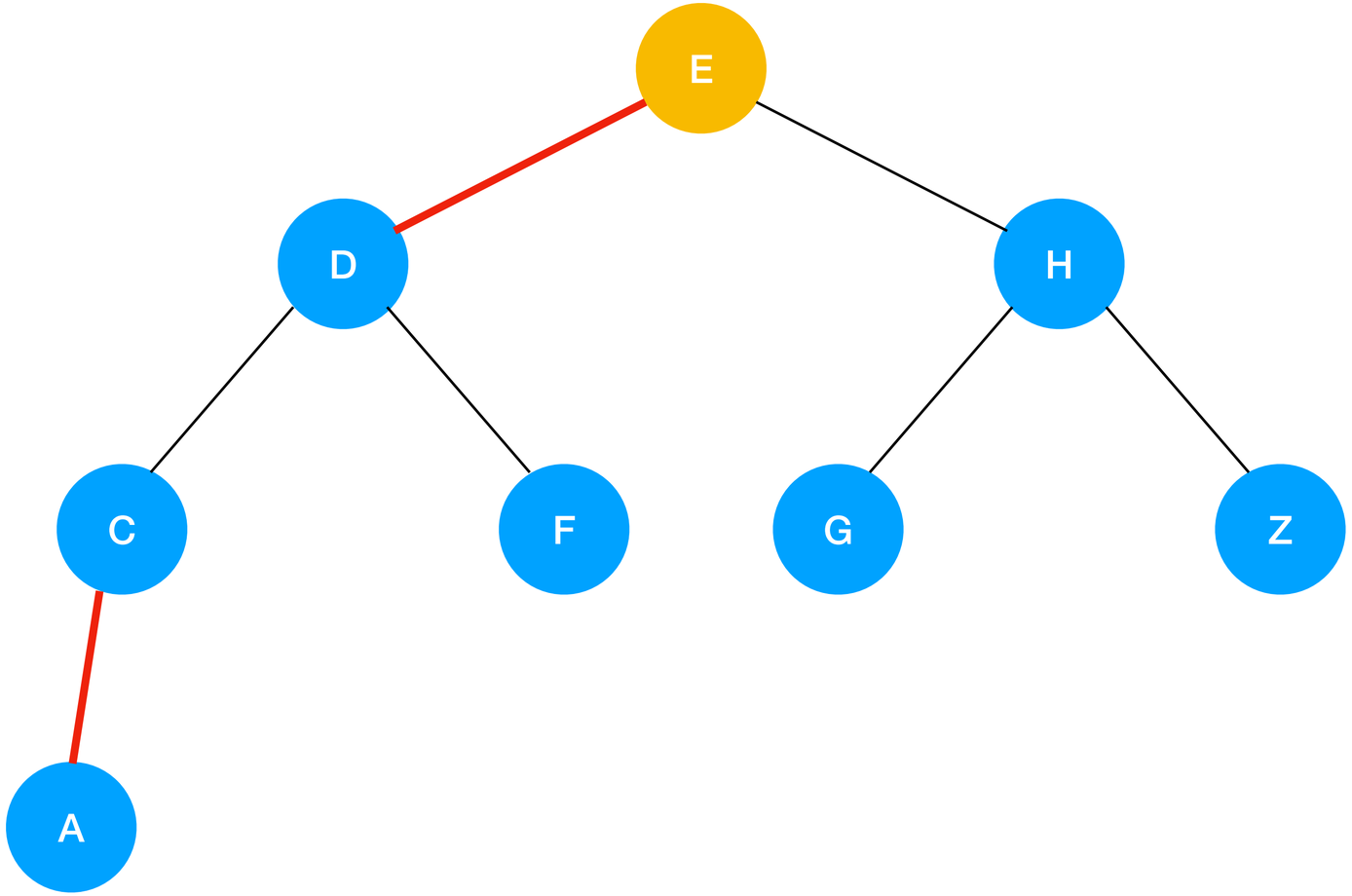
Question 3.1.7 IS RED BLACK



Node =
Black = 0

```
int black = 0;
Node x = root;
```

Question 3.1.7 IS RED BLACK

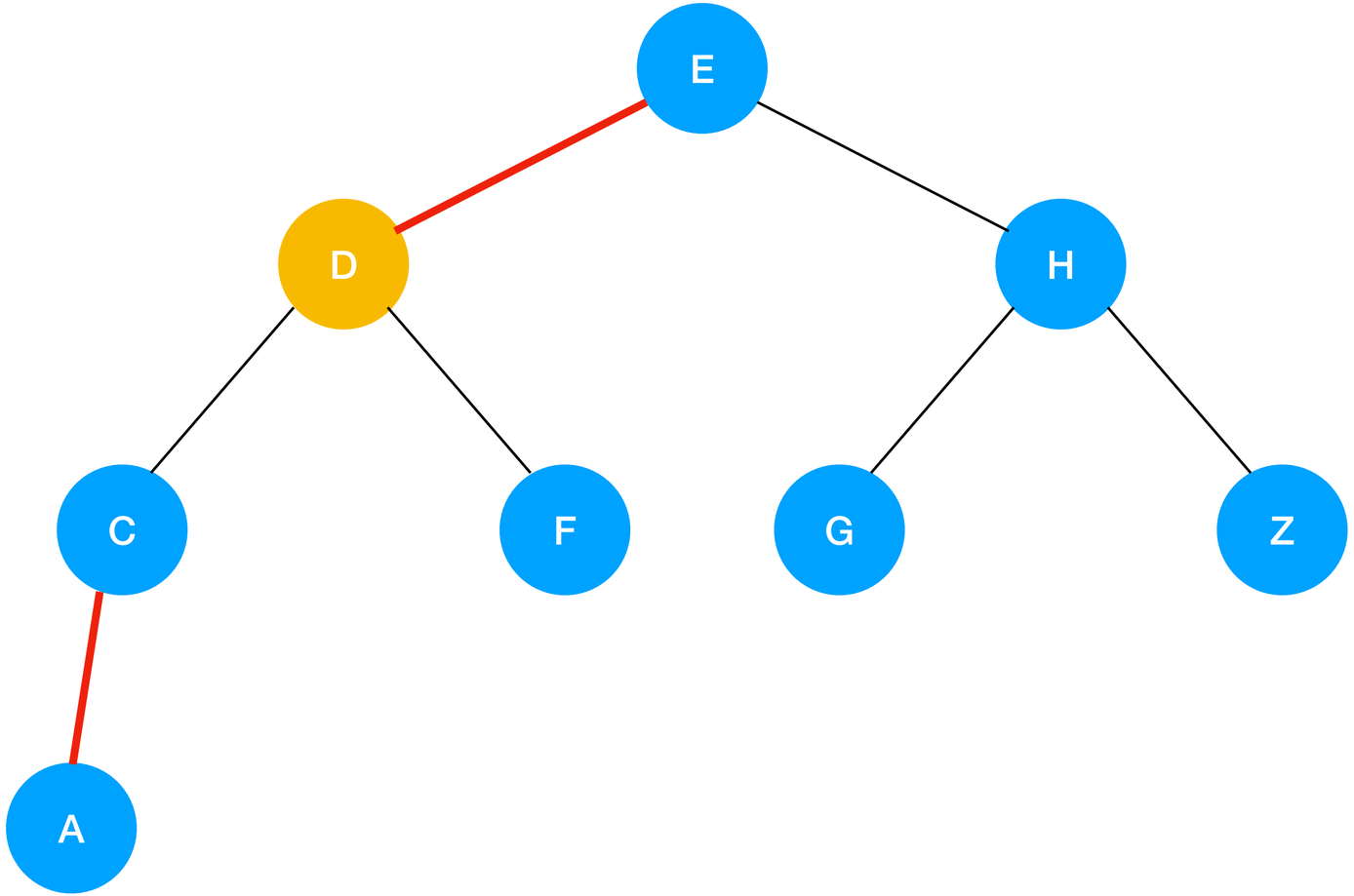


Node = E

Black = 1

```
while (x != null) {
        if (!isRed(x)) black++;
        x = x.left;
}
```

Question 3.1.7 IS RED BLACK

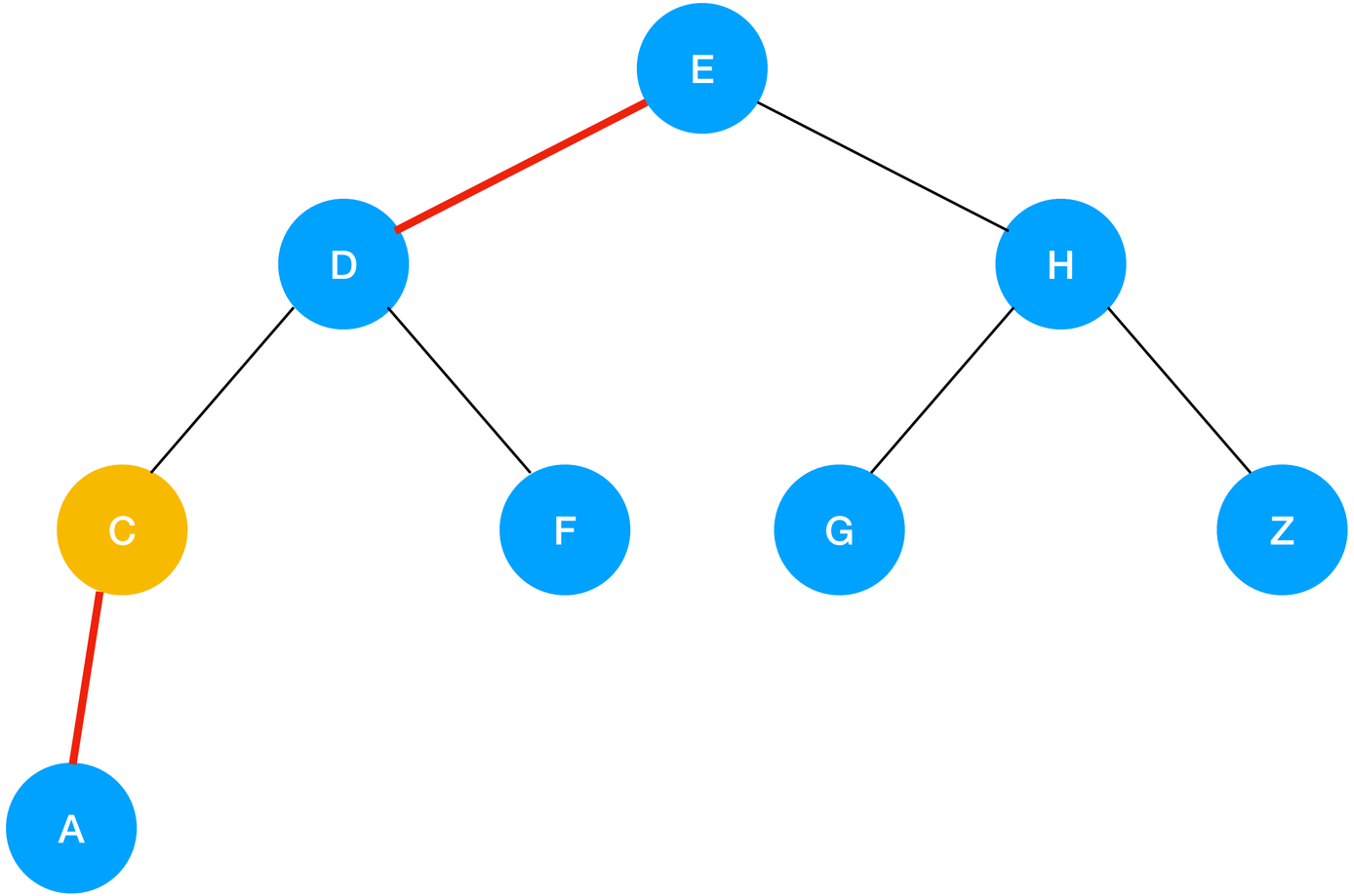


Node = D

Black = 1

```
while (x != null) {
        if (!isRed(x)) black++;
        x = x.left;
}
```

Question 3.1.7 IS RED BLACK

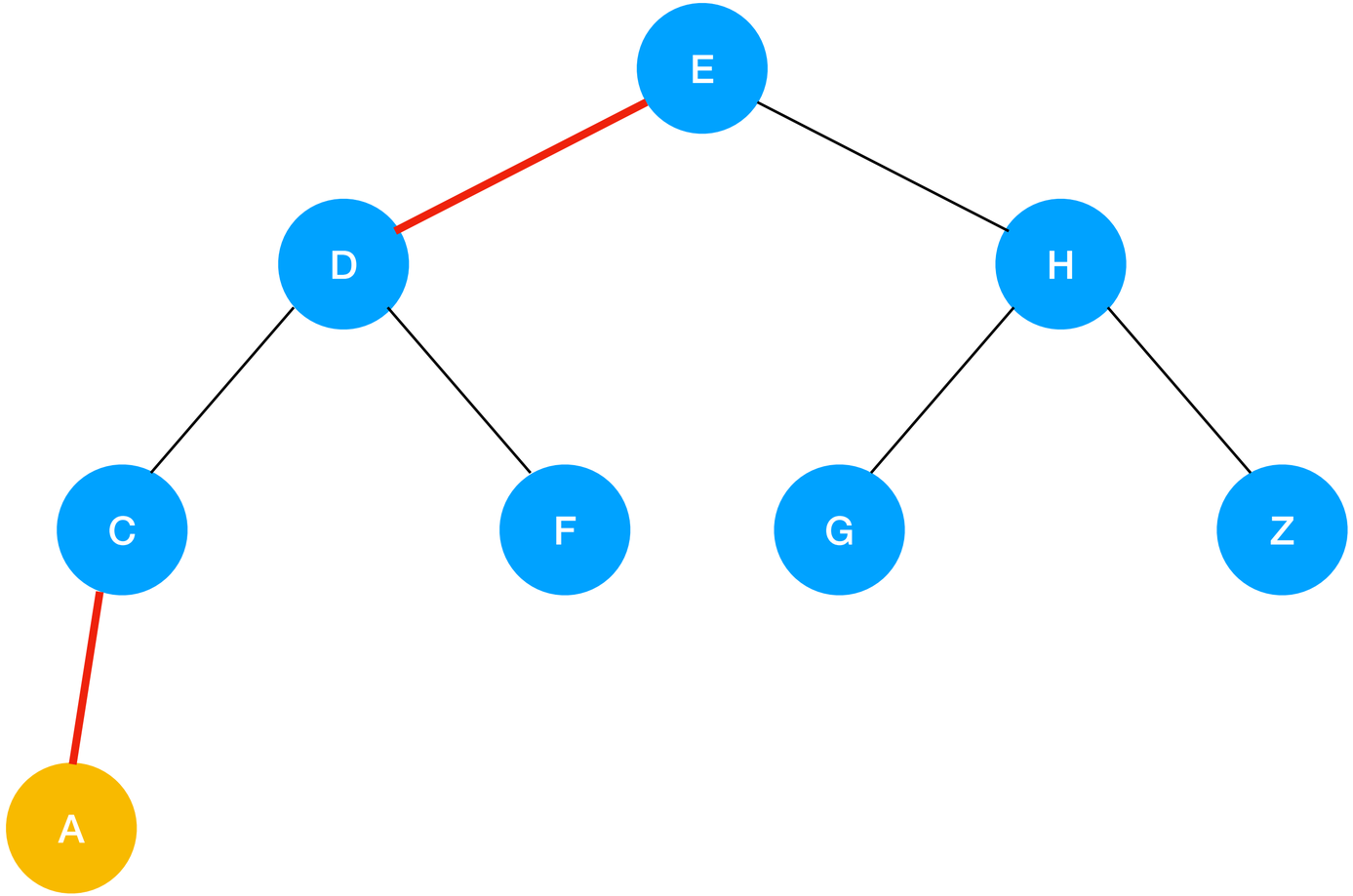


Node = C

Black = 2

```
while (x != null) {
        if (!isRed(x)) black++;
        x = x.left;
}
```

Question 3.1.7 IS RED BLACK

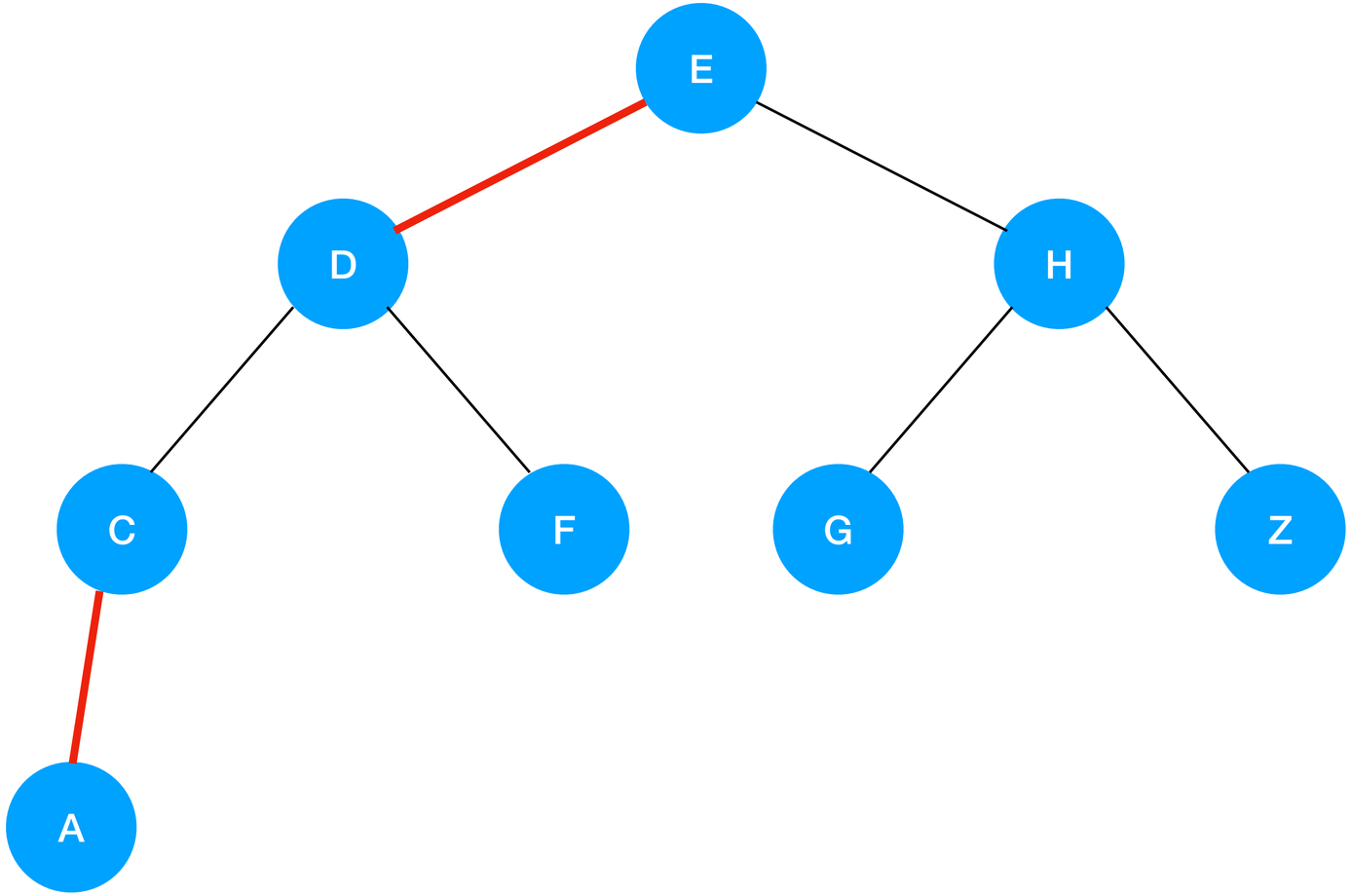


Node = A

Black = 2

```
while (x != null) {
        if (!isRed(x)) black++;
        x = x.left;
}
```

Question 3.1.7 IS RED BLACK

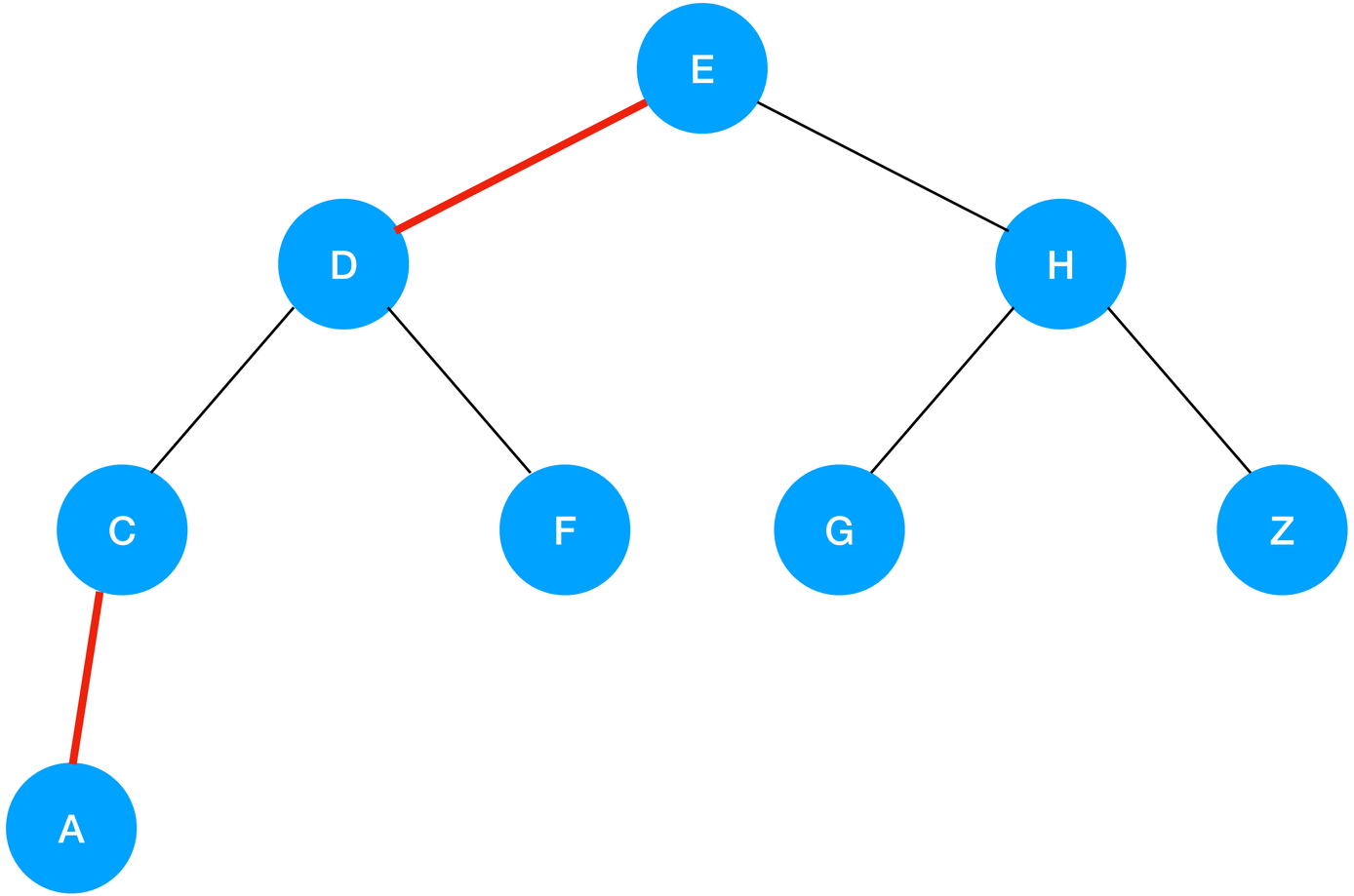


Node = null

Black = 2

```
while (x != null) {
        if (!isRed(x)) black++;
        x = x.left;
}
```

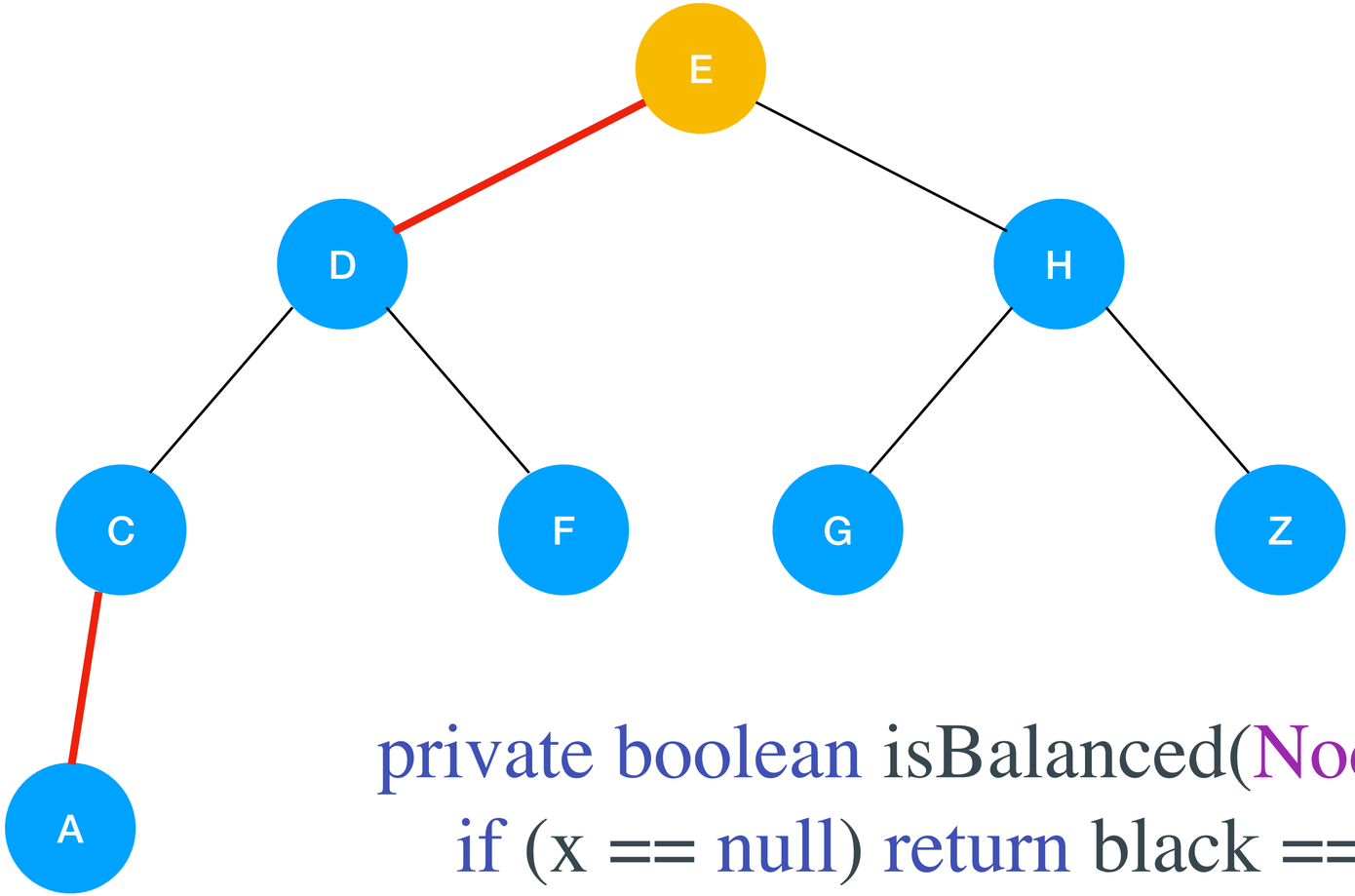
Question 3.1.7 IS RED BLACK



Node =
Black = 2

```
return isBalanced(root, black);
```

Question 3.1.7 IS RED BLACK

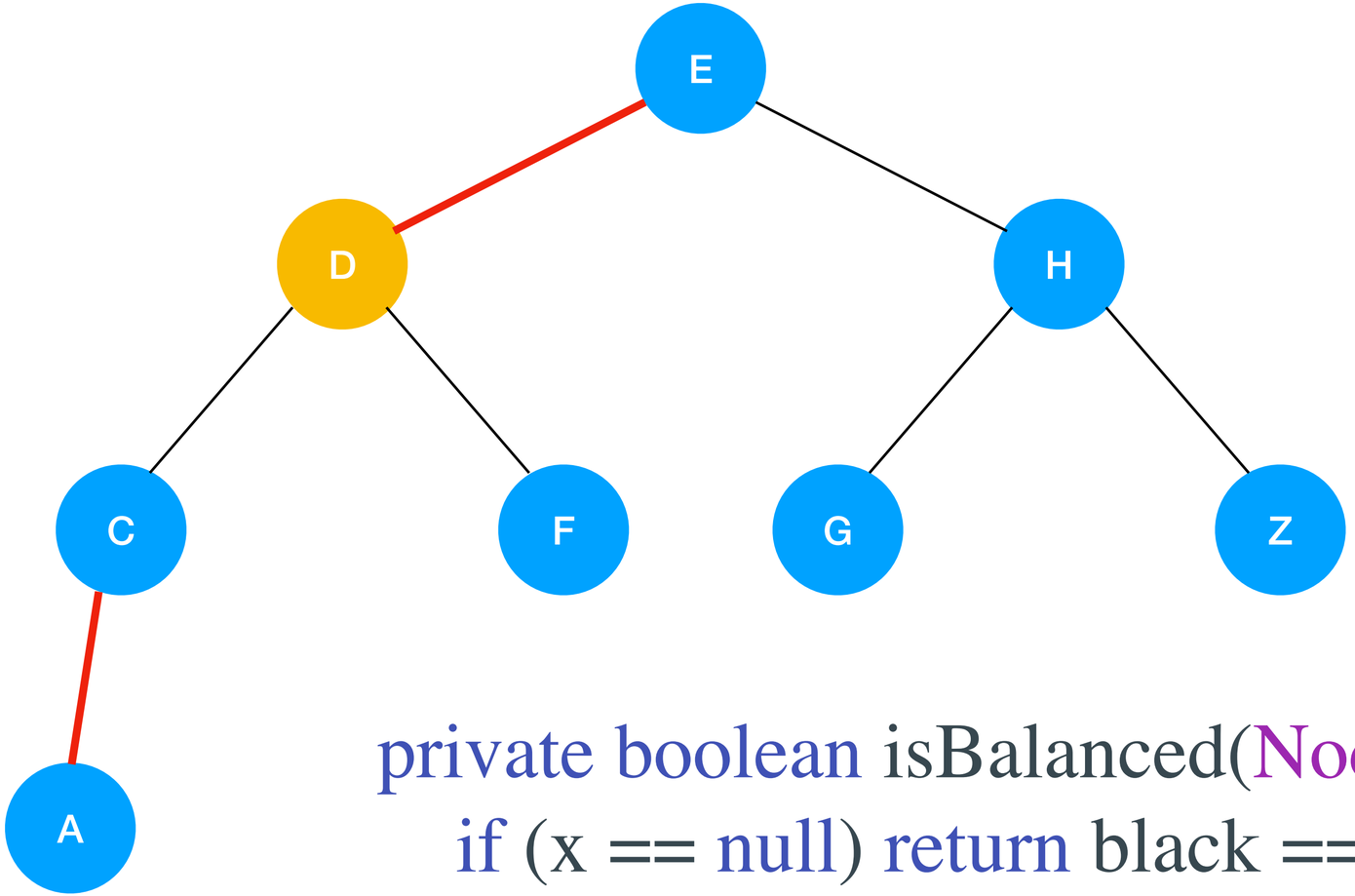


Node = E

Black = 1

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

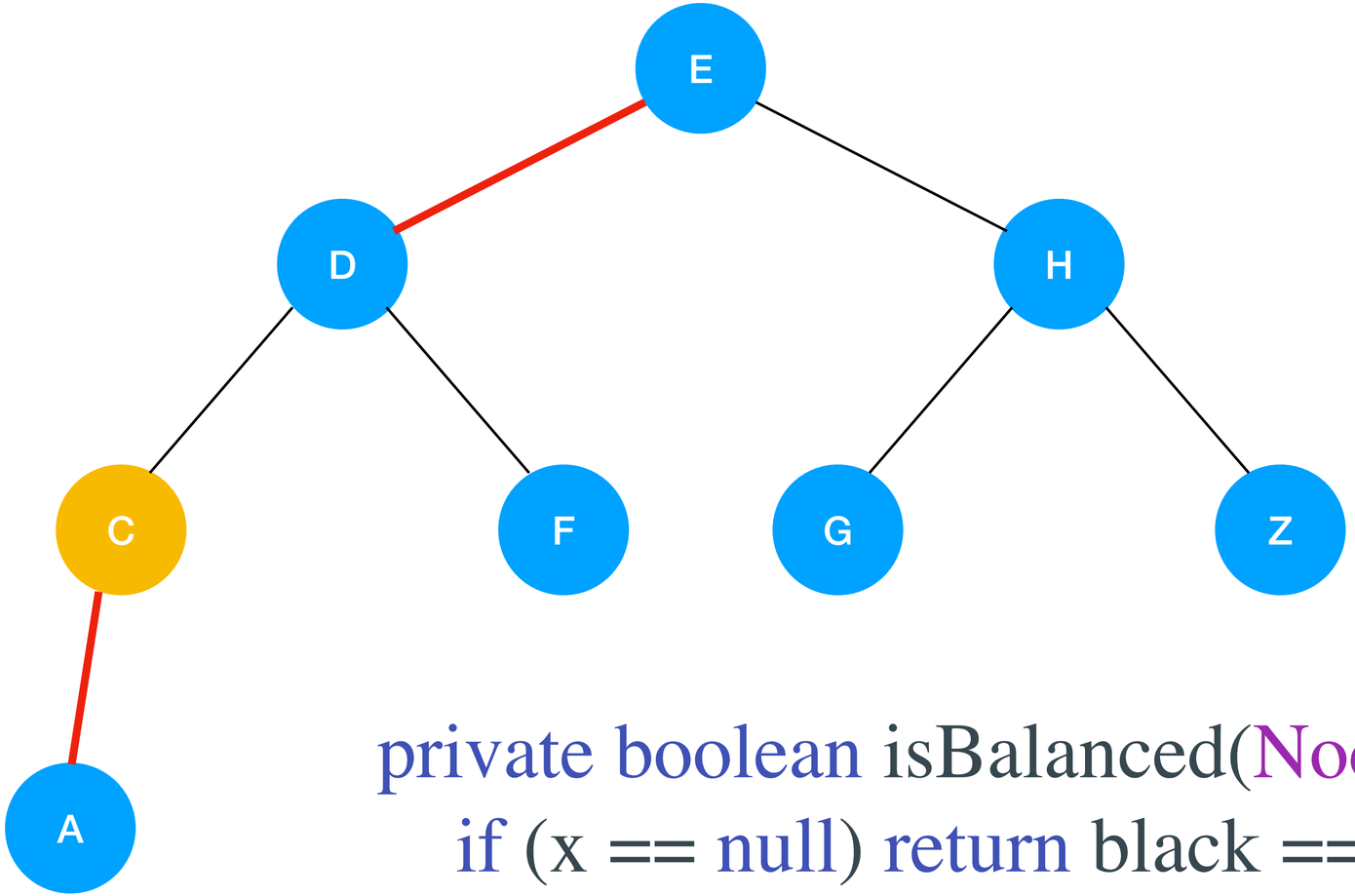
Question 3.1.7 IS RED BLACK



Node = D
Black = 1

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

Question 3.1.7 IS RED BLACK

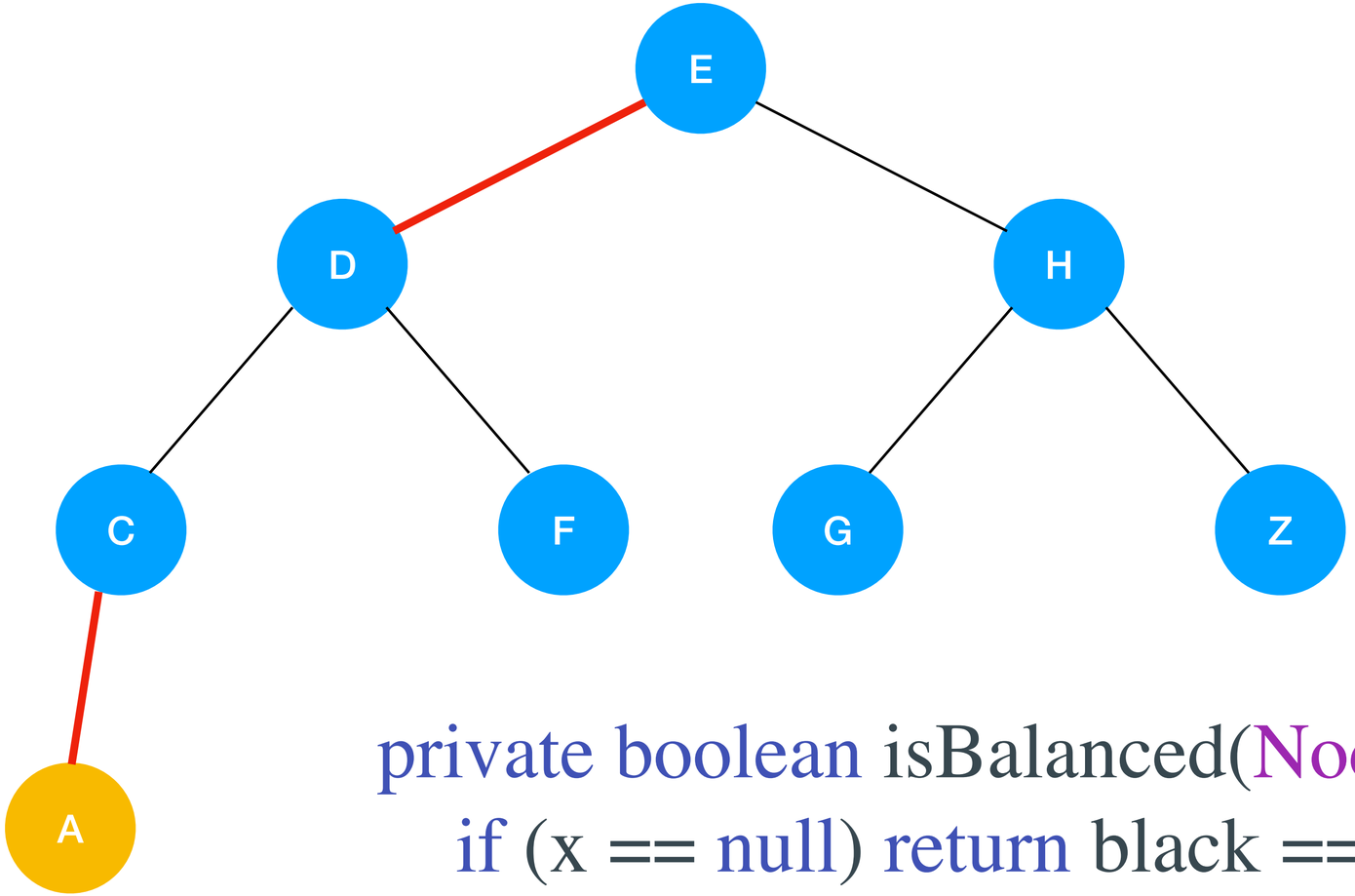


Node = C

Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

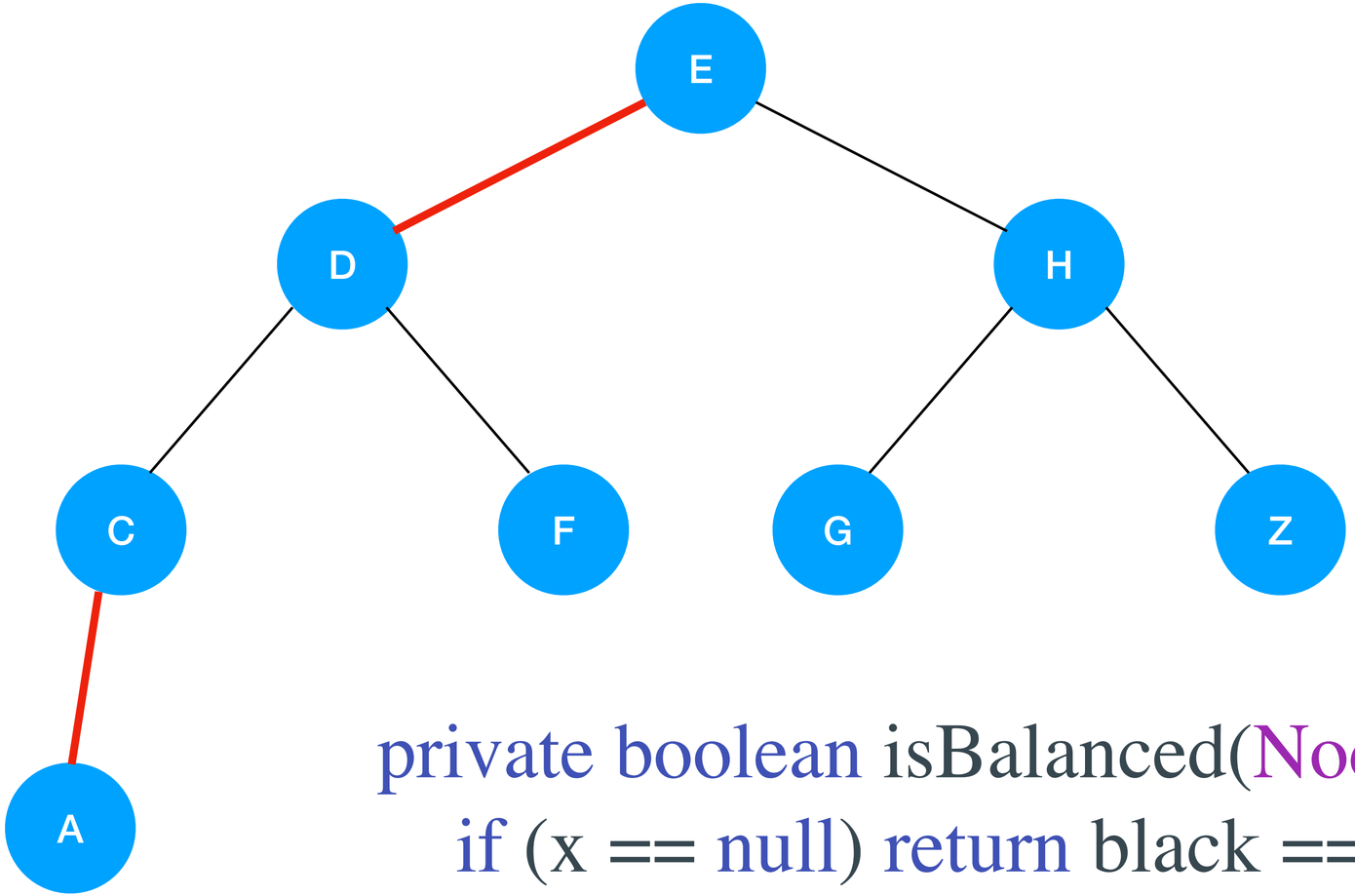
Question 3.1.7 IS RED BLACK



Node = A
Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

Question 3.1.7 IS RED BLACK

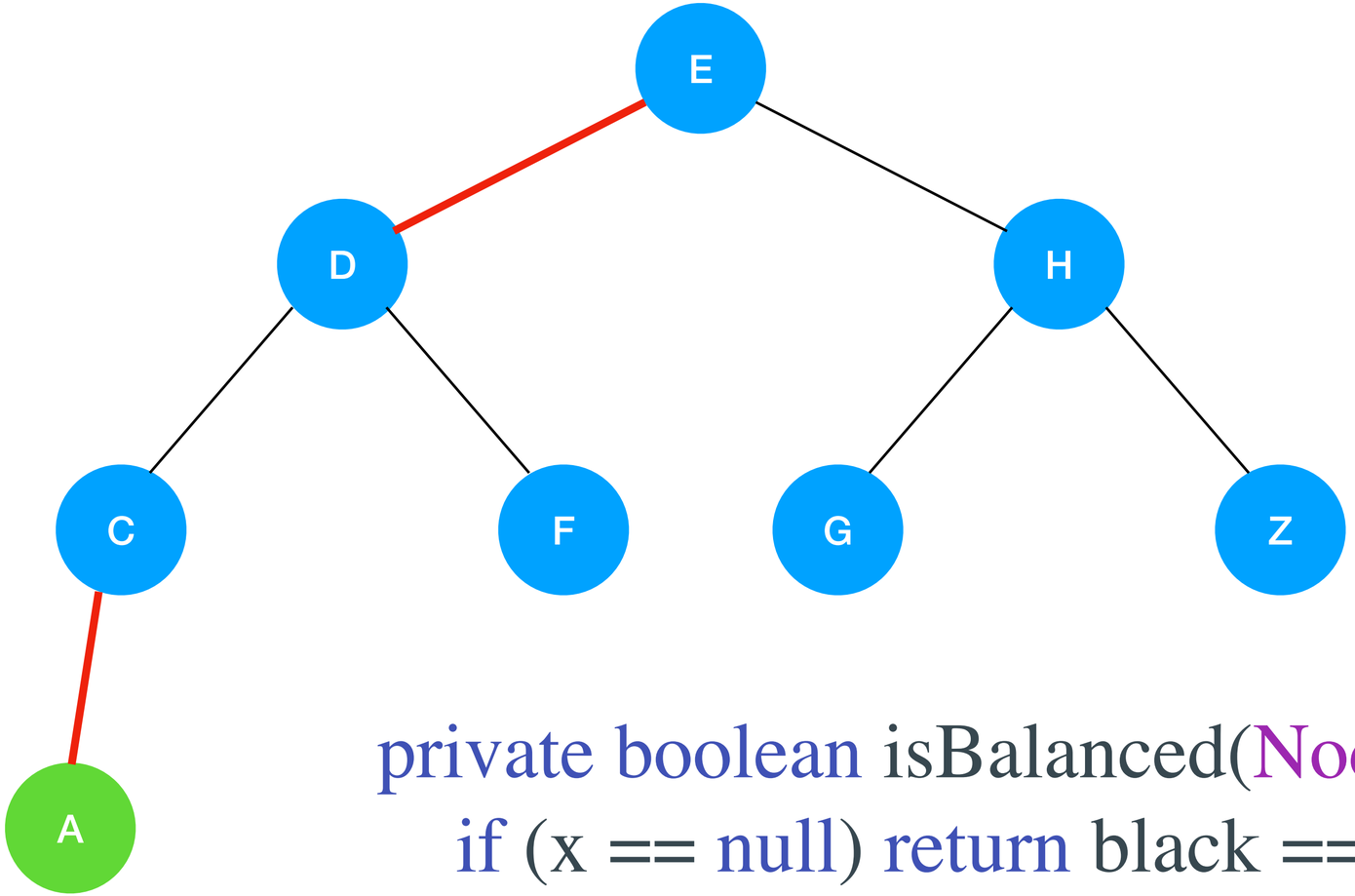


Node = null

Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

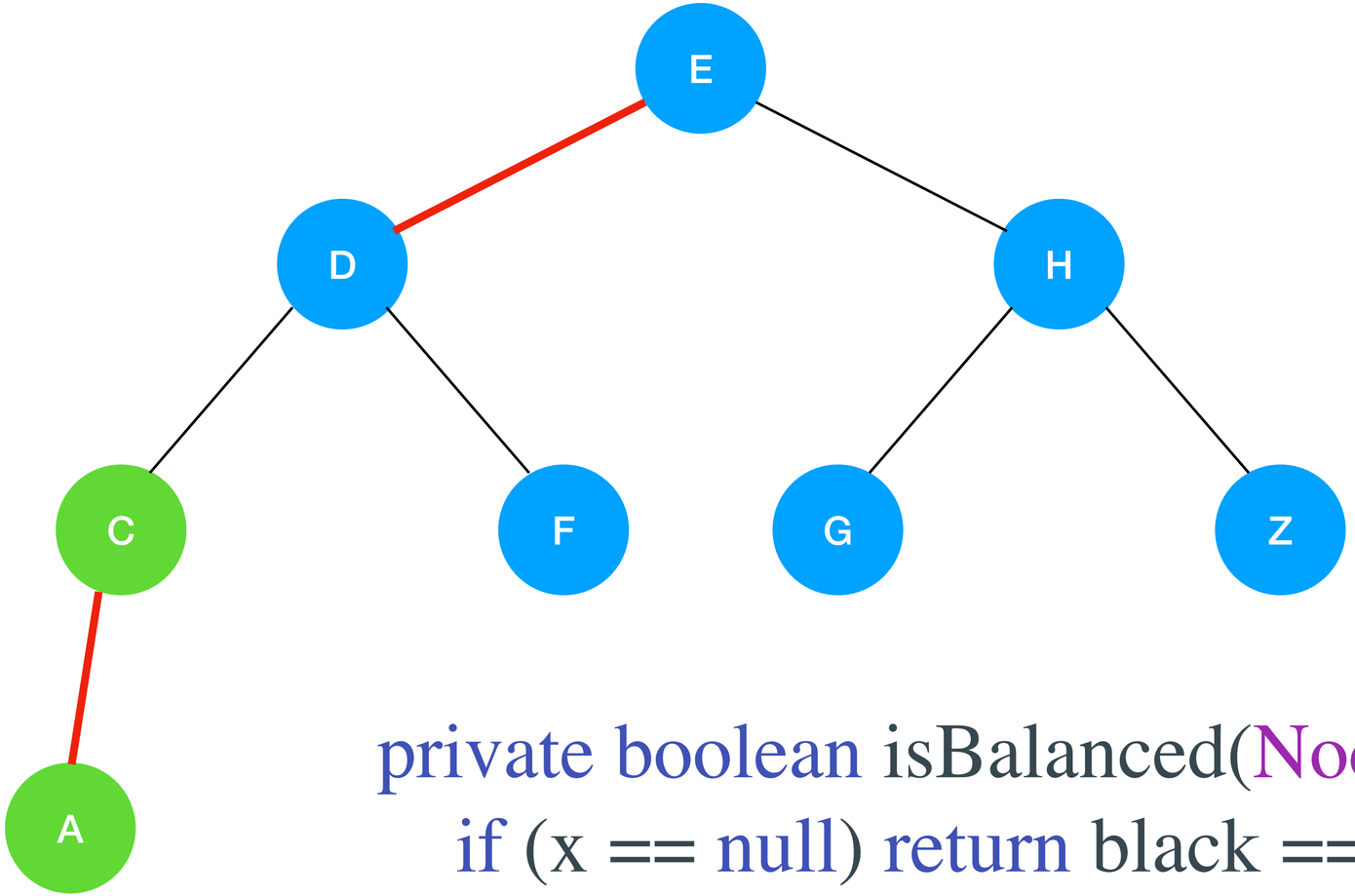
Question 3.1.7 IS RED BLACK



Node = A
Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

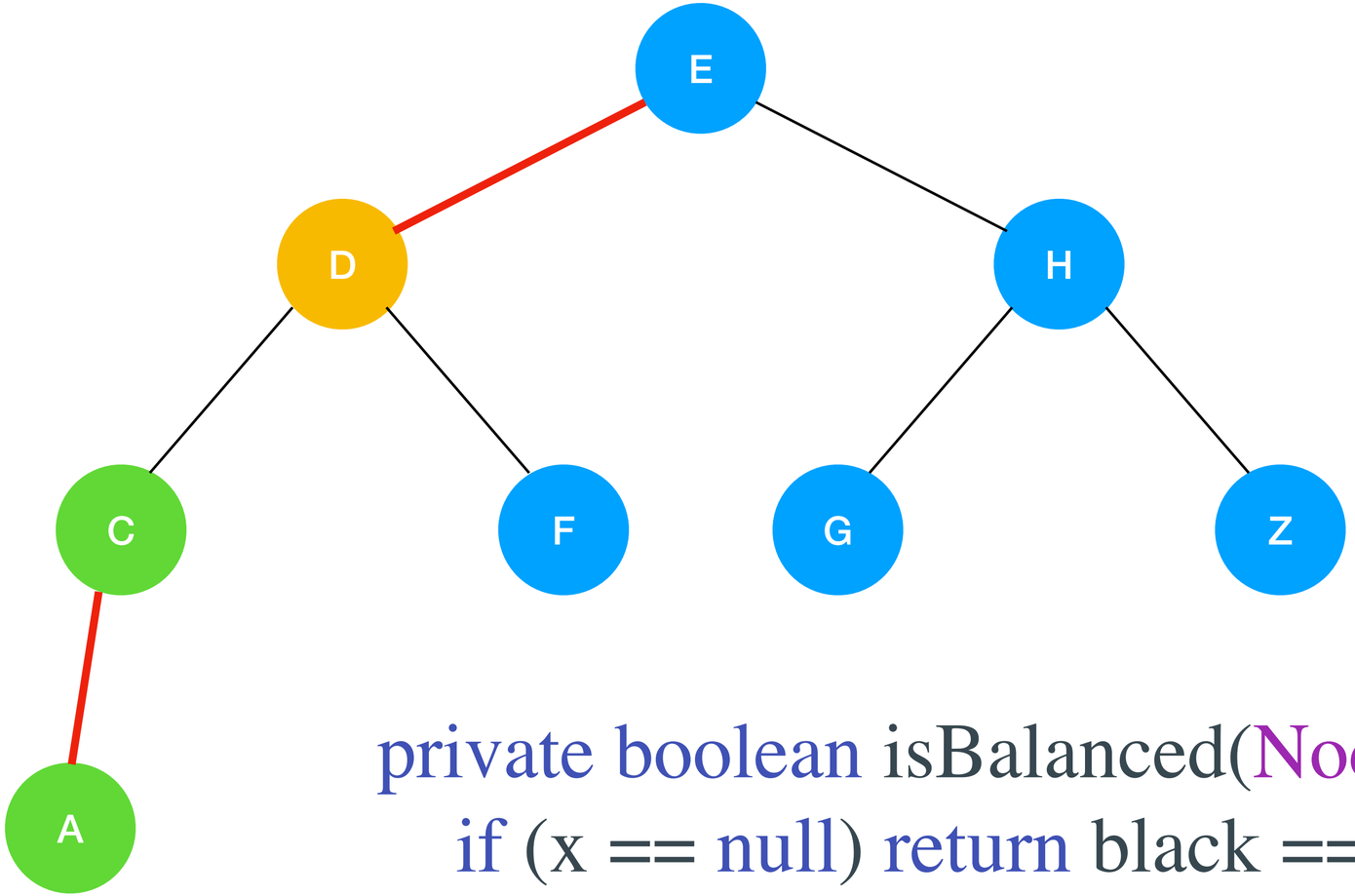
Question 3.1.7 IS RED BLACK



Node = C
Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

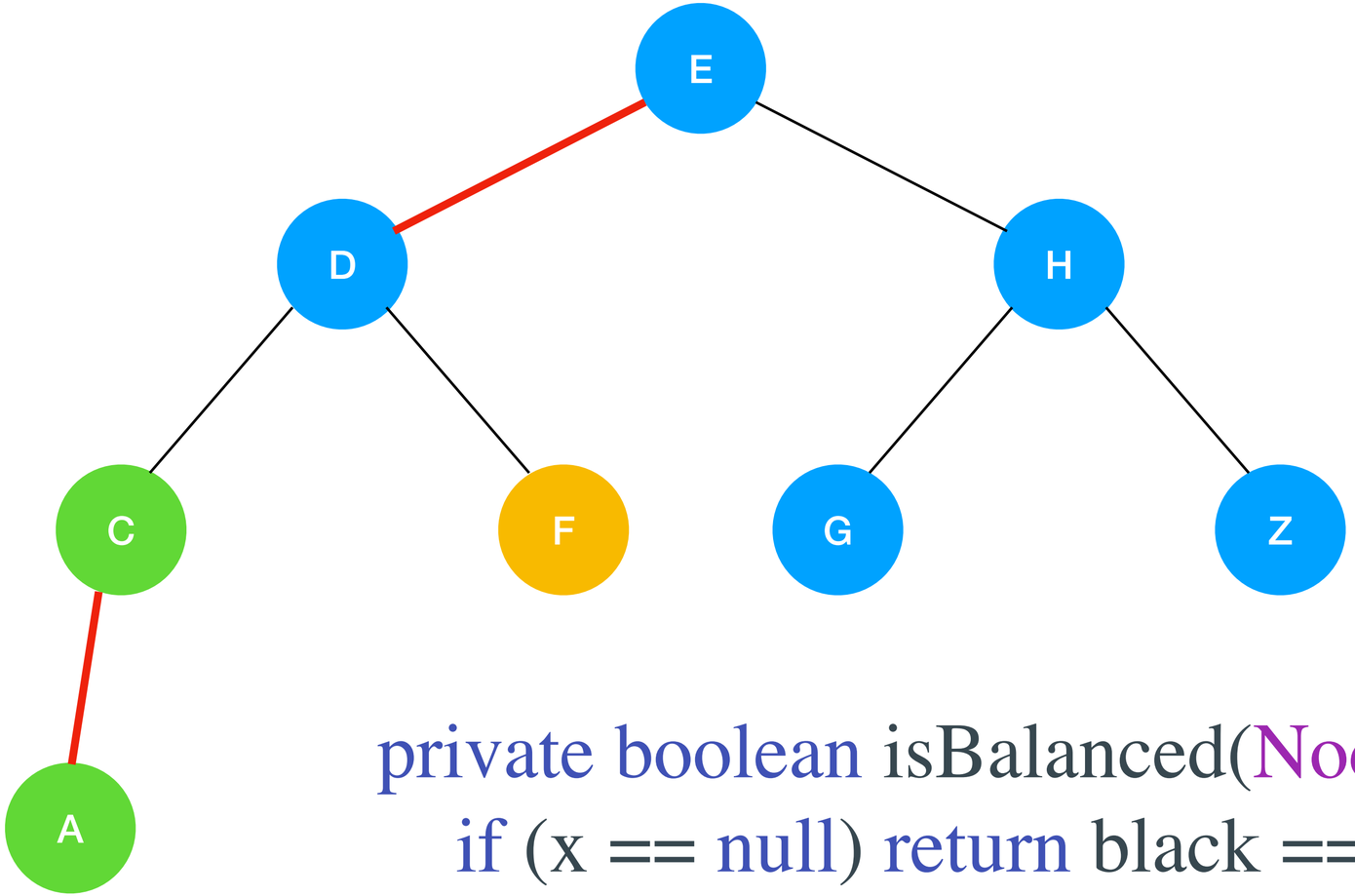
Question 3.1.7 IS RED BLACK



Node = D
Black = 1

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

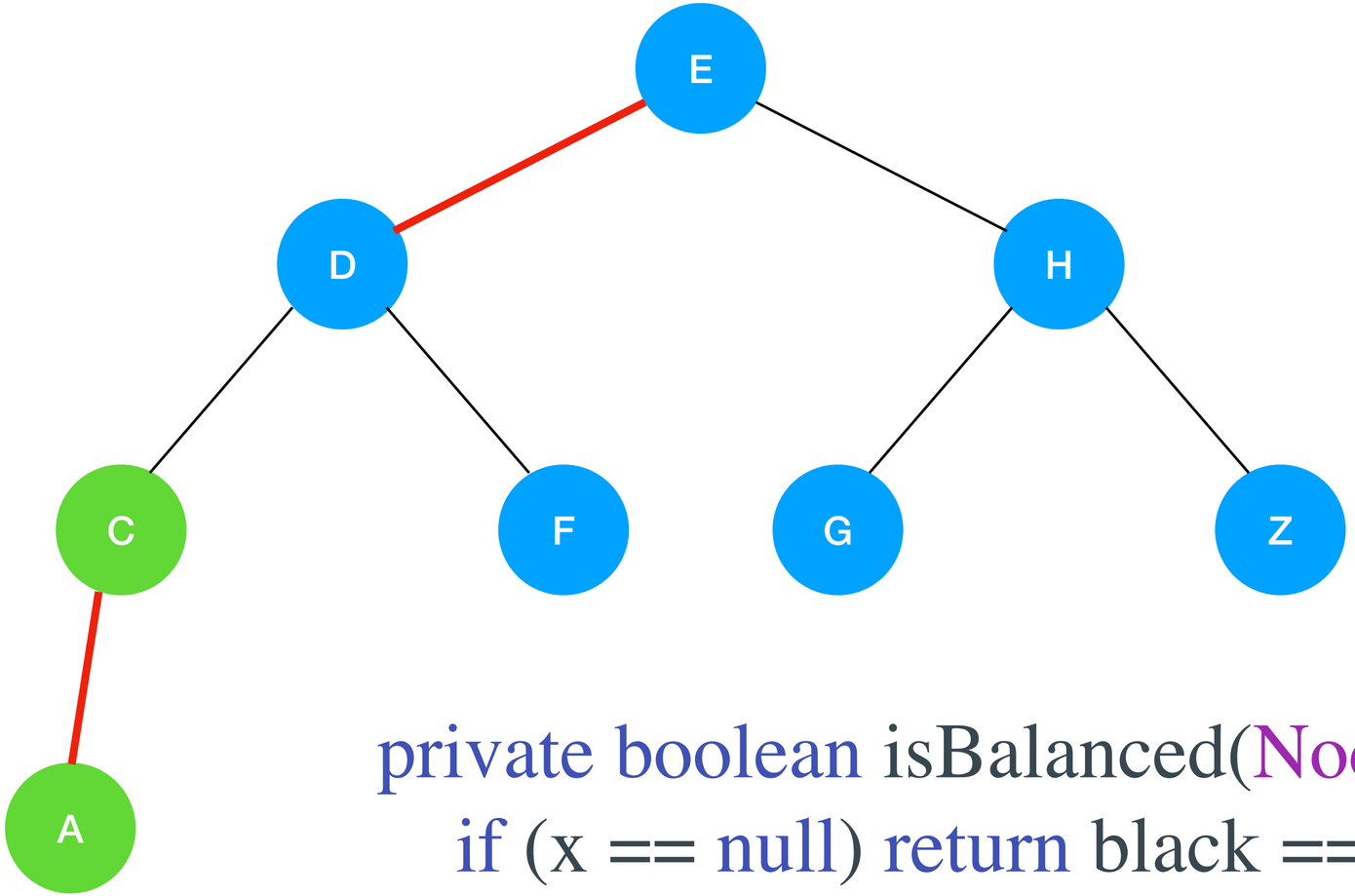
Question 3.1.7 IS RED BLACK



Node = F
Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

Question 3.1.7 IS RED BLACK

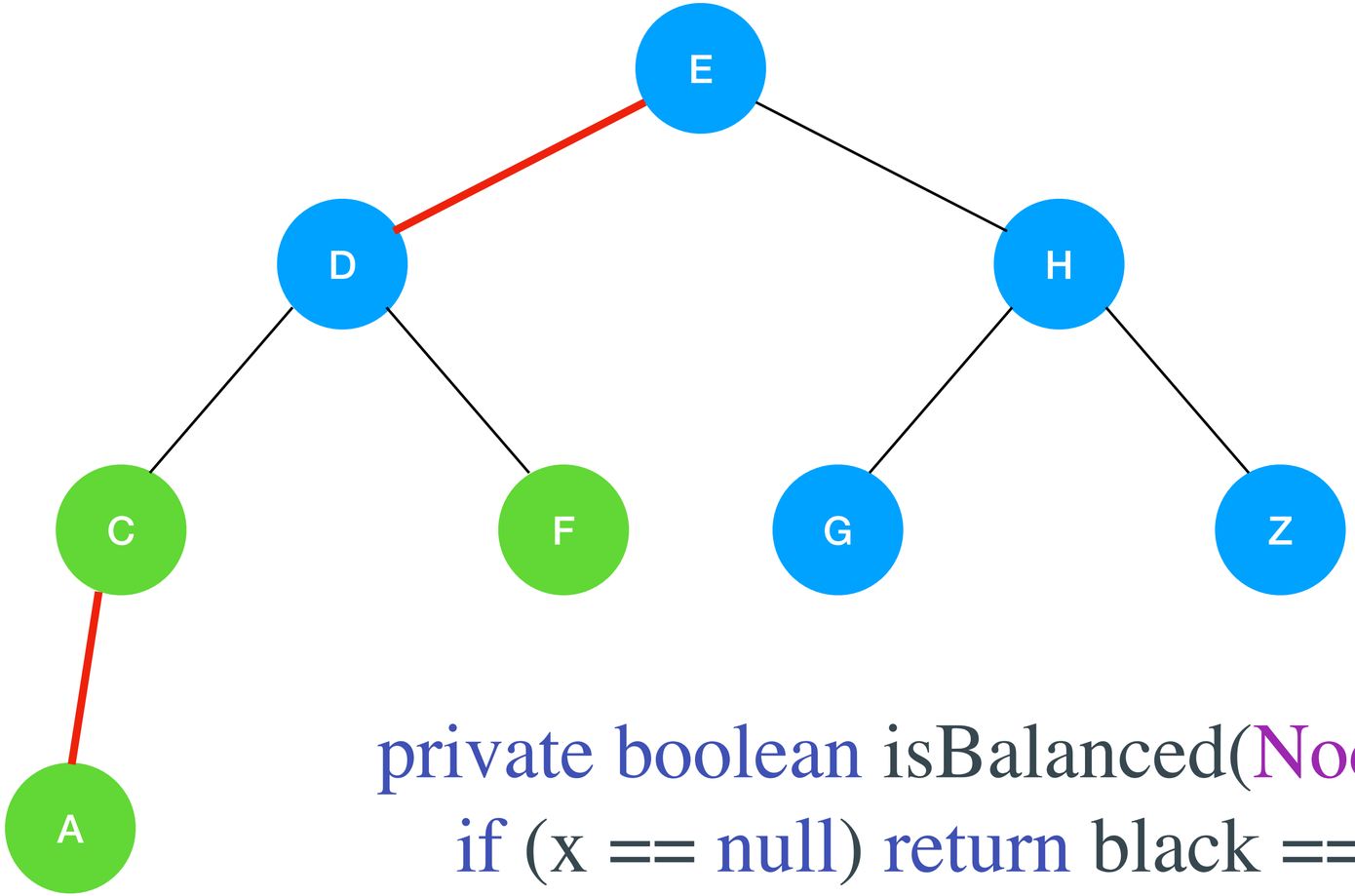


Node = null

Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

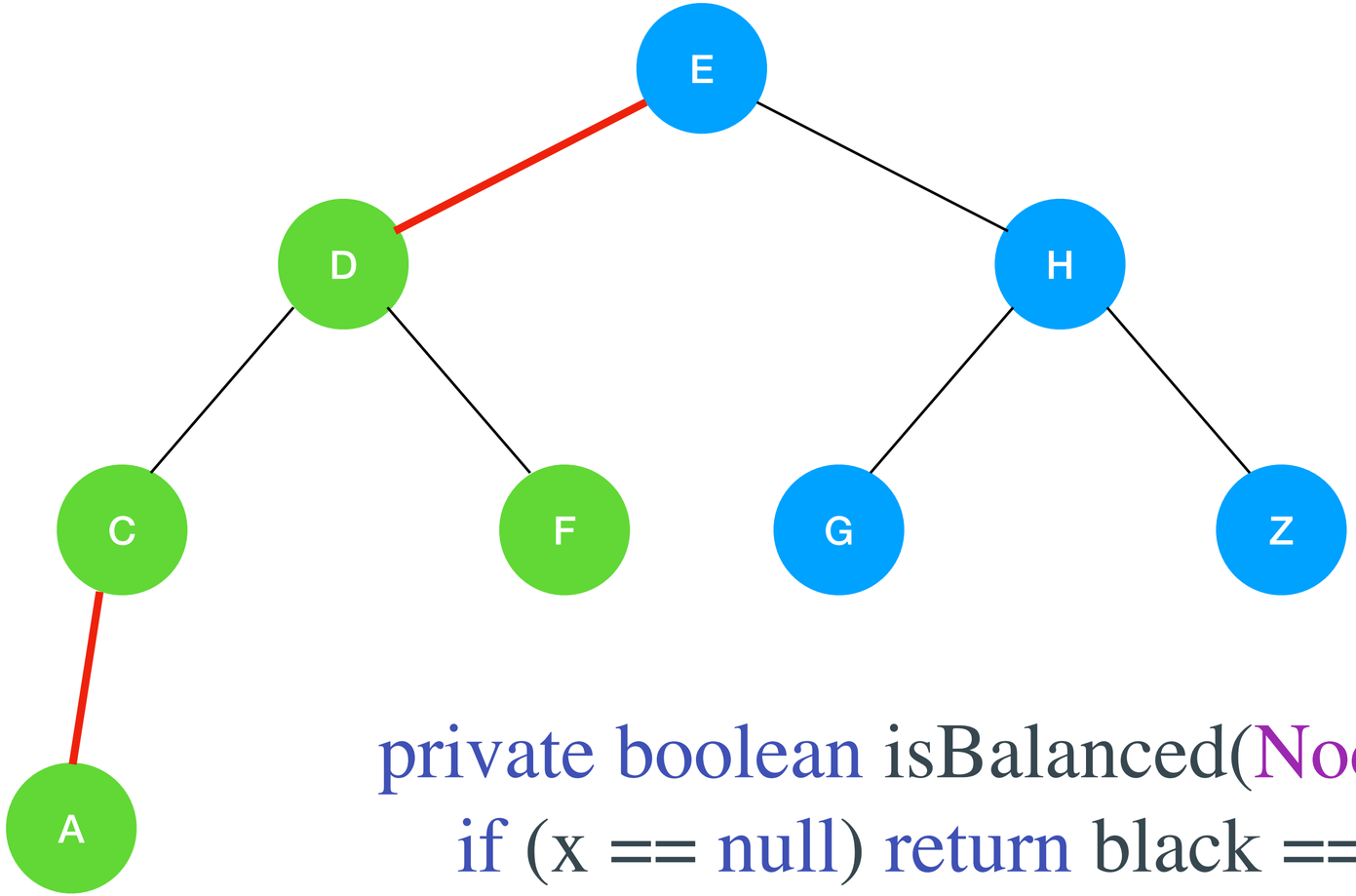
Question 3.1.7 IS RED BLACK



Node = F
Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

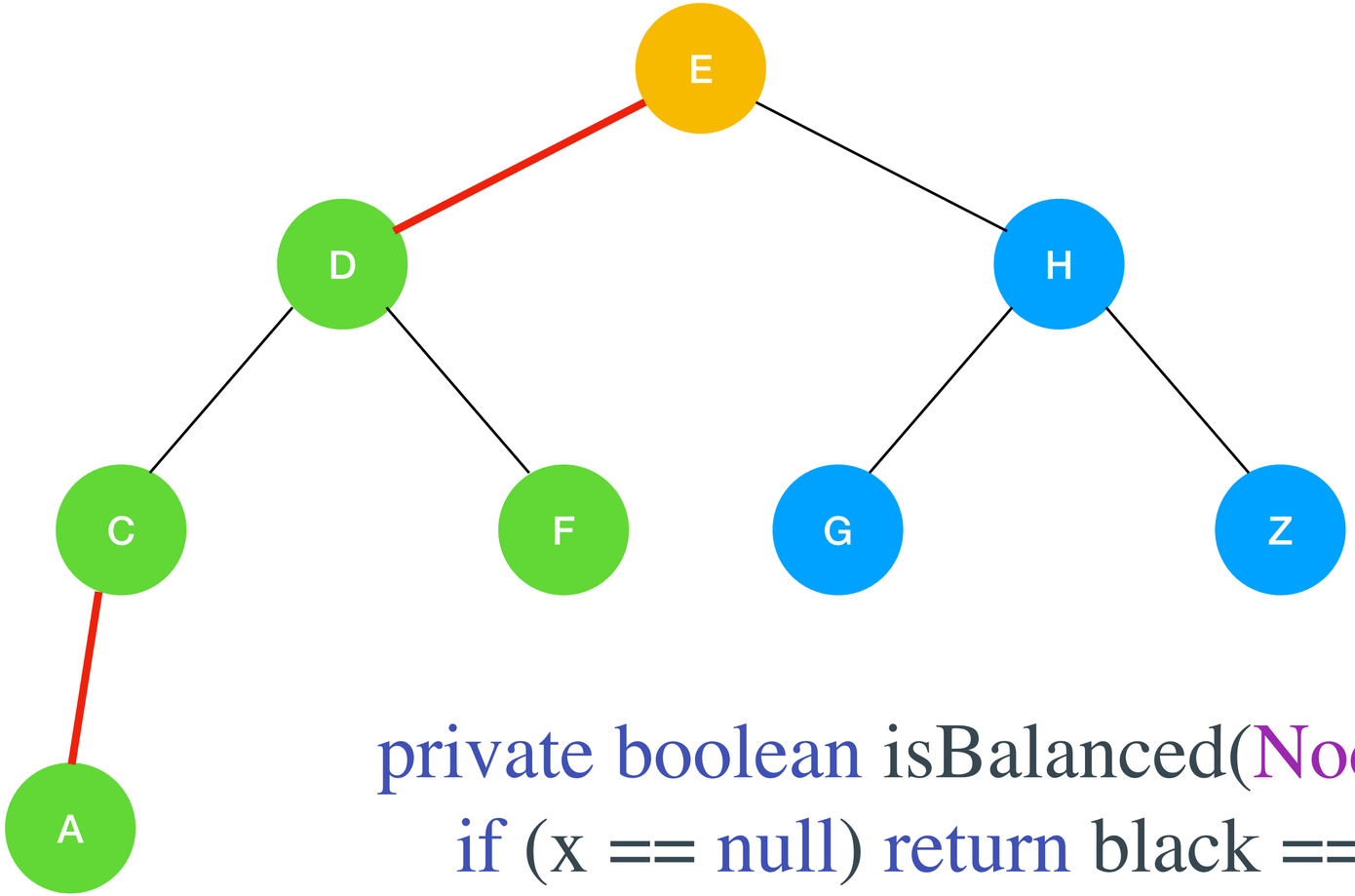
Question 3.1.7 IS RED BLACK



Node = D
Black = 1

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

Question 3.1.7 IS RED BLACK

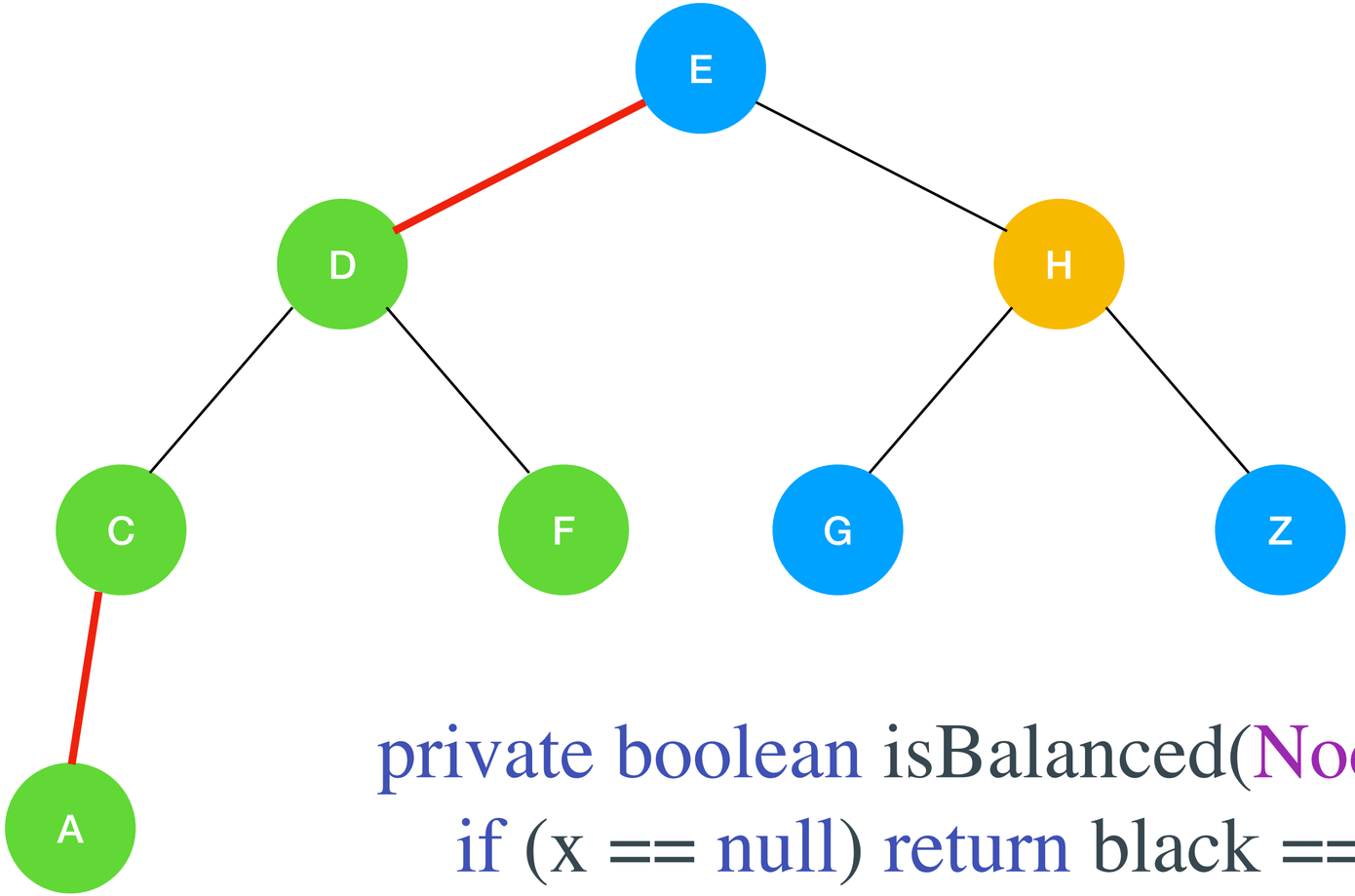


Node = E

Black = 1

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

Question 3.1.7 IS RED BLACK

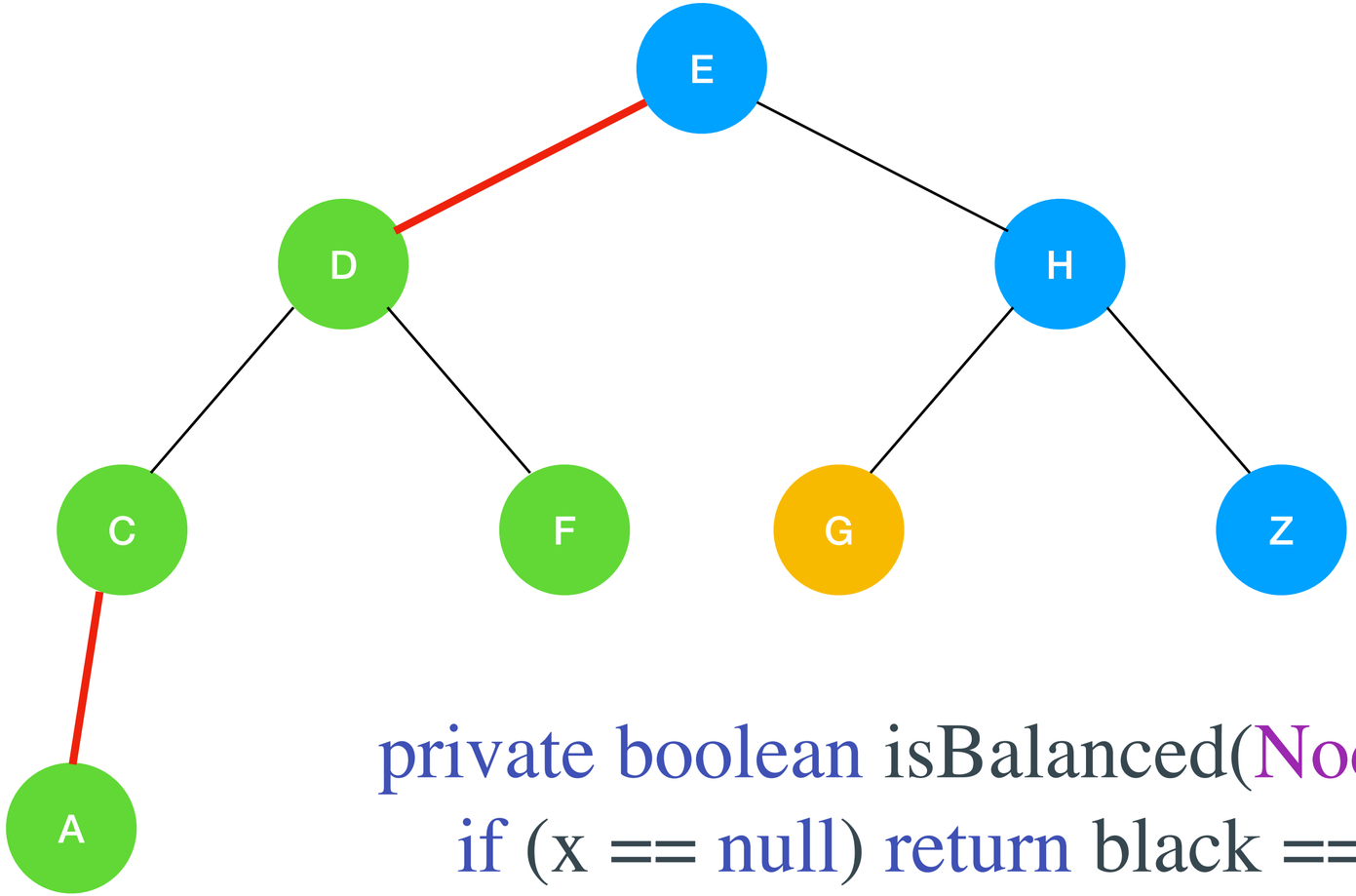


Node = H

Black = 0

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

Question 3.1.7 IS RED BLACK

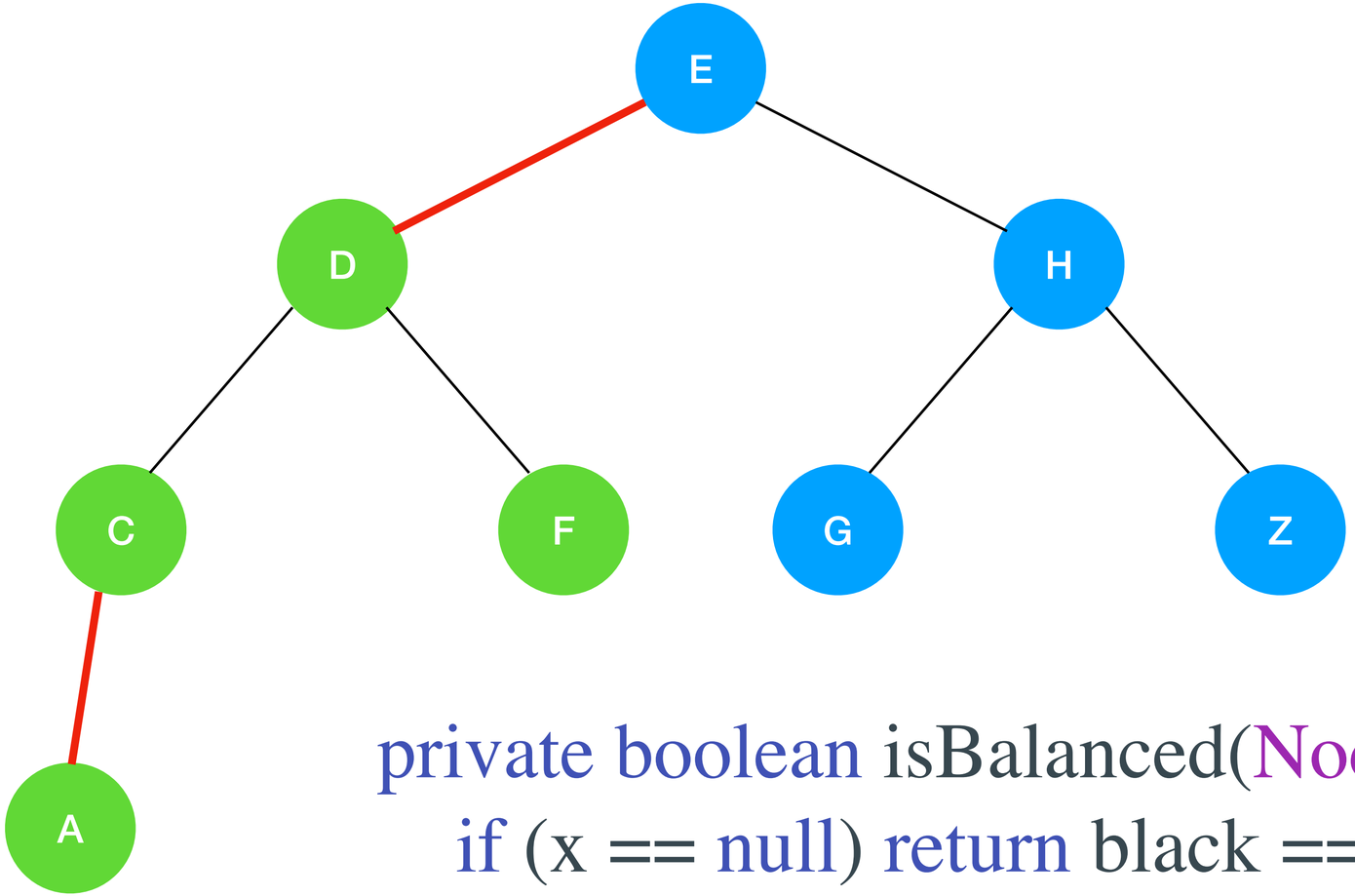


Node = G

Black = -1

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

Question 3.1.7 IS RED BLACK

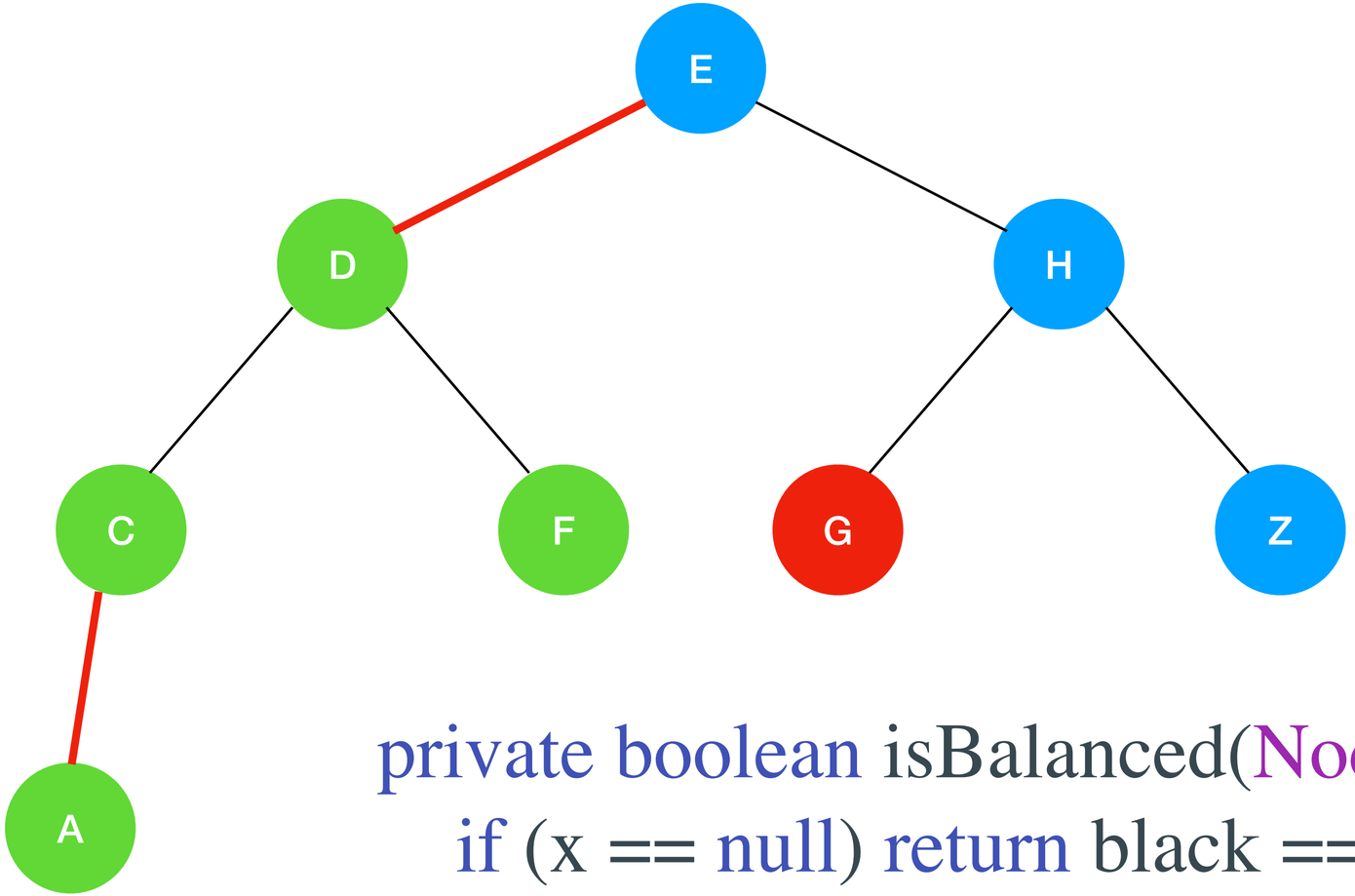


Node = null

Black = -1

```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

Question 3.1.7 IS RED BLACK



Node = G

Black = -1

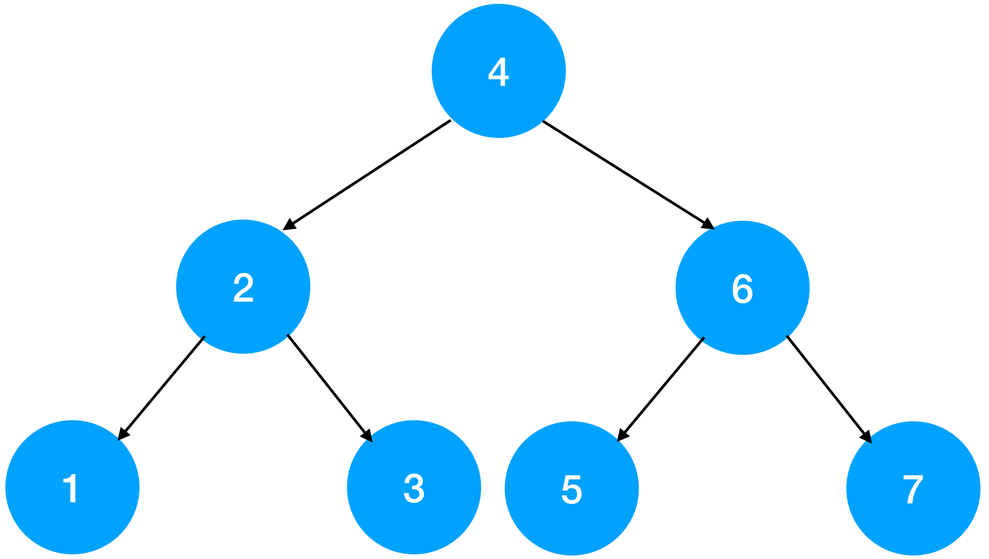
```
private boolean isBalanced(Node x, int black) {
    if (x == null) return black == 0;
    if (!isRed(x)) black--;
    return isBalanced(x.left, black) && isBalanced(x.right, black);
}
```

algorithm (data structure)	worst-case cost (after N inserts)		average-case cost (after N random inserts)		efficiently support ordered operations?
	search	insert	search hit	insert	
sequential search (unordered linked list)	N	N	$N/2$	N	no
binary search (ordered array)	$\lg N$	N	$\lg N$	$N/2$	yes
binary tree search (BST)	N	N	$1.39 \lg N$	$1.39 \lg N$	yes
2-3 tree search (red-black BST)	$2 \lg N$	$2 \lg N$	$1.00 \lg N$	$1.00 \lg N$	yes

Cost summary for symbol-table implementations (updated)

Question 3.1.11 : Pre-OrderTraversal

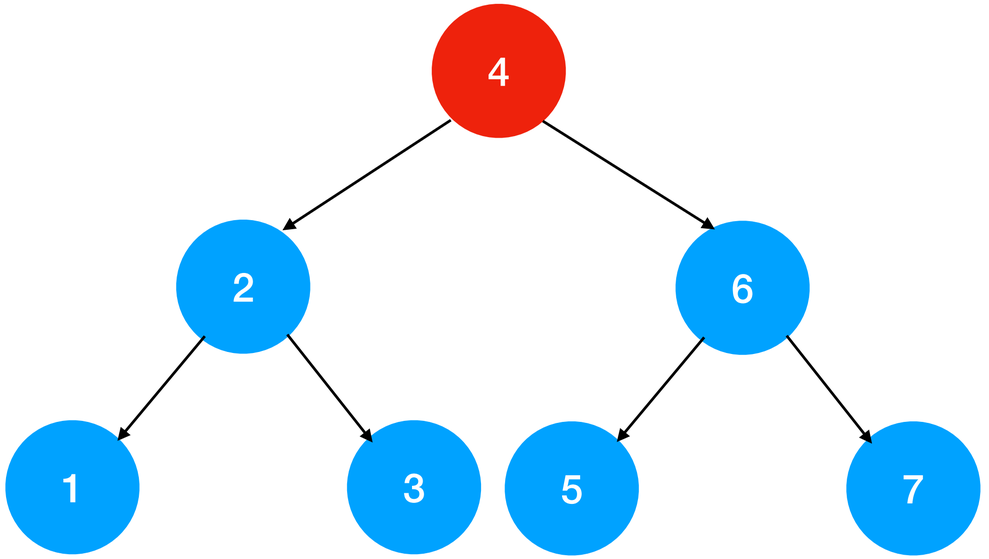
1. **Process the root**
2. **Process left node recursively**
3. **Process right node recursively**



Question 3.1.11 : Pre-OrderTraversal

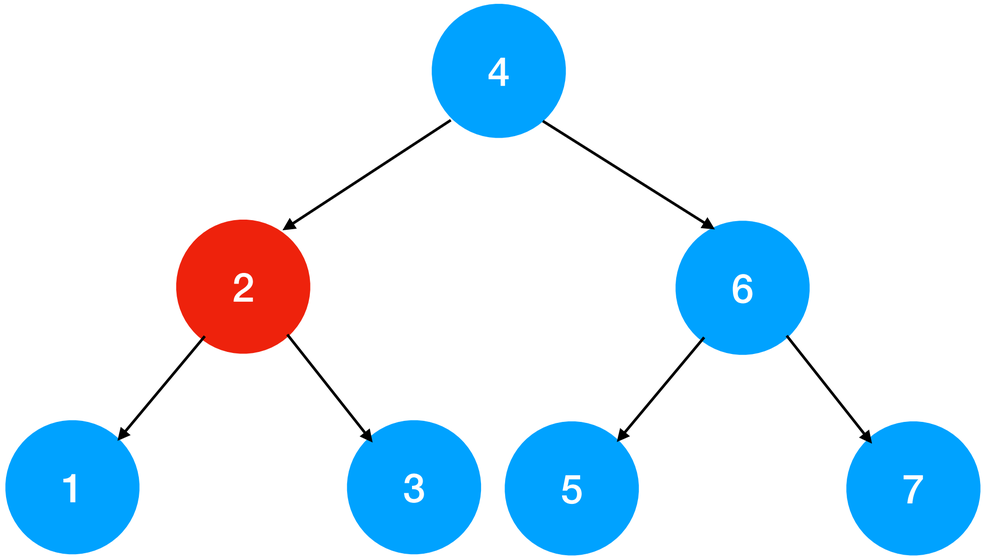
1. **Process the root**
2. **Process left node recursively**
3. **Process right node recursively**

4



Question 3.1.11 : Pre-OrderTraversal

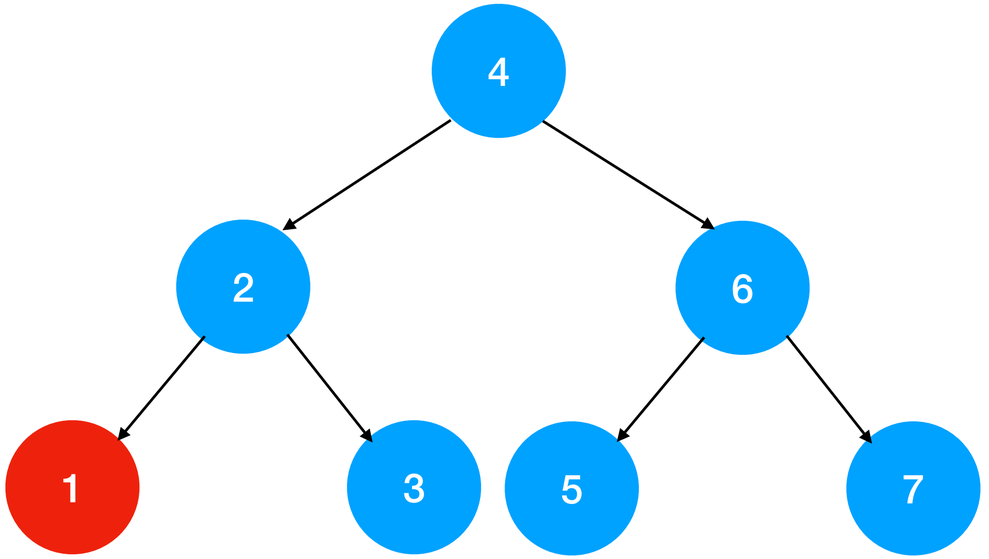
1. **Process the root**
2. **Process left node recursively**
3. **Process right node recursively**



4, 2

Question 3.1.11 : Pre-OrderTraversal

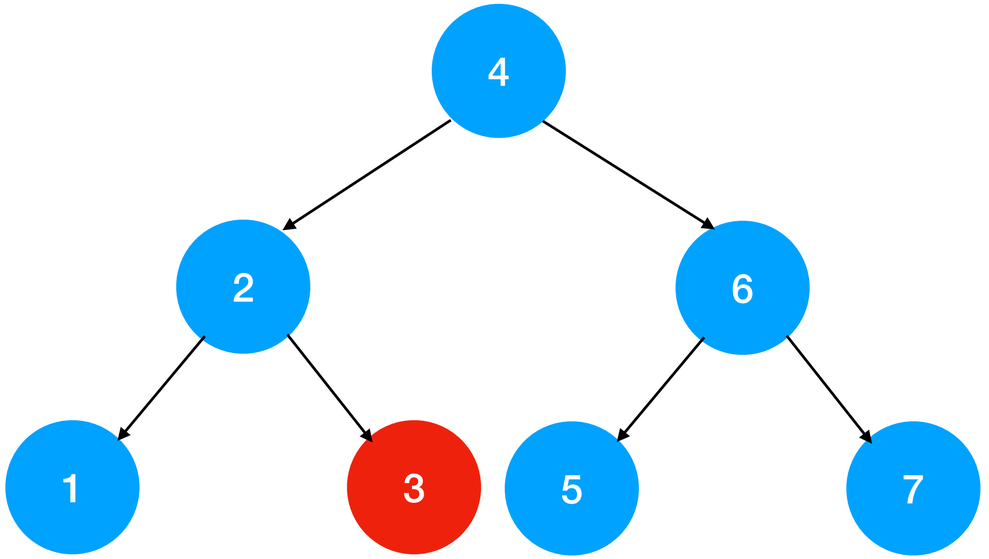
1. **Process the root**
2. **Process left node recursively**
3. **Process right node recursively**



4, 2, 1

Question 3.1.11 : Pre-OrderTraversal

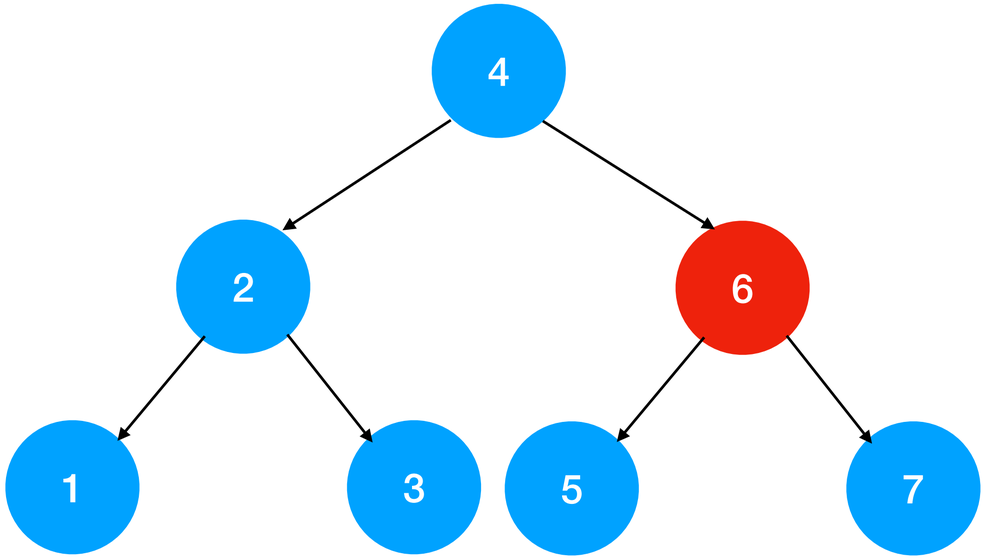
1. **Process the root**
2. **Process left node recursively**
3. **Process right node recursively**



4, 2, 1, 3

Question 3.1.11 : Pre-OrderTraversal

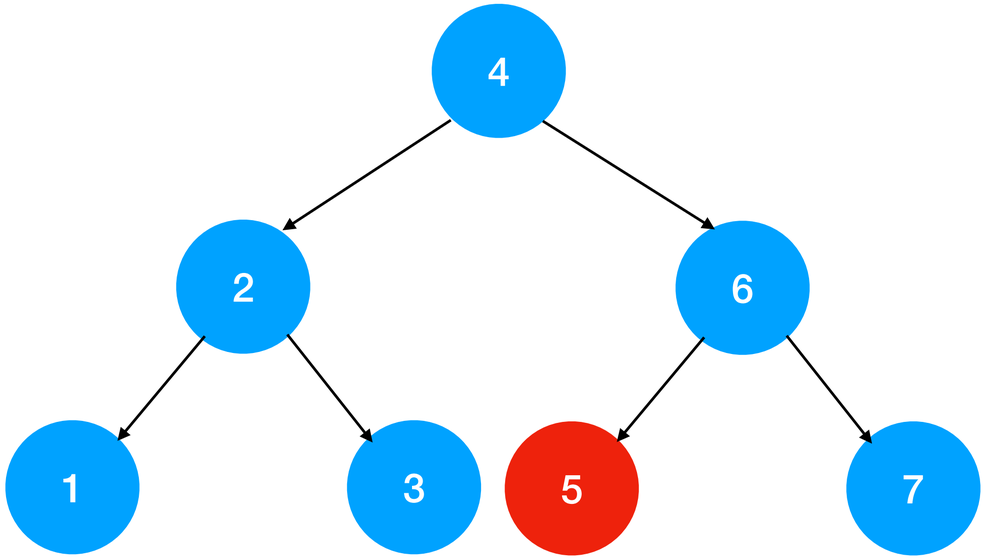
1. **Process the root**
2. **Process left node recursively**
3. **Process right node recursively**



4, 2, 1, 3, 6

Question 3.1.11 : Pre-OrderTraversal

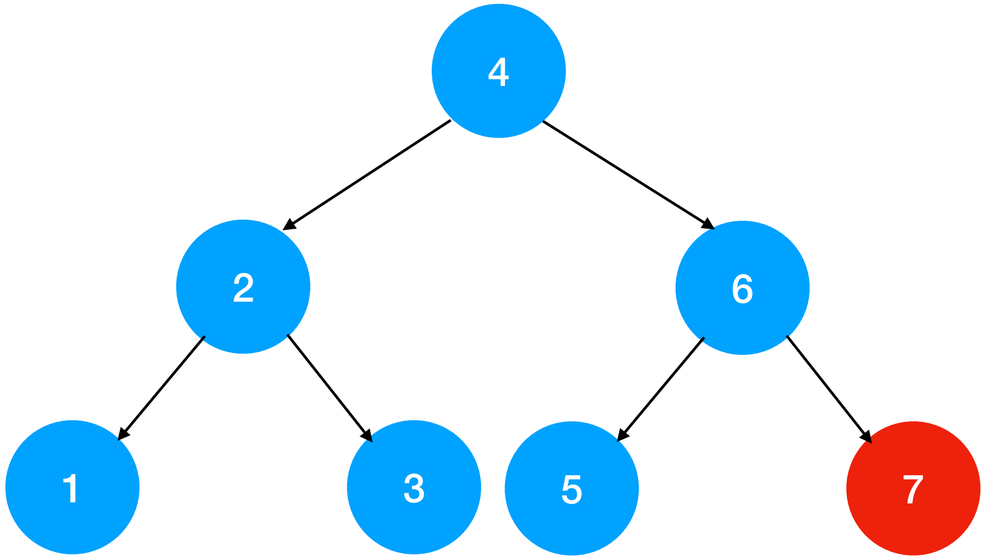
1. **Process the root**
2. **Process left node recursively**
3. **Process right node recursively**



4, 2, 1, 3, 6, 5

Question 3.1.11 : Pre-OrderTraversal

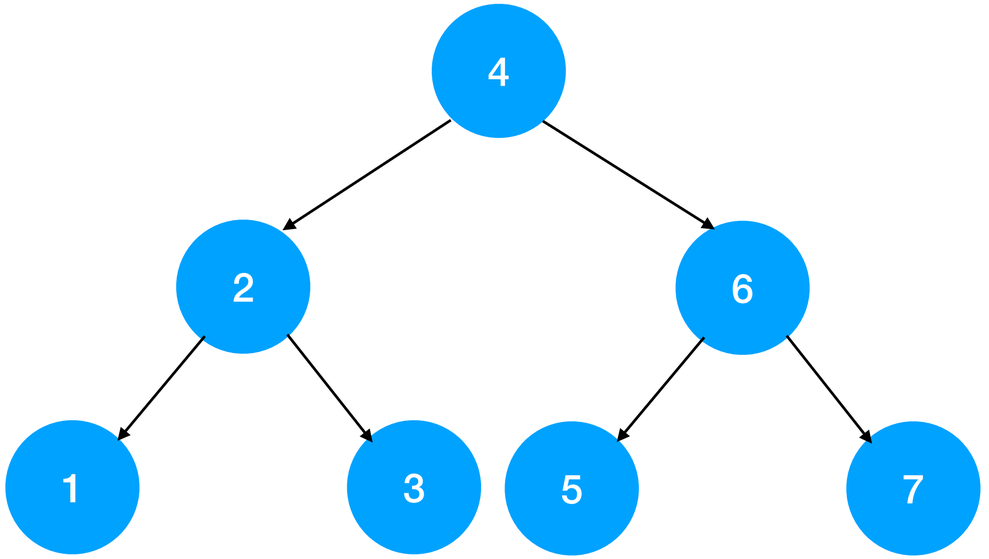
1. **Process the root**
2. **Process left node recursively**
3. **Process right node recursively**



4, 2, 1, 3, 6, 5, 7

Question 3.1.11 : In-Order Traversal

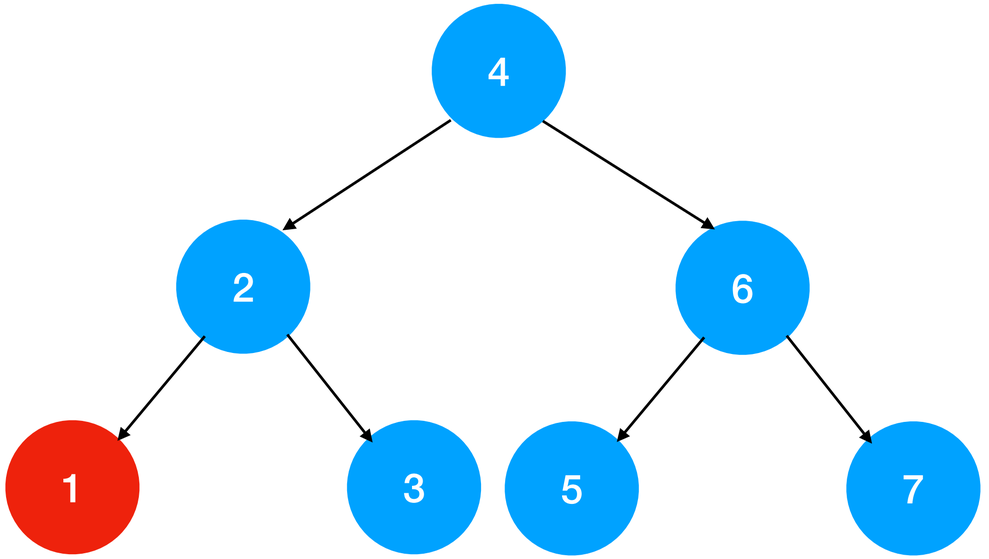
1. **Process left node recursively**
2. **Process the root**
3. **Process right node recursively**



Question 3.1.11 : In-Order Traversal

1. **Process left node recursively**
2. **Process the root**
3. **Process right node recursively**

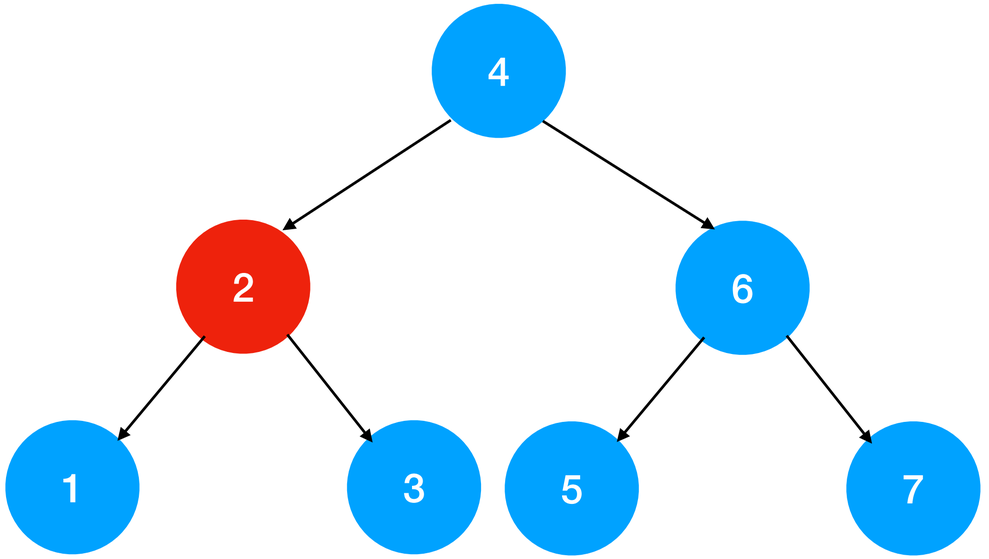
1



Question 3.1.11 : In-Order Traversal

1. **Process left node recursively**
2. **Process the root**
3. **Process right node recursively**

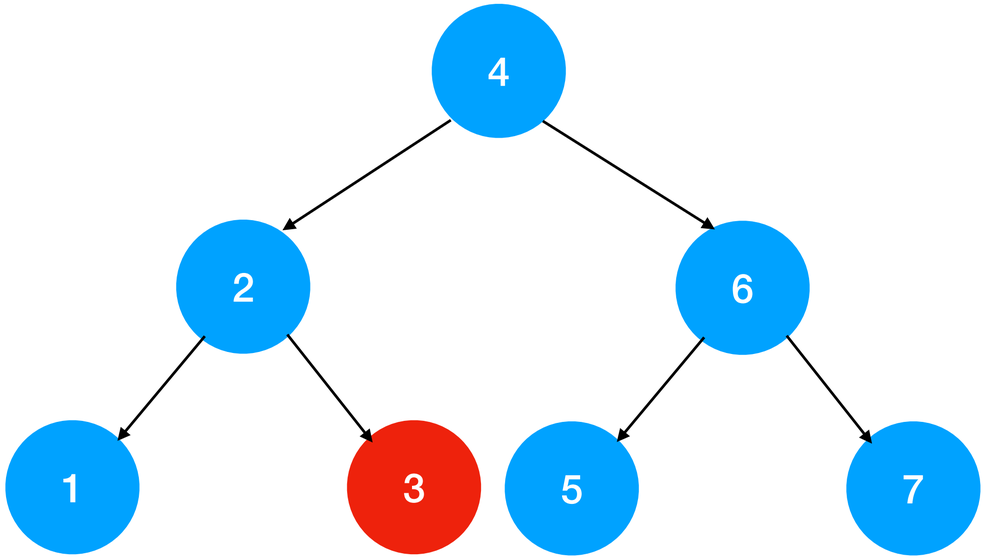
1, 2



Question 3.1.11 : In-Order Traversal

1. **Process left node recursively**
2. **Process the root**
3. **Process right node recursively**

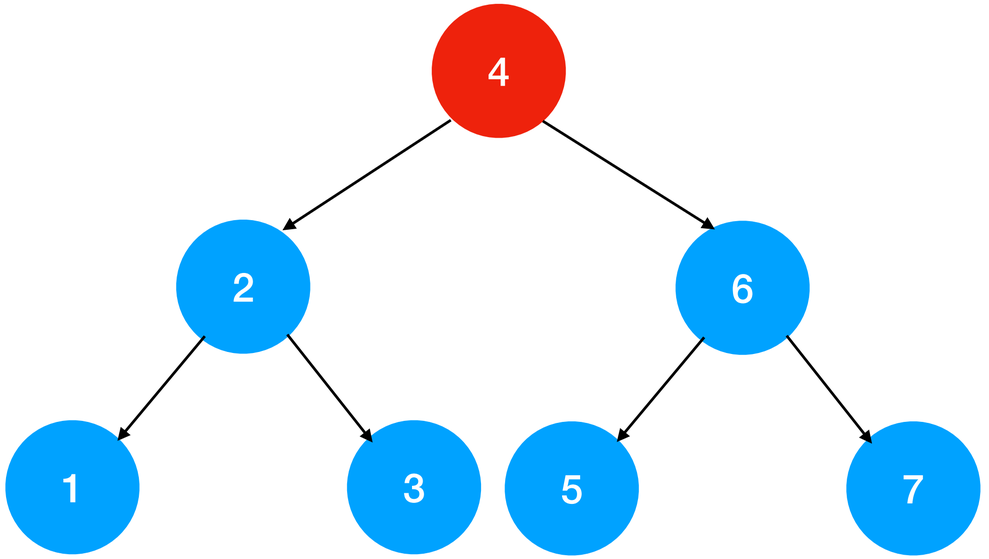
1, 2, 3



Question 3.1.11 : In-Order Traversal

1. Process left node recursively
2. Process the root
3. Process right node recursively

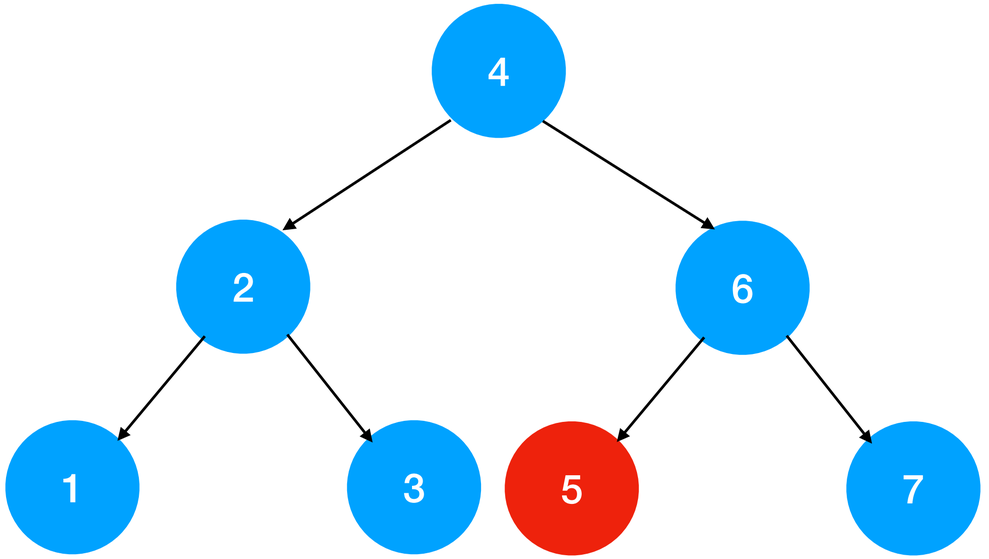
1, 2, 3, 4



Question 3.1.11 : In-Order Traversal

1. **Process left node recursively**
2. **Process the root**
3. **Process right node recursively**

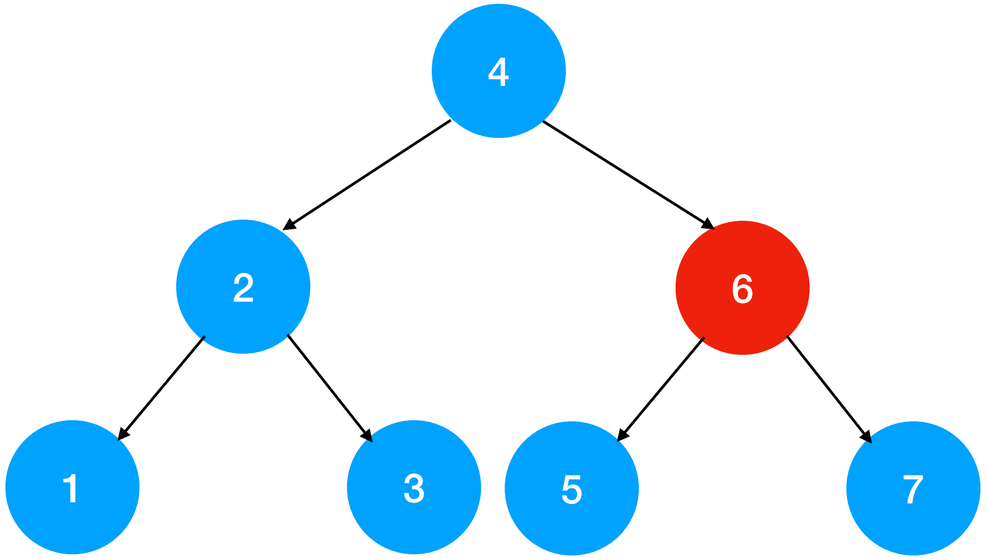
1, 2, 3, 4, 5



Question 3.1.11 : In-Order Traversal

1. **Process left node recursively**
2. **Process the root**
3. **Process right node recursively**

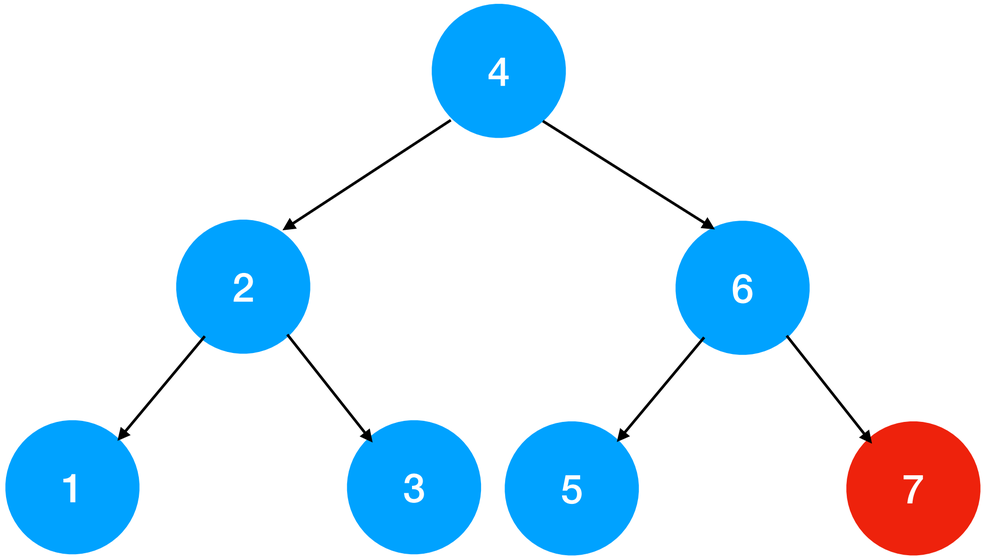
1, 2, 3, 4, 5, 6



Question 3.1.11 : In-Order Traversal

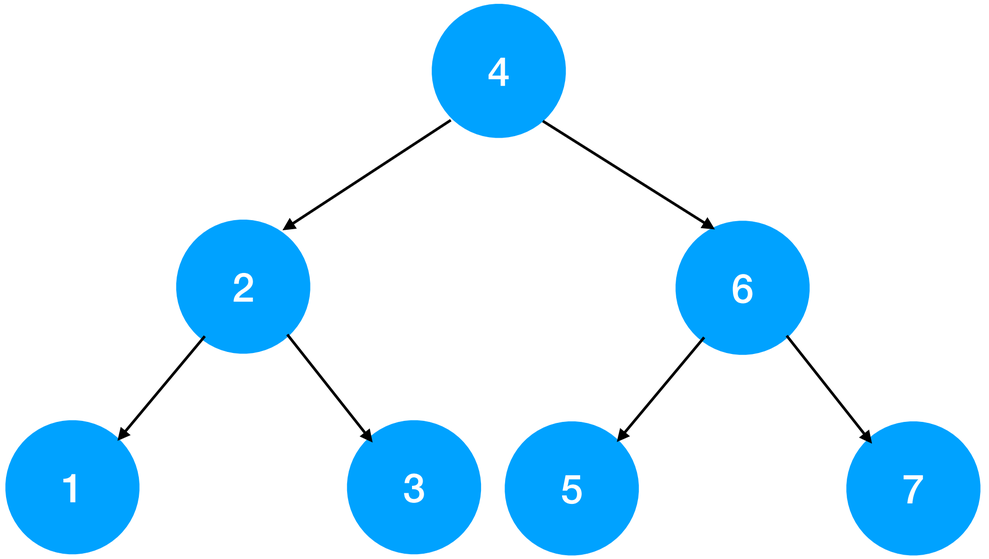
1. **Process left node recursively**
2. **Process the root**
3. **Process right node recursively**

1, 2, 3, 4, 5, 6, 7



Question 3.1.11 : Post-Order Traversal

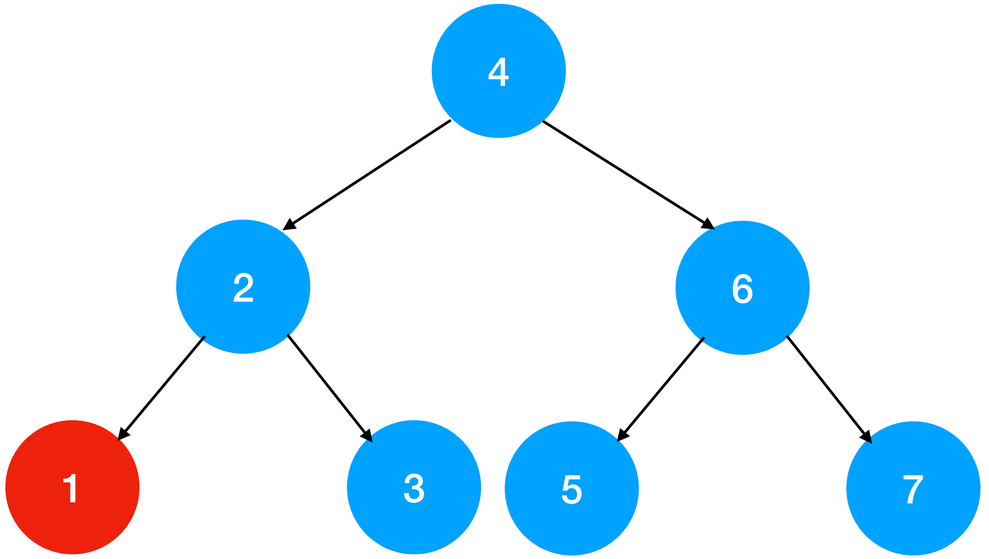
1. **Process left node recursively**
2. **Process right node recursively**
3. **Process the root**



Question 3.1.11 : Post-Order Traversal

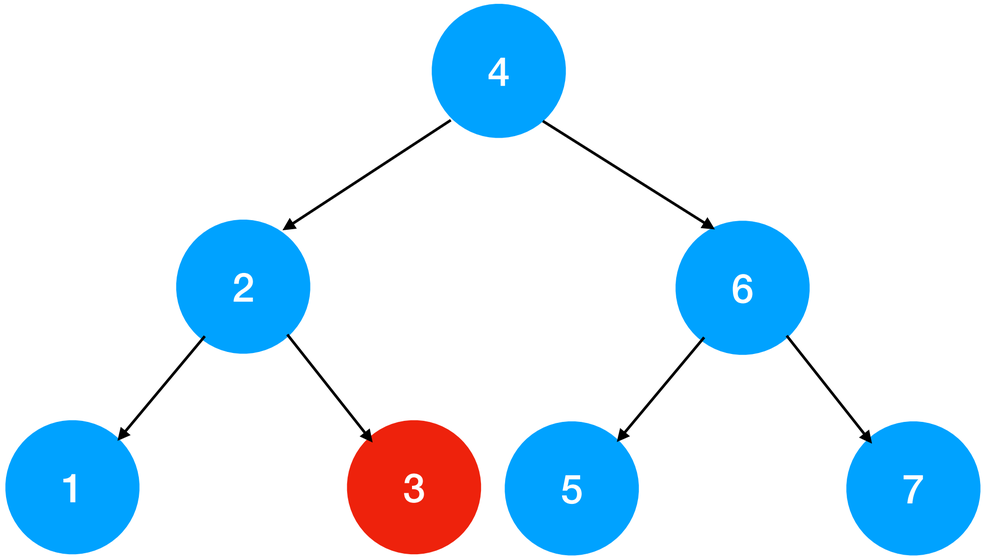
1. **Process left node recursively**
2. **Process right node recursively**
3. **Process the root**

1



Question 3.1.11 : Post-Order Traversal

1. **Process left node recursively**
2. **Process right node recursively**
3. **Process the root**

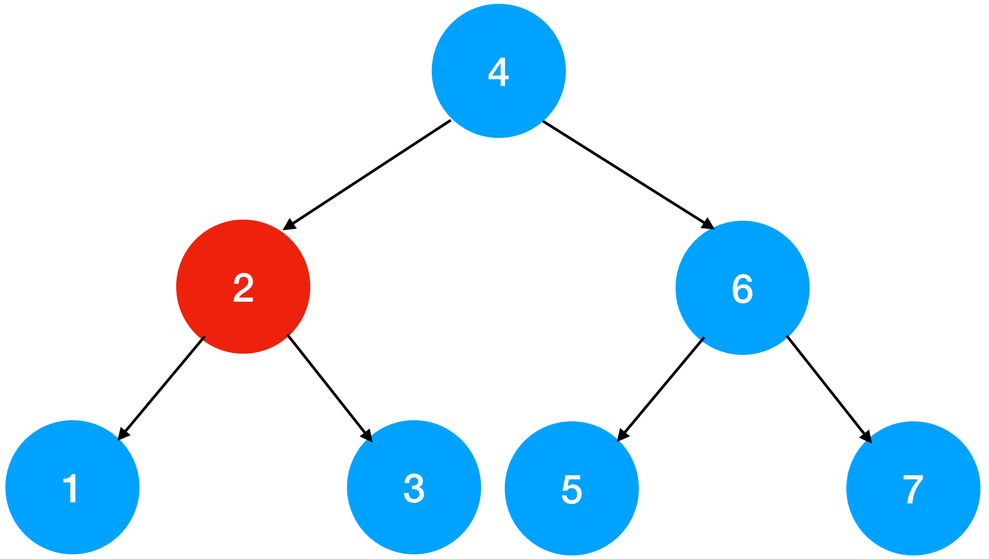


1, 3

Question 3.1.11 : Post-Order Traversal

1. **Process left node recursively**
2. **Process right node recursively**
3. **Process the root**

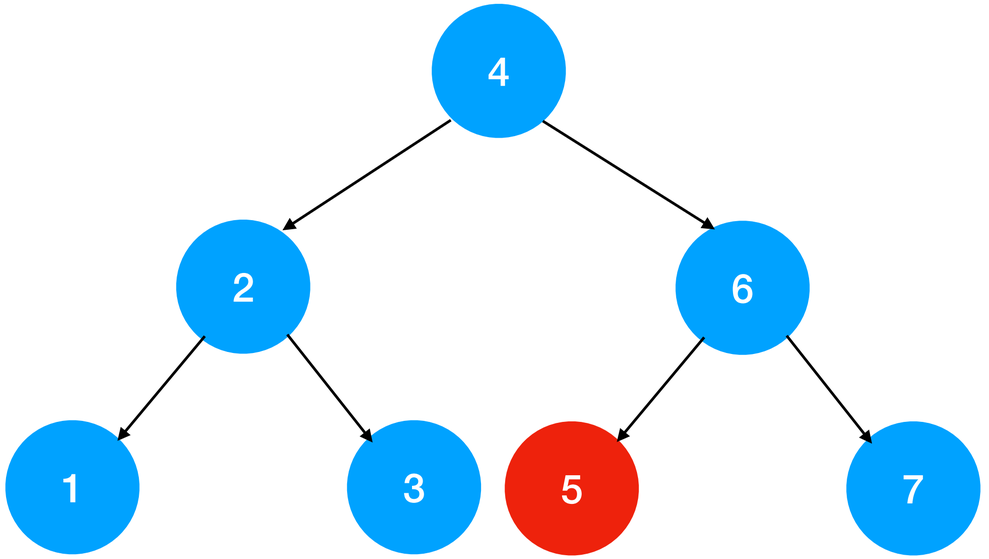
1, 3, 2



Question 3.1.11 : Post-Order Traversal

1. **Process left node recursively**
2. **Process right node recursively**
3. **Process the root**

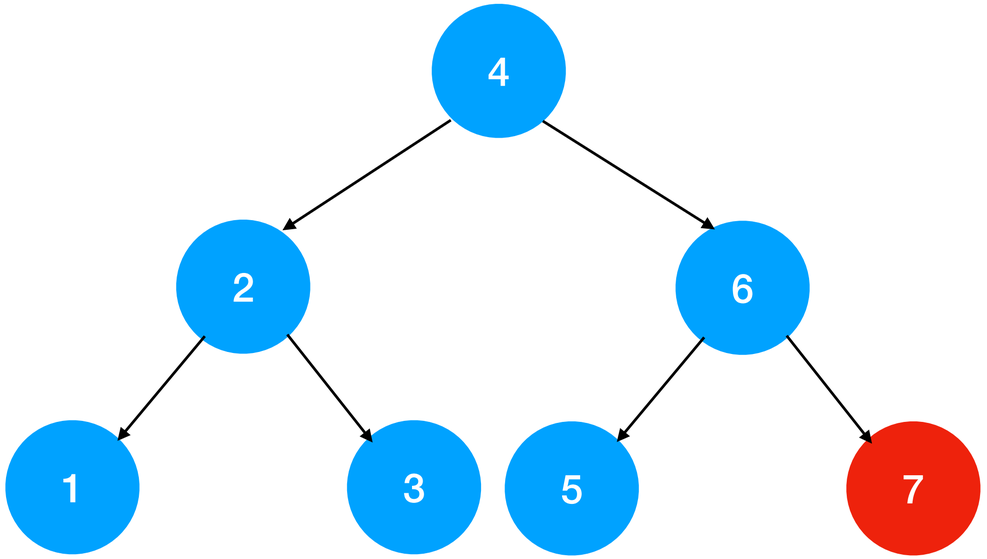
1, 3, 2, 5



Question 3.1.11 : Post-Order Traversal

1. **Process left node recursively**
2. **Process right node recursively**
3. **Process the root**

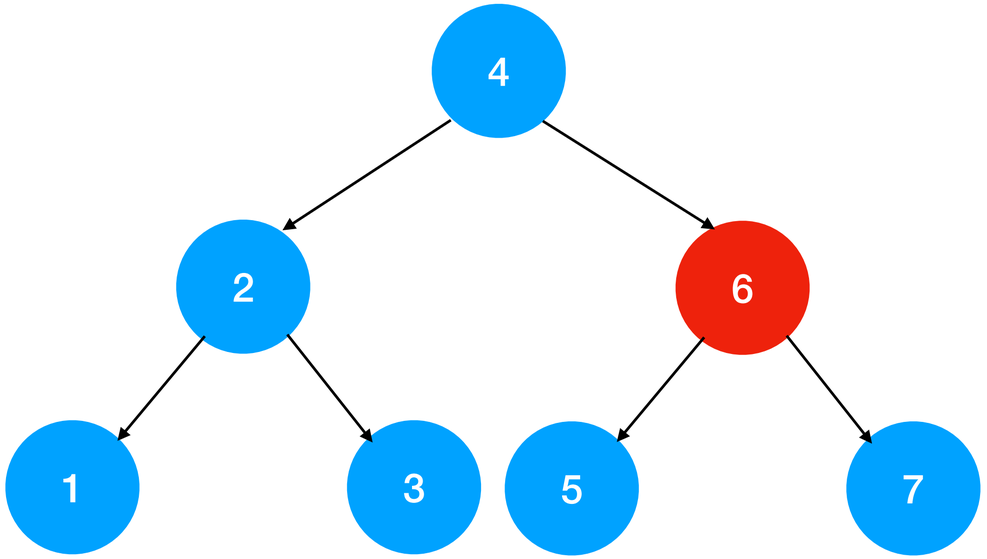
1, 3, 2, 5, 7



Question 3.1.11 : Post-Order Traversal

1. **Process left node recursively**
2. **Process right node recursively**
3. **Process the root**

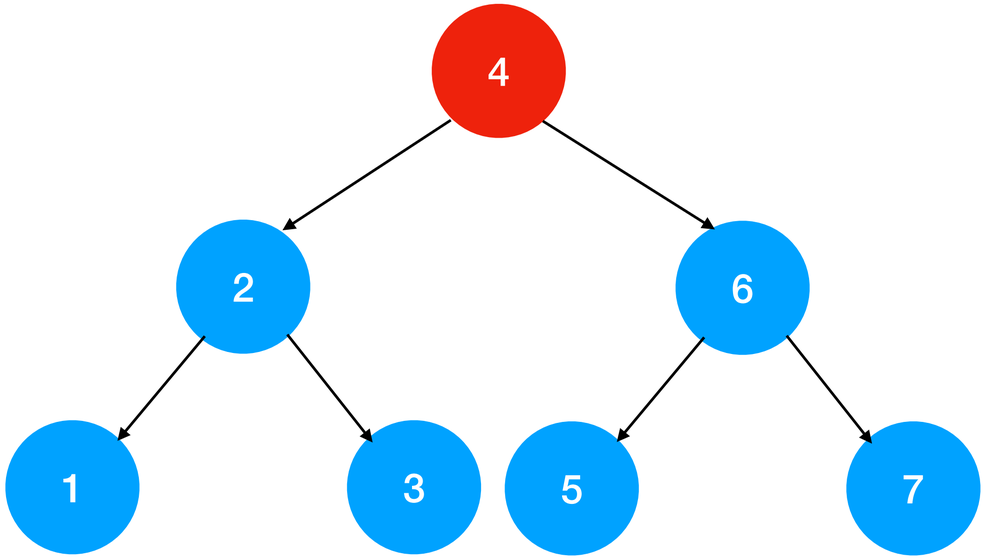
1, 3, 2, 5, 7, 6



Question 3.1.11 : Post-Order Traversal

1. **Process left node recursively**
2. **Process right node recursively**
3. **Process the root**

1, 3, 2, 5, 7, 6, 4



3.1.12 Inorder-Iterator (visit the keys in increasing order)

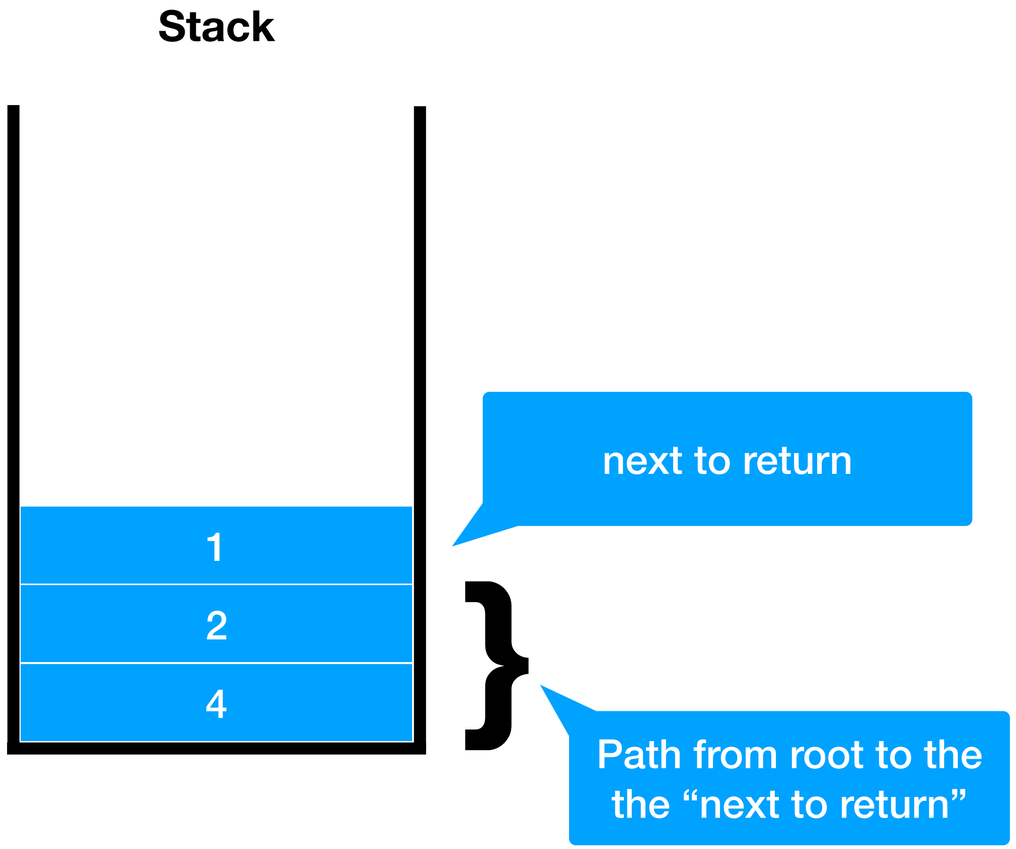
Reminder: Fail-Fast vs Fail-Safe iterator:

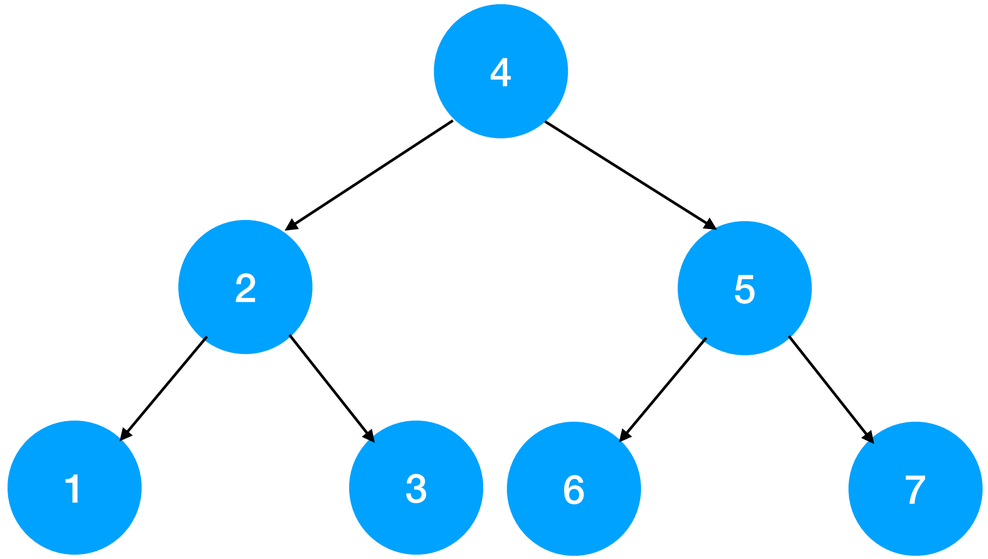
- Fail-Fast:
  - When there is a concurrent modification -> throws an ConcurrentModification Exception
  - Allows to quickly detect errors to debug them
- Fail-Safe:
  - The iterator runs on a copy of the data
  - When there is a concurrent modification -> doesn’t fail but iterates on outdated data

•

3.1.12 Inorder-Iterator (visit the keys in increasing order)

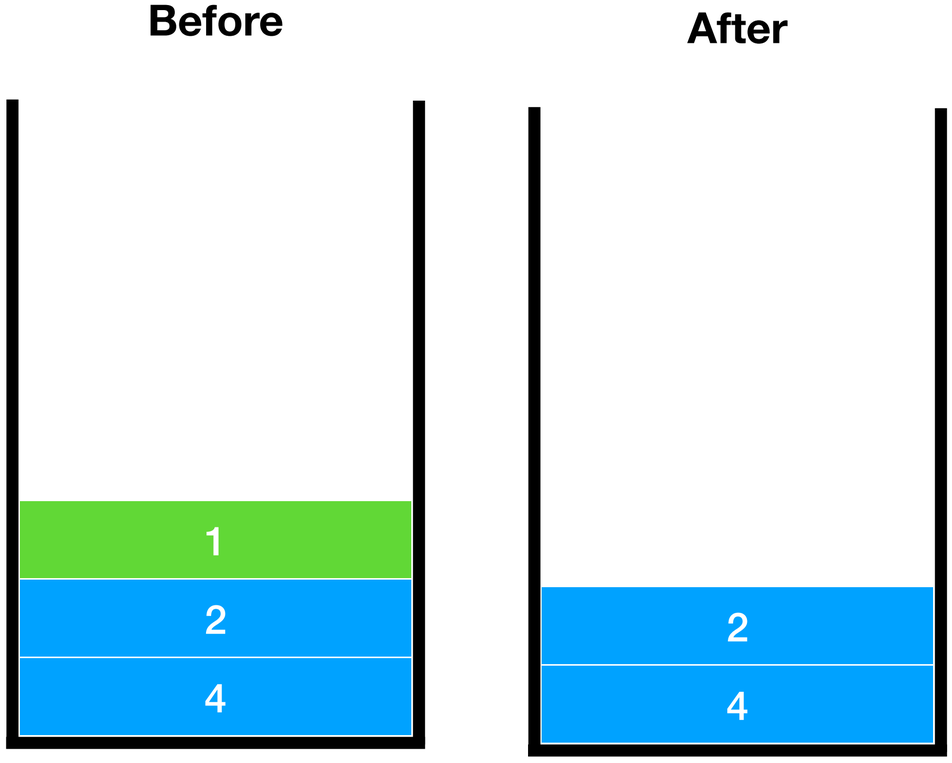
```
public BSTIterator() {
        this.stack = new Stack<>();
        BSTNode<Key> current = root;
        while (current != null) {
            this.stack.push(current);
            current = current.getLeft();
        }
        this.size = size();
    }
```

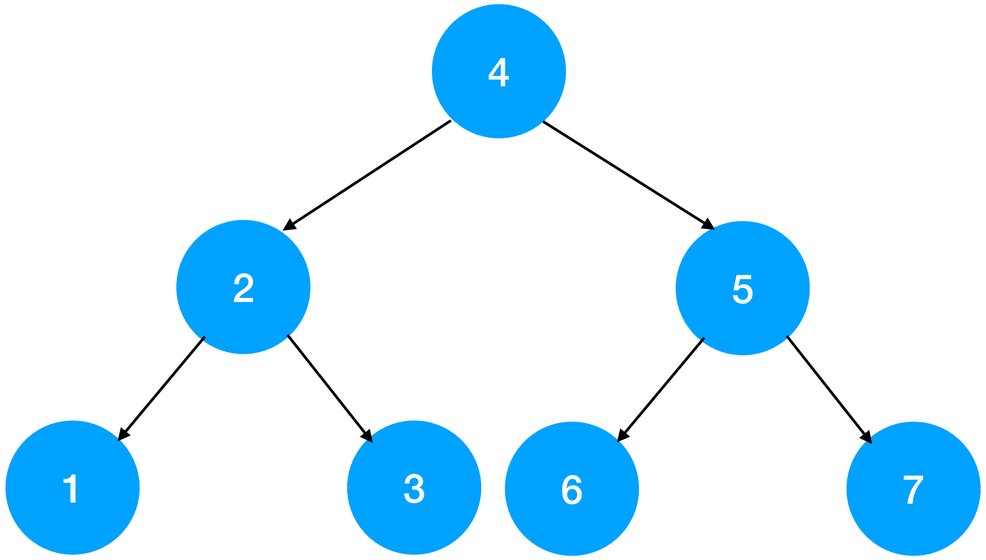




3.1.12 Inorder-Iterator (visit the keys in increasing order)

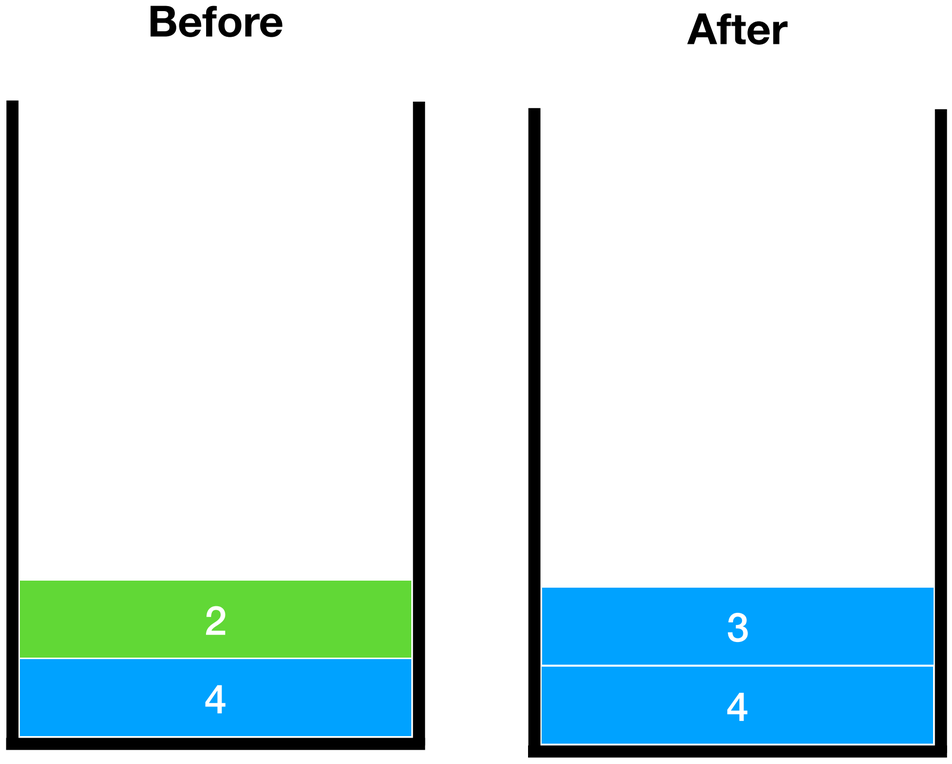
```
public Key next() {
    if (this.size != size()) {
        throw new ConcurrentModificationException();
    }
    if (!this.hasNext()) {
        throw new NoSuchElementException();
    }
    BSTNode<Key> node = this.stack.pop();
    Key key = node.getKey();
    if (node.getRight() != null) {
        node = node.getRight();
        while (node != null) {
            this.stack.push(node);
            node = node.getLeft();
        }
    }
    return key;
}
```

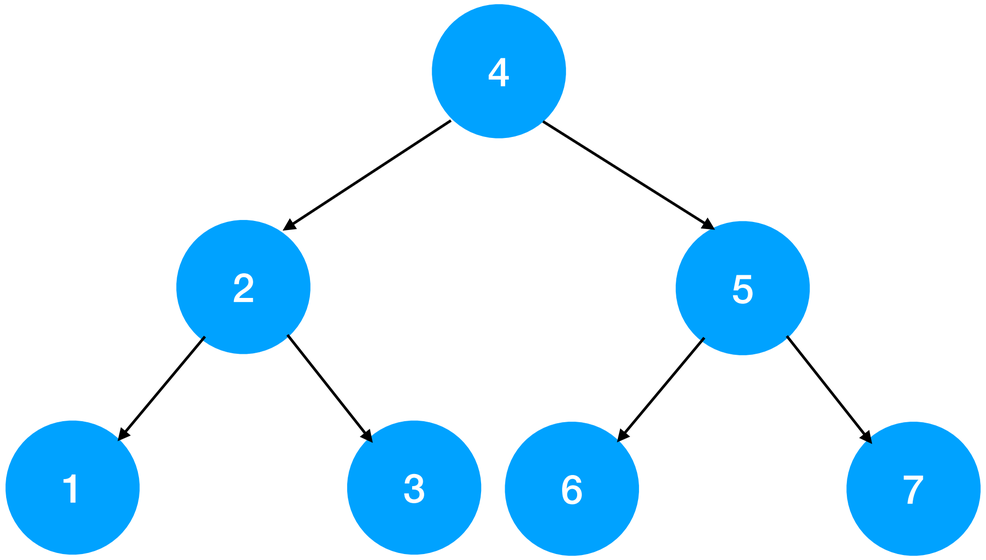




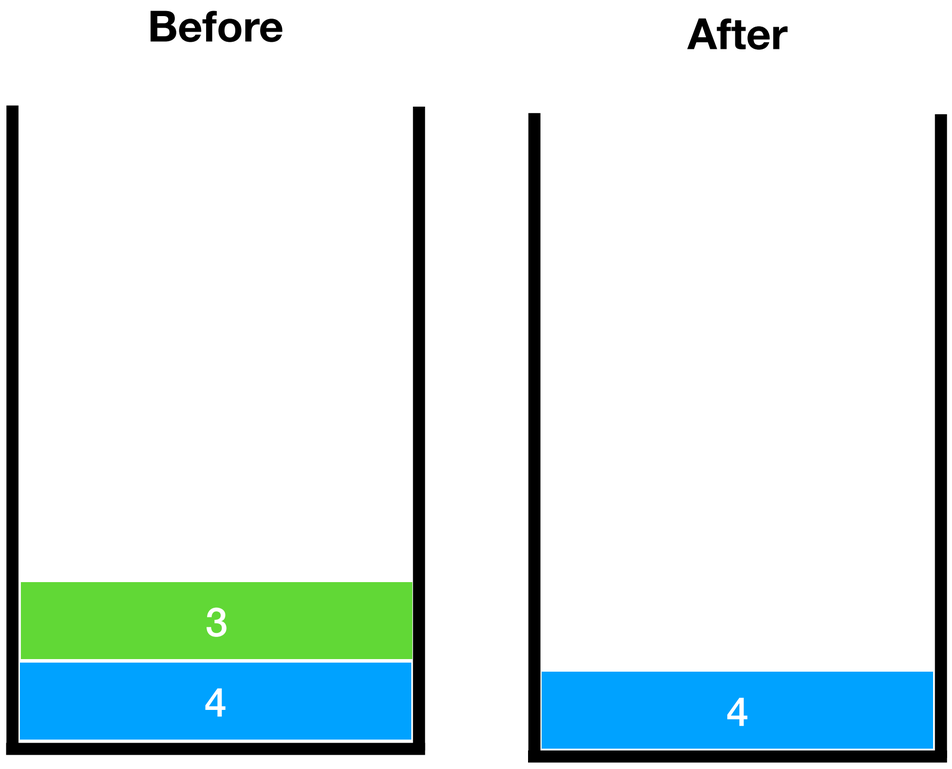
3.1.12 Inorder-Iterator (visit the keys in increasing order)

```
public Key next() {
    if (this.size != size()) {
        throw new ConcurrentModificationException();
    }
    if (!this.hasNext()) {
        throw new NoSuchElementException();
    }
    BSTNode<Key> node = this.stack.pop();
    Key key = node.getKey();
    if (node.getRight() != null) {
        node = node.getRight();
        while (node != null) {
            this.stack.push(node);
            node = node.getLeft();
        }
    }
    return key;
}
```

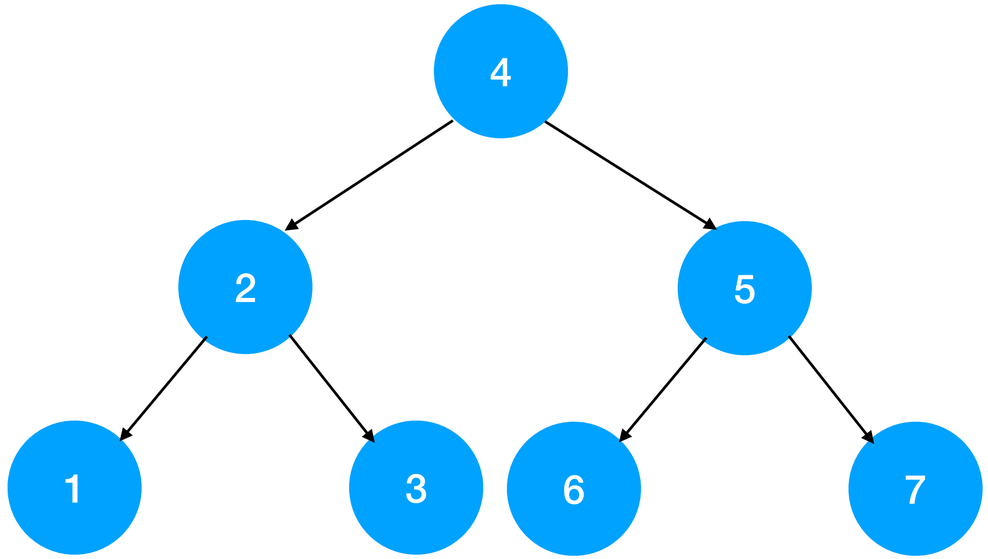




3.1.12 Inorder-Iterator (visit the keys in increasing order)

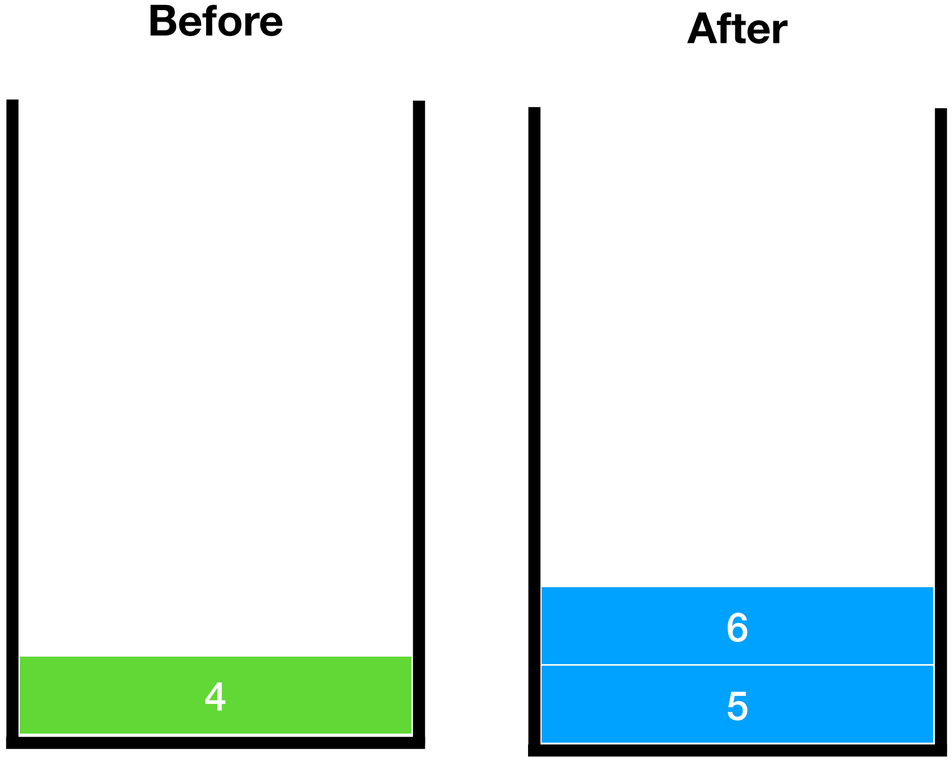


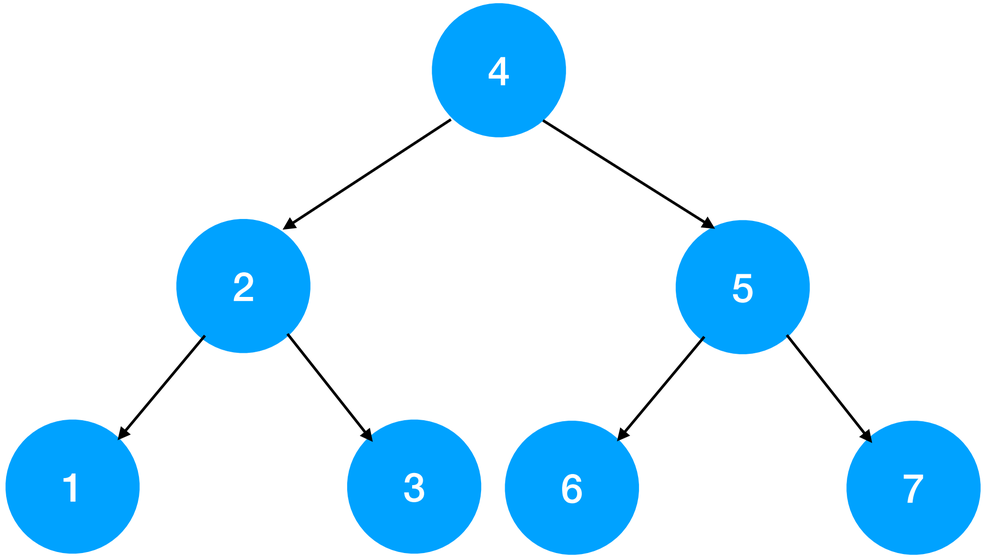

```
public Key next() {
    if (this.size != size()) {
        throw new ConcurrentModificationException();
    }
    if (!this.hasNext()) {
        throw new NoSuchElementException();
    }
    BSTNode<Key> node = this.stack.pop();
    Key key = node.getKey();
    if (node.getRight() != null) {
        node = node.getRight();
        while (node != null) {
            this.stack.push(node);
            node = node.getLeft();
        }
    }
    return key;
}
```



3.1.12 Inorder-Iterator (visit the keys in increasing order)

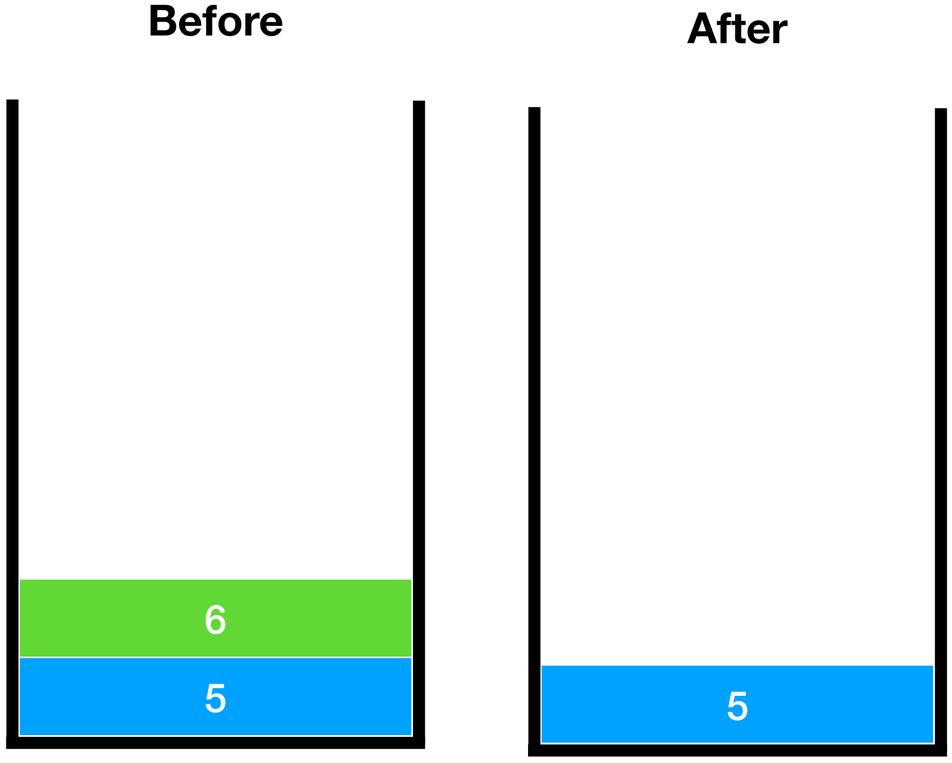
```
public Key next() {
    if (this.size != size()) {
        throw new ConcurrentModificationException();
    }
    if (!this.hasNext()) {
        throw new NoSuchElementException();
    }
    BSTNode<Key> node = this.stack.pop();
    Key key = node.getKey();
    if (node.getRight() != null) {
        node = node.getRight();
        while (node != null) {
            this.stack.push(node);
            node = node.getLeft();
        }
    }
    return key;
}
```

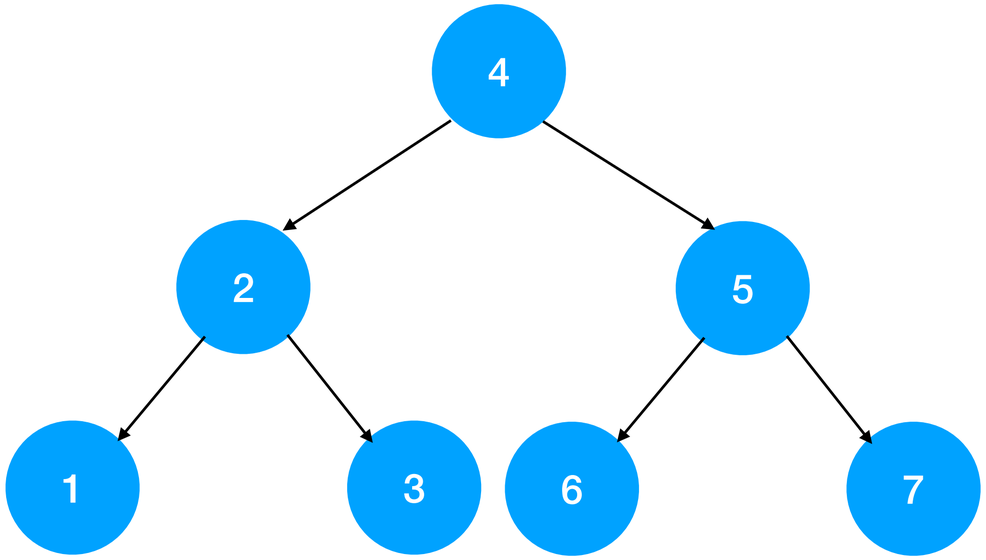




3.1.12 Inorder-Iterator (visit the keys in increasing order)

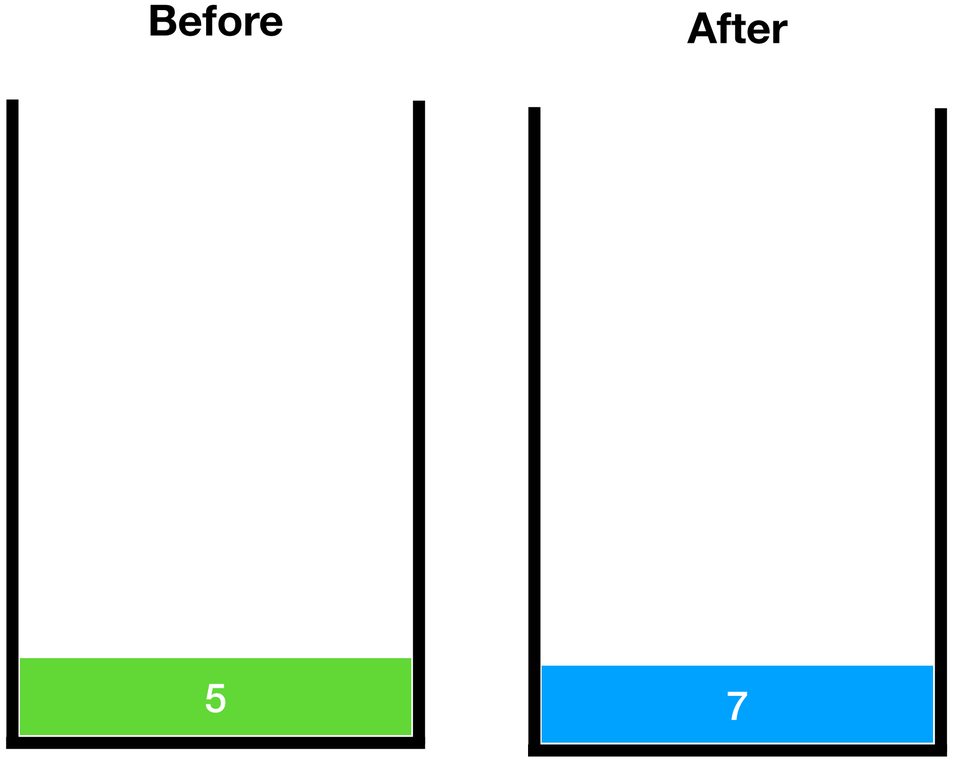
```
public Key next() {
    if (this.size != size()) {
        throw new ConcurrentModificationException();
    }
    if (!this.hasNext()) {
        throw new NoSuchElementException();
    }
    BSTNode<Key> node = this.stack.pop();
    Key key = node.getKey();
    if (node.getRight() != null) {
        node = node.getRight();
        while (node != null) {
            this.stack.push(node);
            node = node.getLeft();
        }
    }
    return key;
}
```

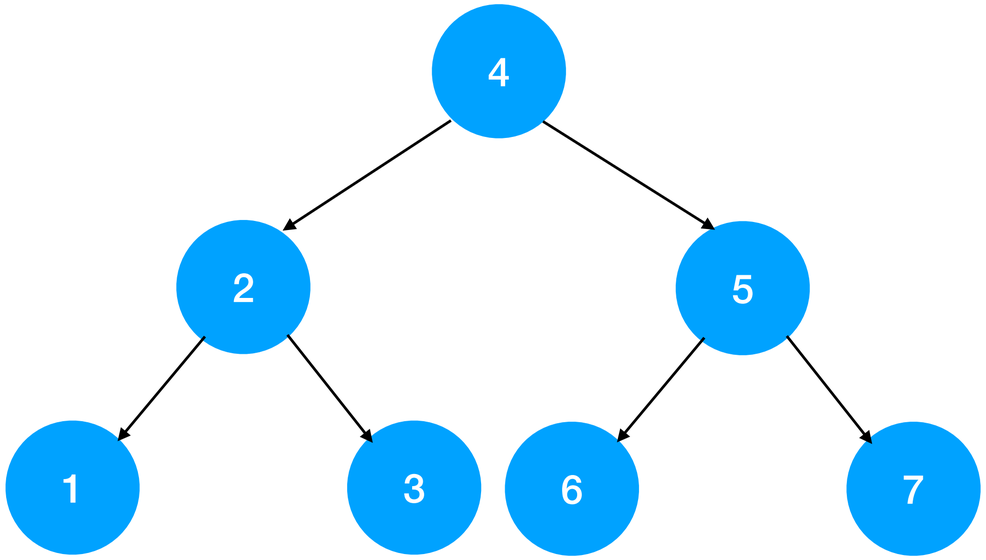




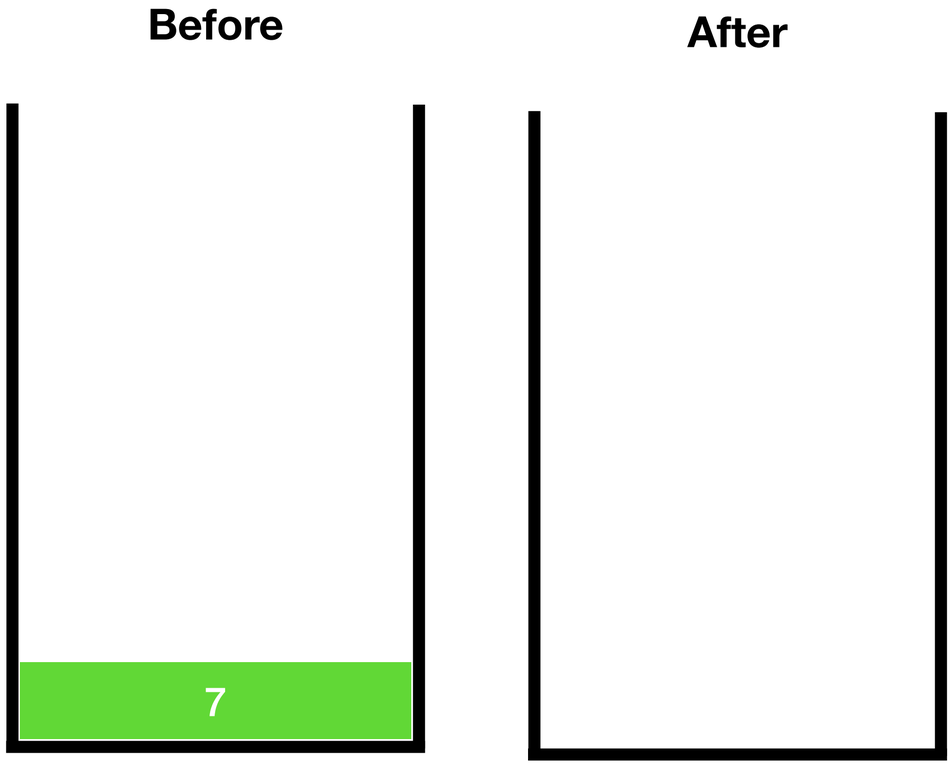
3.1.12 Inorder-Iterator (visit the keys in increasing order)

```
public Key next() {
    if (this.size != size()) {
        throw new ConcurrentModificationException();
    }
    if (!this.hasNext()) {
        throw new NoSuchElementException();
    }
    BSTNode<Key> node = this.stack.pop();
    Key key = node.getKey();
    if (node.getRight() != null) {
        node = node.getRight();
        while (node != null) {
            this.stack.push(node);
            node = node.getLeft();
        }
    }
    return key;
}
```

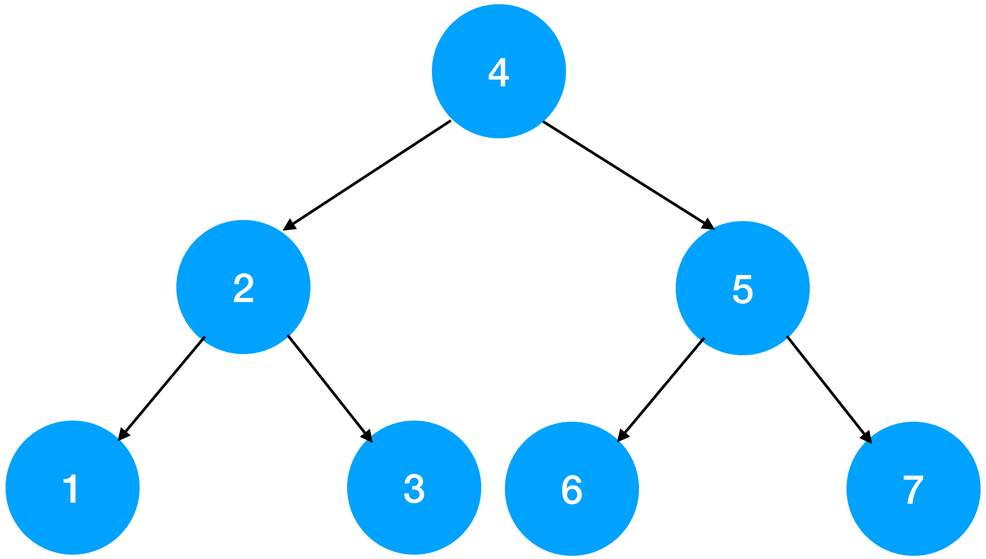




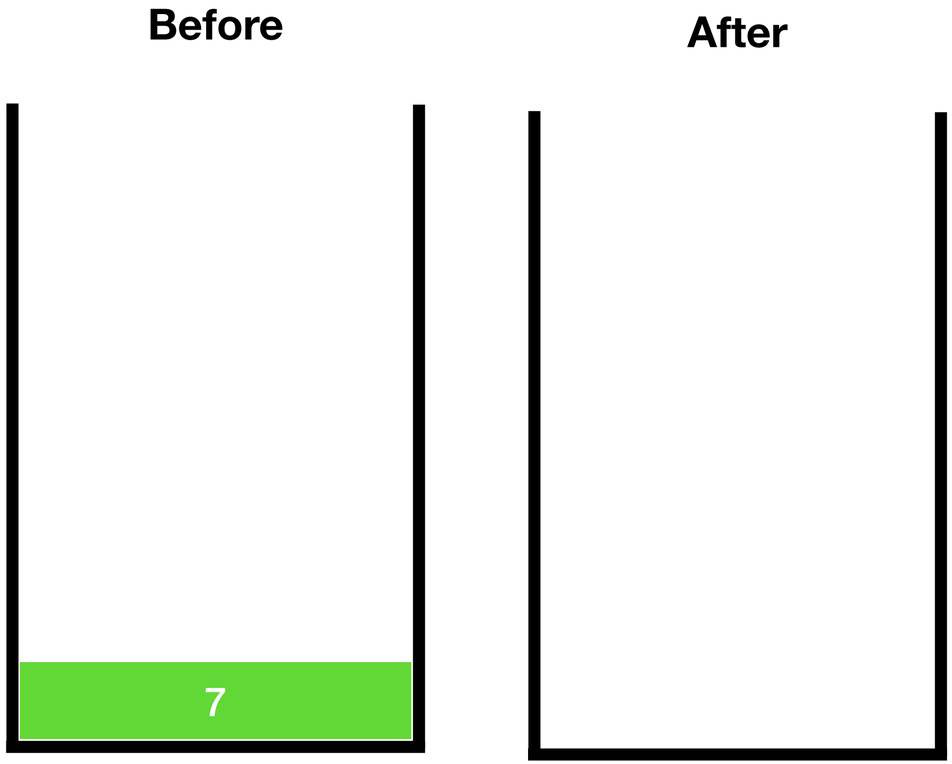
3.1.12 Inorder-Iterator (visit the keys in increasing order)



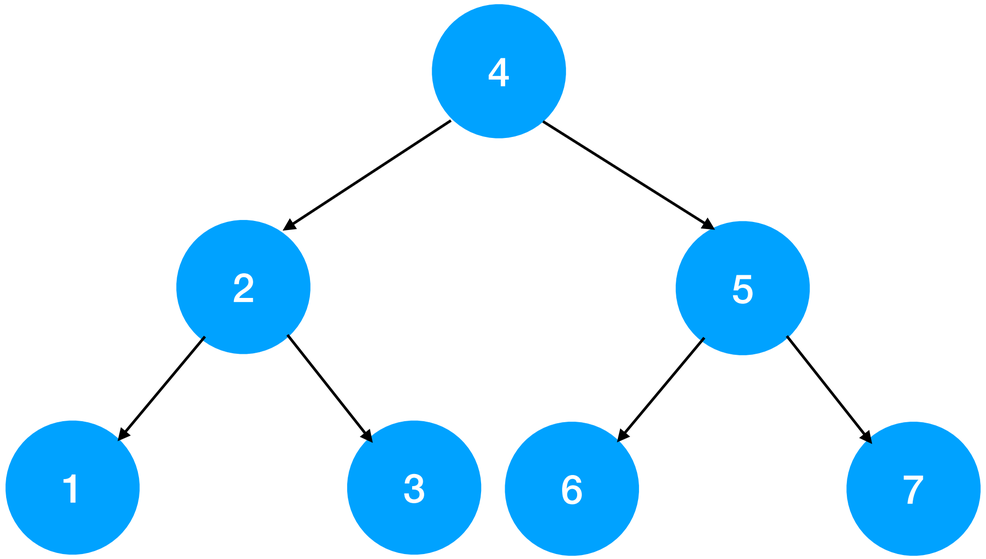

```
public Key next() {
    if (this.size != size()) {
        throw new ConcurrentModificationException();
    }
    if (!this.hasNext()) {
        throw new NoSuchElementException();
    }
    BSTNode<Key> node = this.stack.pop();
    Key key = node.getKey();
    if (node.getRight() != null) {
        node = node.getRight();
        while (node != null) {
            this.stack.push(node);
            node = node.getLeft();
        }
    }
    return key;
}
```



3.1.12 Inorder-Iterator (visit the keys in increasing order)



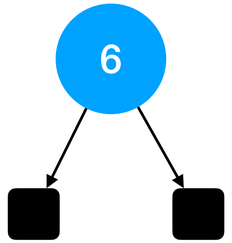

```
public Key next() {
    if (this.size != size()) {
        throw new ConcurrentModificationException();
    }
    if (!this.hasNext()) {
        throw new NoSuchElementException();
    }
    BSTNode<Key> node = this.stack.pop();
    Key key = node.getKey();
    if (node.getRight() != null) {
        node = node.getRight();
        while (node != null) {
            this.stack.push(node);
            node = node.getLeft();
        }
    }
    return key;
}
```



Array BST

```
public ArrayList<Key> keys;
public ArrayList<Value> values;

public ArrayList<Integer> idxLeftNode; // idxLeftNode[i] = index of left node of i
public ArrayList<Integer> idxRightNode; // idxRightNode[i] = index of right node of i
```

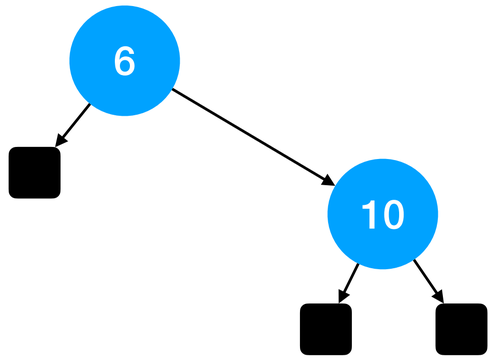


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Key	6														
Left	-														
Right	-														

Array BST

```
public ArrayList<Key> keys;
public ArrayList<Value> values;

public ArrayList<Integer> idxLeftNode; // idxLeftNode[i] = index of left node of i
public ArrayList<Integer> idxRightNode; // idxRightNode[i] = index of right node of i
```

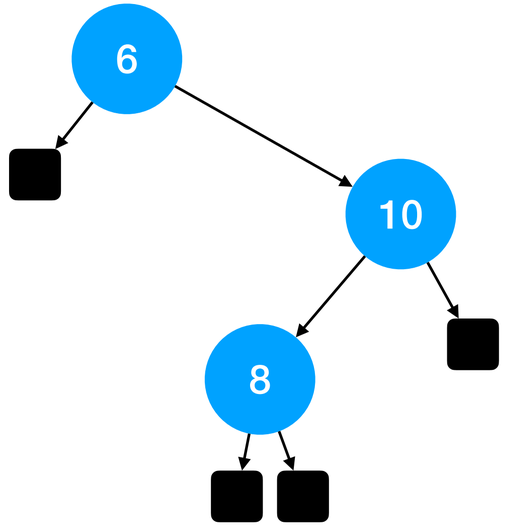


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Key	6	10													
Left	-	-													
Right	1	-													

Array BST

```
public ArrayList<Key> keys;
public ArrayList<Value> values;

public ArrayList<Integer> idxLeftNode; // idxLeftNode[i] = index of left node of i
public ArrayList<Integer> idxRightNode; // idxRightNode[i] = index of right node of i
```

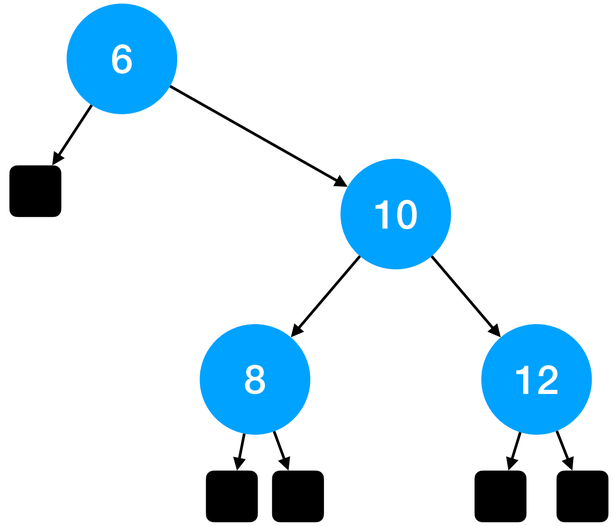


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Key	6	10	8												
Left	-	2	-												
Right	1	-	-												

Array BST

```
public ArrayList<Key> keys;
public ArrayList<Value> values;

public ArrayList<Integer> idxLeftNode; // idxLeftNode[i] = index of left node of i
public ArrayList<Integer> idxRightNode; // idxRightNode[i] = index of right node of i
```

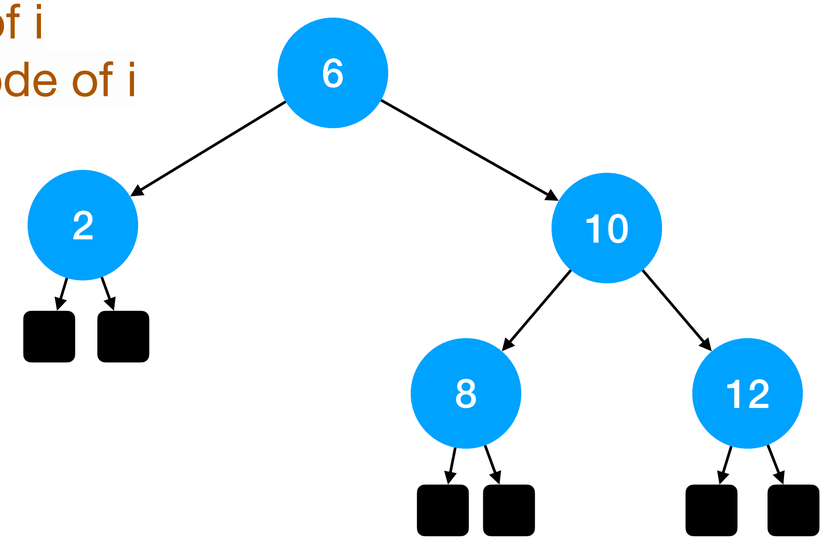


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Key	6	10	8	12											
Left	-	2	-	-											
Right	1	3	-	-											

Array BST

```
public ArrayList<Key> keys;
public ArrayList<Value> values;

public ArrayList<Integer> idxLeftNode; // idxLeftNode[i] = index of left node of i
public ArrayList<Integer> idxRightNode; // idxRightNode[i] = index of right node of i
```

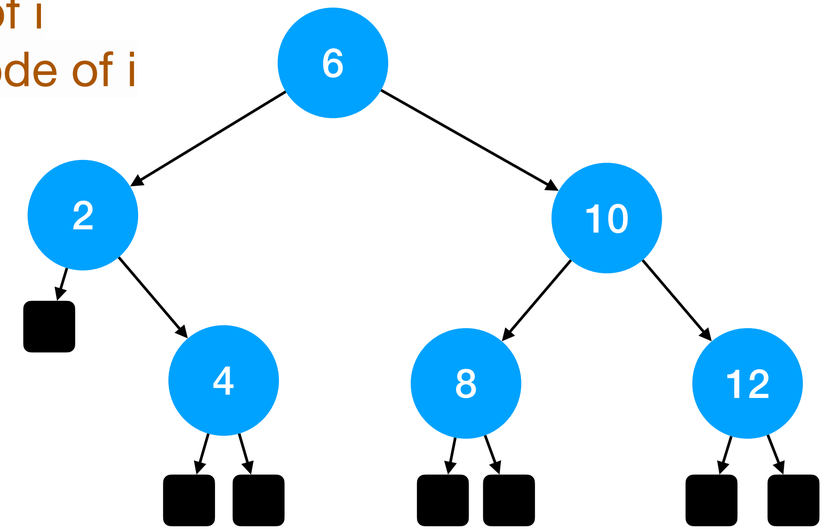


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Key	6	10	8	12	2										
Left	4	2	-	-	-										
Right	1	3	-	-	-										

Array BST

```
public ArrayList<Key> keys;
public ArrayList<Value> values;

public ArrayList<Integer> idxLeftNode; // idxLeftNode[i] = index of left node of i
public ArrayList<Integer> idxRightNode; // idxRightNode[i] = index of right node of i
```

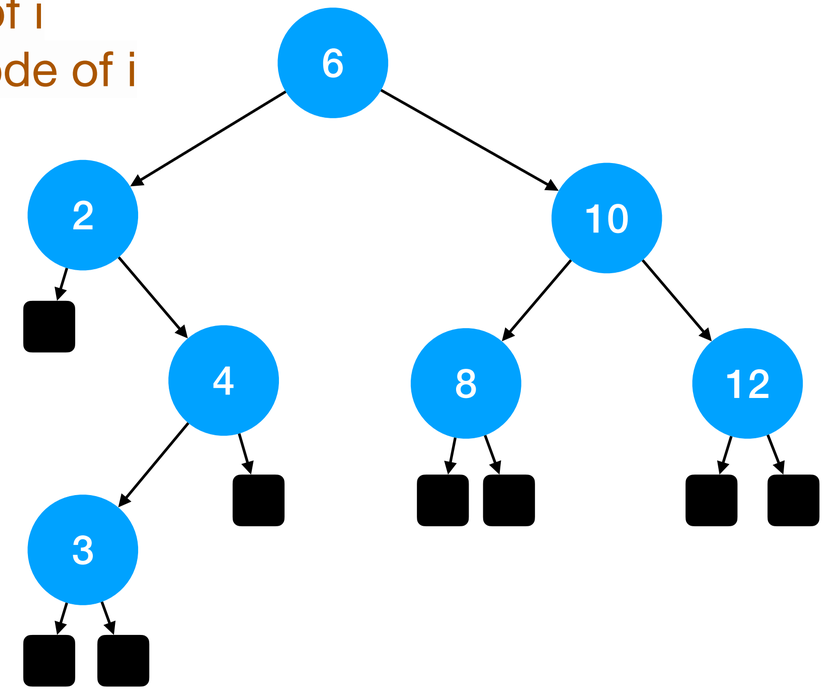


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Key	6	10	8	12	2	4									
Left	4	2	-	-	-	-									
Right	1	3	-	-	5	-									

Array BST

```
public ArrayList<Key> keys;
public ArrayList<Value> values;

public ArrayList<Integer> idxLeftNode; // idxLeftNode[i] = index of left node of i
public ArrayList<Integer> idxRightNode; // idxRightNode[i] = index of right node of i
```



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Key	6	10	8	12	2	4	3								
Left	4	2	-	-	-	6	-								
Right	1	3	-	-	5	-	-								